

Chapter 1: Building Blocks

1. Which of the following are legal entry point methods that can be run from the command line? (Choose all that apply.)
 - A. `private static void main(String[] args)`
 - B. `public static final main(String[] args)`
 - C. `public void main(String[] args)`
 - D. `public static final void main(String[] args)`
 - E. `public static void main(String[] args)`
 - F. `public static main(String[] args)`
2. Which answer options represent the order in which the following statements can be assembled into a program that will compile successfully? (Choose all that apply.)

```
X: class Rabbit {}  
Y: import java.util.*;  
Z: package animals;
```

- A. X, Y, Z
- B. Y, Z, X
- C. Z, Y, X
- D. Y, X
- E. Z, X
- F. X, Z

Explanation: The package and import statements are both optional. If both are present, the order must be package, then import, and then class.

3. Which of the following are true? (Choose all that apply.)

```
public class Bunny {  
    public static void main(String[] x) {  
        Bunny bun = new Bunny();  
    }  
}
```

- A. Bunny is a class.
- B. bun is a class.
- C. main is a class.
- D. Bunny is a reference to an object.
- E. bun is a reference to an object.
- F. main is a reference to an object.
- G. The main() method doesn't run because the parameter name is incorrect.

4. Which of the following are valid Java identifiers? (Choose all that apply.)

- A. `_`
- B. `_helloWorld$`
- C. `true`
- D. `java.lang`
- E. `Public`

F. 1980_s

G. Q2

5. Which statements about the following program are correct? (Choose all that apply.)

```
2: public class Bear {
3:     private Bear pandaBear;
4:     private void roar(Bear b) {
5:         System.out.println("Roar!");
6:         pandaBear = b;
7:     }
8:     public static void main(String[] args) {
9:         Bear brownBear = new Bear();
10:        Bear polarBear = new Bear();
11:        brownBear.roar(polarBear);
12:        polarBear = null;
13:        brownBear = null;
14:        System.gc(); } }
```

- A. The object created on line 9 is first eligible for garbage collection after line 13.
- B. The object created on line 9 is first eligible for garbage collection after line 14.
- C. The object created on line 10 is first eligible for garbage collection after line 12.
- D. The object created on line 10 is first eligible for garbage collection after line 13.
- E. Garbage collection is guaranteed to run.
- F. Garbage collection might or might not run.
- G. The code does not compile.

Explanation: Garbage collection **is never guaranteed to run**, making option F correct and option E incorrect. The Bear object created on line 10 is accessible via both the polarBear reference and the brownBear.pandaBear reference. After line 12, the object is still accessible via brownBear.pandaBear. After line 13, though, it is no longer accessible since brownBear.

6. Assuming the following class compiles, how many variables defined in the class or method are in scope on the line marked on line 14?

```
1: public class Camel {
2:     { int hairs = 3_000_0; }
3:     long water, air=2;
4:     boolean twoHumps = true;
5:     public void spit(float distance) {
6:         var path = "";
7:         { double teeth = 32 + distance++; }
8:         while(water> 0) {
9:             int age = twoHumps ? 1 : 2;
10:            short i=-1;
11:            for(i=0; i<10; i++) {
12:                var Private = 2;
13:            }
14:            // SCOPE
15:        }
16:    }
17: }
```

- A. 2
- B. 3
- C. 4
- D. 5
- E. 6
- F. 7**
- G. None of the above

Explanation: water, air, twoHumps, distance(from method), path, age and i

7. Which are true about this code? (Choose all that apply.)

```
public class KitchenSink {  
    private int numForks;  
  
    public static void main(String[] args) {  
        int numKnives;  
        System.out.print("""  
            "# forks = " + numForks +  
            " # knives = " + numKnives +  
            # cups = 0""");  
    }  
}
```

- A. The output includes # forks = 0.
- B. The output includes # knives = 0.
- C. The output includes # cups = 0.**
- D. The output includes a blank line.
- E. The output includes one or more lines that begin with whitespace.**
- F. The code does not compile.

Explanation: The first thing to recognize is that this is a text block and the code inside the """ is just text. Option E is also an answer since " # knives is indented.

8. Which of the following code snippets about var compile without issue when used in a method? (Choose all that apply.)

- A. var spring = null;
- B. var fall = "leaves";**
- C. var evening = 2; evening = null;
- D. var night = Integer.valueOf(3);**
- E. var day = 1/0;**
- F. var winter = 12, cold;
- G. var fall = 2, autumn = 2;
- H. var morning = ""; morning = null;**

Explanation: Option E is correct as this is a valid numeric expression. You might know that dividing by zero produces a runtime exception, but the question was only about whether the code compiled.

9. Which of the following is correct?

- A. An instance variable of type float defaults to 0.
- B. An instance variable of type char defaults to null.
- C. A local variable of type double defaults to 0.0.
- D. A local variable of type int defaults to null.
- E. A class variable of type String defaults to null.
- F. A class variable of type String defaults to the empty string "".
- G. None of the above.

10. Which of the following expressions, when inserted independently into the blank line, allow the code to compile? (Choose all that apply.)

```
public void printMagicData() {  
    var magic = ;  
    System.out.println(magic);  
}
```

- A. 3_1
- B. 1_329_.0
- C. 3_13.0_
- D. 5_291._2
- E. 2_234.0_0
- F. 9_6
- G. _1_3_5_0

11. Given the following two class files, what is the maximum number of imports that can be removed and have the code still compile?

```
// Water.java  
package aquarium;  
public class Water { }  
  
// Tank.java  
package aquarium;  
import java.lang.*;  
import java.lang.System;  
import aquarium.Water;  
import aquarium.*;  
public class Tank {  
    public void print(Water water) {  
        System.out.println(water); } } }
```

- A. 0
- B. 1
- C. 2
- D. 3
- E. 4
- F. Does not compile

Explanation: The first two imports can be removed because `java.lang` is automatically imported. The following two imports can be removed because Tank and Water are in the same package.

12. Which statements about the following class are correct? (Choose all that apply.)

```
1: public class ClownFish {
2:     int gills = 0, double weight=2;
3:     { int fins = gills; }
4:     void print(int length = 3) {
5:         System.out.println(gills);
6:         System.out.println(weight);
7:         System.out.println(fins);
8:         System.out.println(length);
9: } }
```

- A. Line 2 generates a compiler error.
- B. Line 3 generates a compiler error.
- C. Line 4 generates a compiler error.
- D. Line 7 generates a compiler error.
- E. The code prints 0.
- F. The code prints 2.0.
- G. The code prints 2.
- H. The code prints 3.

Explanation: Line 4 does not compile because `Java does not support setting default method parameter values`, making option C correct.

13. Given the following classes, which of the following snippets can independently be inserted in place of INSERT IMPORTS HERE and have the code compile? (Choose all that apply.)

```
package aquarium;
public class Water {
    boolean salty = false;
}

package aquarium.jellies;
public class Water {
    boolean salty = true;
}

package employee;
INSERT IMPORTS HERE
public class WaterFiller {
    Water water;
}
```

- A. `import aquarium.*;`
- B. `import aquarium.Water;`
- C. `import aquarium.jellies.*;`
- D. `import aquarium.*;`
- E. `import aquarium.jellies.Water;`
- F. `import aquarium.*;`

- G. `import aquarium.jellies.*;`
- H. `import aquarium.Water;`
- I. `import aquarium.jellies.Water;`
- J. None of the above

Explanation: Option A is correct because it imports all the classes in the aquarium package including `aquarium.Water`. Options B and C are correct because they import `Water` by class name. Since importing by class name takes precedence over wildcards, these compile.

14. Which of the following statements about the code snippet are true? (Choose all that apply.)

```
3: short numPets = 5L;  
4: int numGrains = 2.0;  
5: String name = "Scruffy";  
6: int d = numPets.length();  
7: int e = numGrains.length;  
8: int f = name.length();
```

- A. Line 3 generates a compiler error.
- B. Line 4 generates a compiler error.
- C. Line 5 generates a compiler error.
- D. Line 6 generates a compiler error.
- E. Line 7 generates a compiler error.
- F. Line 8 generates a compiler error.

15. Which of the following statements about garbage collection are correct? (Choose all that apply.)

- A. Calling `System.gc()` is guaranteed to free up memory by destroying objects eligible for garbage collection.
- B. Garbage collection runs on a set schedule.
- C. Garbage collection allows the JVM to reclaim memory for other objects.
- D. Garbage collection runs when your program has used up half the available memory.
- E. An object may be eligible for garbage collection but never removed from the heap.
- F. An object is eligible for garbage collection once no references to it are accessible in the program.
- G. Marking a variable `final` means its associated object will never be garbage collected.

Explanation: In Java, there are no guarantees about when garbage collection will run. The JVM is free to ignore calls to `System.gc()`. For this reason, options A, B, and D are incorrect.

16. Which are true about this code? (Choose all that apply.)

```
var blocky = "  
  squirrel \s  
  pigeon  \  
  termite""";  
System.out.print(blocky);
```

- A. It outputs two lines.
- B. It outputs three lines.
- C. It outputs four lines.
- D. There is one line with trailing whitespace.

- E. There are two lines with trailing whitespace.
- F. If we indented each line five characters, it would change the output.

Explanation: Remember that a backslash means to skip the line break. Option D is also correct as `\s` means to keep whitespace. In a text block, incidental indentation is ignored, making option F incorrect.

17. What lines are printed by the following program? (Choose all that apply.)

```
1: public class WaterBottle {
2:     private String brand;
3:     private boolean empty;
4:     public static float code;
5:     public static void main(String[] args) {
6:         WaterBottle wb = new WaterBottle();
7:         System.out.println("Empty = " + wb.empty);
8:         System.out.println("Brand = " + wb.brand);
9:         System.out.println("Code = " + code);
10:    }
```

- A. Line 8 generates a compiler error.
- B. Line 9 generates a compiler error.
- C. Empty =
- D. Empty = false
- E. Brand =
- F. Brand = null
- G. Code = 0.0
- H. Code = 0f

18. Which of the following statements about var are true? (Choose all that apply.)

- A. A var can be used as a constructor parameter.
- B. The type of a var is known at compile time.
- C. A var cannot be used as an instance variable.
- D. A var can be used in a multiple variable assignment statement.
- E. The value of a var cannot change at runtime.
- F. The type of a var cannot change at runtime.
- G. The word var is a reserved word in Java.

19. Which are true about the following code? (Choose all that apply.)

```
var num1 = Integer.parseInt("11");
var num2 = Integer.valueOf("B", 16);
System.out.println(Integer.max(num1, num2));
```

- A. The output is 11.
- B. The output is B.
- C. The code does not compile.
- D. num1 is a primitive.
- E. num2 is a primitive.
- F. A NumberFormatException is thrown.

20. Which statement about the following class is correct?

```
1: public class PoliceBox {
2:     String color;
3:     long age;
4:     public void PoliceBox() {
5:         color = "blue";
6:         age = 1200;
7:     }
8:     public static void main(String []time) {
9:         var p = new PoliceBox();
10:        var q = new PoliceBox();
11:        p.color = "green";
12:        p.age = 1400;
13:        p = q;
14:        System.out.println("Q1="+q.color);
15:        System.out.println("Q2="+q.age);
16:        System.out.println("P1="+p.color);
17:        System.out.println("P2="+p.age);
18: } }
```

- A. It prints Q1=blue.
- B. It prints Q2=1200.
- C. It prints P1=null.
- D. It prints P2=1400.
- E. Line 4 does not compile.
- F. Line 12 does not compile.
- G. Line 13 does not compile.
- H. None of the above.

21. What is the output of executing the following class?

```
1: public class Salmon {
2:     int count;
3:     { System.out.print(count+"-"); }
4:     { count++; }
5:     public Salmon() {
6:         count = 4;
7:         System.out.print(2+"-");
8:     }
9:     public static void main(String[] args) {
10:        System.out.print(7+"-");
11:        var s = new Salmon();
12:        System.out.print(s.count+"-"); } }
```

- A. 7-0-2-1-
- B. 7-0-1-
- C. 0-7-2-1-
- D. 7-0-2-4-
- E. 0-7-1-
- F. The class does not compile because of line 3.

G. The class does not compile because of line 4.

H. None of the above.

Explanation: The default value of an instance variable of type int is 0, so 0- is printed next, and count is assigned a value of 1. **Next, the constructor is called.**

22. Given the following class, which of the following lines of code can independently replace INSERT CODE HERE to make the code compile? (Choose all that apply.)

```
public class Price {  
    public void admission() {  
        INSERT CODE HERE  
        System.out.print(amount);  
    }  
}
```

A. int amount = 0b11;

B. int amount = 9L;

C. int amount = 0xE //14;

D. int amount = 1_2.0;

E. double amount = 1_0_.0;

F. int amount = 0b101 //5;

G. double amount = 9_2.1_2;

H. double amount = 1_2_.0_0;

Explanation: First, 0b is the prefix for a binary value, and 0x is the prefix for a hexadecimal value. These values can be assigned to many primitive types, including int and double, making options C and F correct.

23. Which statements about the following class are true? (Choose all that apply.)

```
1: public class River {  
2:     int Depth = 1;  
3:     float temp = 50.0;  
4:     public void flow() {  
5:         for (int i = 0; i < 1; i++) {  
6:             int depth = 2;  
7:             depth++;  
8:             temp--;  
9:         }  
10:        System.out.println(depth);  
11:        System.out.println(temp); }  
12:    public static void main(String... s) {  
13:        new River().flow();  
14:    } }
```

A. Line 3 generates a compiler error.

B. Line 6 generates a compiler error.

C. Line 7 generates a compiler error.

D. Line 10 generates a compiler error.

E. The program prints 3 on line 10.

F. The program prints 4 on line 10.

- G. The program prints 50.0 on line 11.
- H. The program prints 49.0 on line 11.

Chapter 2: Operators

1. Which of the following Java operators can be used with boolean variables? (Choose all that apply.)

- A. `==`
- B. `+`
- C. `--`
- D. `!`
- E. `%`
- F. `-`
- G. Cast with `(boolean)`

2. What data type (or types) will allow the following code snippet to compile? (Choose all that apply.)

```
byte apples = 5;  
short oranges = 10;  
_____ bananas = apples + oranges;
```

- A. `int`
- B. `long`
- C. `boolean`
- D. `double`
- E. `short`
- F. `byte`

Explanation: The expression `apples + oranges` is automatically promoted to `int`, so `int` and data types that can be promoted automatically from `int` will work, making options A, B, and D correct.

3. What change, when applied independently, would allow the following code snippet to compile? (Choose all that apply.)

```
3: long ear = 10;  
4: int hearing = 2 * ear;
```

- A. No change; it compiles as is.
- B. Cast `ear` on line 4 to `int`.
- C. Change the data type of `ear` on line 3 to `short`.
- D. Cast `2 * ear` on line 4 to `int`.
- E. Change the data type of `hearing` on line 4 to `short`.
- F. Change the data type of `hearing` on line 4 to `long`.

Explanation: The code will not compile as is, so option A is not correct. The value `2 * ear` is automatically promoted to `long` and cannot be automatically stored in `hearing`, which is an `int` value.

4. What is the output of the following code snippet?

```
3: boolean canine = true, wolf = true;  
4: int teeth = 20;  
5: canine = (teeth != 10) ^ (wolf=false);  
6: System.out.println(canine+"", "+teeth+", "+wolf);
```

- A. true, 20, true
- B. true, 20, false
- C. false, 10, true
- D. false, 20, false
- E. The code will not compile because of line 5.
- F. None of the above.

5. Which of the following operators are ranked in increasing or the same order of precedence? Assume the + operator is binary addition, not the unary form. (Choose all that apply.)

- A. +, *, %, --
- B. ++, (int), *
- C. =, ==, !
- D. (short), =, !, *
- E. *, /, %, +, ==
- F. !, ||, &
- G. ^, +, =, +=

Explanation: Options A and C are correct, as they show operators in increasing or the same order of precedence.

6. What is the output of the following program?

```
1: public class CandyCounter {
2:     static long addCandy(double fruit, float vegetables) {
3:         return (int)fruit+vegetables;
4:     }
5:
6:     public static void main(String[] args) {
7:         System.out.print(addCandy(1.4, 2.4f) + ", ");
8:         System.out.print(addCandy(1.9, (float)4) + ", ");
9:         System.out.print(addCandy((long)(int)(short)2, (float)4)); }
}
```

- A. 4, 6, 6.0
- B. 3, 5, 6
- C. 3, 6, 6
- D. 4, 5, 6
- E. The code does not compile because of line 9.
- F. None of the above.

Explanation: The code does not compile because line 3 contains a compilation error. The cast (int) is applied to fruit, not the expression fruit+vegetables. Since the cast operator has a higher operator precedence than the addition operator, it is applied to fruit, but the expression is promoted to a float, due to vegetables being float.

7. What is the output of the following code snippet?

```
int ph = 7, vis = 2;
boolean clear = vis > 1 & (vis < 9 || ph < 2);
boolean safe = (vis > 2) && (ph++ > 1);
boolean tasty = 7 <= --ph;
System.out.println(clear + "-" + safe + "-" + tasty);
```

- A. true-true-true
- B. true-true-false
- C. true-false-true
- D. true-false-false
- E. false-true-true
- F. false-true-false
- G. false-false-true
- H. false-false-false

8. What is the output of the following code snippet?

```
4: int pig = (short)4;
5: pig = pig++;
6: long goat = (int)2;
7: goat -= 1.0;
8: System.out.print(pig + " - " + goat);
```

- A. 4 - 1
- B. 4 - 2
- C. 5 - 1
- D. 5 - 2
- E. The code does not compile due to line 7.
- F. None of the above.

Explanation: Line 5 increments pig by 1, but it returns the original value of 4 since it is using the post-increment operator.

9. What are the unique outputs of the following code snippet? (Choose all that apply.)

```
int a = 2, b = 4, c = 2;
System.out.println(a > 2 ? --c : b++);
System.out.println(b = (a != c ? a : b++));
System.out.println(a > b ? b < c ? b : 2 : 1);
```

- A. 1
- B. 2
- C. 3
- D. 4
- E. 5
- F. 6
- G. The code does not compile.

10. Which is not an output of the following code snippet?

```
short height = 1, weight = 3;
short zebra = (byte) weight * (byte) height;
double ox = 1 + height * 2 + weight;
long giraffe = 1 + 9 % height + 1;
System.out.println(zebra);
System.out.println(ox);
System.out.println(giraffe);
```

- A. 2
- B. 3
- C. 6
- D. 6.0
- E. The code does not compile.

Explanation: The code does not compile due to an error on the second line, making option E correct. Even though both height and weight are cast to byte, the multiplication operator automatically promotes them to int, resulting in an attempt to store an int in a short variable.

11. What is the output of the following code?

```
11: int sample1 = (2 * 4) % 3;
12: int sample2 = 3 * 2 % -3;
13: int sample3 = 5 * (1 % 2);
14: System.out.println(sample1 + ", " + sample2 + ", " + sample3);
```

- A. 0, 0, 5
- B. 1, 2, 10
- C. 2, 1, 5
- D. 2, 0, 5
- E. 3, 1, 10
- F. 3, 2, 6
- G. The code does not compile.

12. The _____ operator increases a value and returns the original value, while the _____ operator decreases a value and returns the new value.

- A. post-increment, post-increment
- B. pre-decrement, post-decrement
- C. post-increment, post-decrement
- D. post-increment, pre-decrement
- E. pre-increment, pre-decrement
- F. pre-increment, post-decrement

13. What is the output of the following code snippet?

```
boolean sunny = true, raining = false, sunday = true;
boolean goingToTheStore = sunny & raining ^ sunday;
boolean goingToTheZoo = sunday && !raining;
boolean stayingHome = !(goingToTheStore && goingToTheZoo);
System.out.println(goingToTheStore + "-" + goingToTheZoo
    + "-" + stayingHome);
```

- A. true-false-false
- B. false-true-false
- C. true-true-true
- D. false-true-true
- E. false-false-false
- F. true-true-false
- G. None of the above

14. Which of the following statements are correct? (Choose all that apply.)

- A. The return value of an assignment operation expression can be void.
- B. The inequality operator (!=) can be used to compare objects.
- C. The equality operator (==) can be used to compare a boolean value with a numeric value.
- D. During runtime, the & and | operators may cause only the left side of the expression to be evaluated.
- E. The return value of an assignment operation expression is the value of the newly assigned variable.
- F. In Java, 0 and false may be used interchangeably.
- G. The logical complement operator (!) cannot be used to flip numeric values.

15. Which operator takes three operands or values?

- A. =
- B. &&
- C. *=
- D. ?:
- E. &
- F. ++
- G. /

16. How many lines of the following code contain compiler errors?

```
int note = 1 * 2 + (long)3;  
short melody = (byte)(double)(note *= 2);  
double song = melody;  
float symphony = (float)((song == 1_000f) ? song * 2L : song);
```

- A. 0
- B. 1
- C. 2
- D. 3
- E. 4

Explanation: The first line contains a compilation error. The value 3 is cast to long. The 1 * 2 value is evaluated as int but promoted to long when added to the 3. Trying to store a long value in an int variable triggers a compiler error.

17. Given the following code snippet, what are the values of the variables after it is executed? (Choose all that apply.)

```
int ticketsTaken = 1;
int ticketsSold = 3;
ticketsSold += 1 + ticketsTaken++;
ticketsTaken *= 2;
ticketsSold += (long)1;
```

- A. ticketsSold is 8.
- B. ticketsTaken is 2.
- C. ticketsSold is 6.
- D. ticketsTaken is 6.
- E. ticketsSold is 7.
- F. ticketsTaken is 4.
- G. The code does not compile.

18. Which of the following can be used to change the order of operation in an expression?

- A. []
- B. <>
- C. ()
- D. \ /
- E. { }
- F. " "

19. What is the result of executing the following code snippet? (Choose all that apply.)

```
3: int start = 7;
4: int end = 4;
5: end += ++start;
6: start = (byte)(Byte.MAX_VALUE + 1);
```

- A. start is 0.
- B. start is -128.
- C. start is 127.
- D. end is 8.
- E. end is 11.
- F. end is 12.
- G. The code does not compile.
- H. The code compiles but throws an exception at runtime.

Explanation: On line 5, the pre-increment operator is executed first, so start is incremented to 8.

20. Which of the following statements about unary operators are true? (Choose all that apply.)

- A. Unary operators are always executed before any surrounding numeric binary or ternary operators.
- B. The - operator can be used to flip a boolean value.
- C. The pre-increment operator (++) returns the value of the variable before the increment is applied.
- D. The post-decrement operator (--) returns the value of the variable before the decrement is applied.
- E. The ! operator cannot be used on numeric values.
- F. None of the above.

Chapter 3: Making Decisions

1. What is the output of the following code snippet?

```
32: Object skips = 10;
33: switch (skips) {
34:     case a when a < 10 -> System.out.print(2);
35:     case b when b >= 10 -> System.out.print(4);
36:     case null -> System.out.print(6);
37:     default -> System.out.print(8);
38: }
```

- A. 2
 - B. 4
 - C. 6
 - D. 8
 - E. Exactly one line does not compile.
 - F. Exactly two lines do not compile.
 - G. None of the above.
2. Which of the following data types can be used in a switch expression? (Choose all that apply.)

- A. enum
- B. int
- C. Byte
- D. long
- E. boolean
- F. double

3. What is the output of the following code snippet?

```
3: int temperature = 4;
4: long humidity = -temperature + temperature * 3;
5: if (temperature >= 4)
6: if (humidity < 6) System.out.println("Too Low");
7: else System.out.println("Just Right");
8: else System.out.println("Too High");
```

- A. Too Low
- B. Just Right
- C. Too High
- D. A NullPointerException is thrown at runtime.
- E. The code will not compile because of line 7.
- F. The code will not compile because of line 8.

4. Which of the following data types are permitted on the right side of a for-each expression? (Choose all that apply.)

A. Double[][]

B. Object

C. Map

D. List

E. String

F. char[]

G. Exception

5. What is the output of calling printReptile(6)?

```
void printReptile(int category) {  
    var type = switch (category) {  
        case 1,2 -> "Snake";  
        case 3,4 -> "Lizard";  
        case 5,6 -> "Turtle";  
        case 7,8 -> "Alligator";  
    };  
    System.out.print(type);  
}
```

A. Snake

B. Lizard

C. Turtle

D. Alligator

E. TurtleAlligator

F. None of the above

6. What is the output of the following code snippet?

```
List<Integer> myFavoriteNumbers = new ArrayList<>();  
myFavoriteNumbers.add(10);  
myFavoriteNumbers.add(14);  
for (var a : myFavoriteNumbers) {  
    System.out.print(a + ", ");  
    break;  
}  
  
for (int b : myFavoriteNumbers) {  
    continue;  
    System.out.print(b + ", ");  
}  
  
for (Object c : myFavoriteNumbers)  
    System.out.print(c + ", ");
```

A. It compiles and runs without issue but does not produce any output.

- B. 10, 14,
- C. 10, 10, 14,
- D. 10, 10, 14, 10, 14,
- E. Exactly one line of code does not compile.
- F. Exactly two lines of code do not compile.
- G. Three or more lines of code do not compile.
- H. The code contains an infinite loop and does not terminate.

Explanation: The second for-each loop contains a continue followed by a print() statement. Because the continue is not conditional and always included as part of the body of the for-each loop, the print() statement is not reachable.

7. Assuming weather is a well-formed nonempty array, which code snippet, when inserted independently into the blank in the following code, prints all of the elements of weather? (Choose all that apply.)

```
private void print(int[] weather) {
    for () {
        System.out.println(weather[i]);
    }
}
```

- A. int i=weather.length; i>0; i--
- B. int i=0; i<=weather.length-1; ++i
- C. var w : weather
- D. int i=weather.length-1; i>=0; i--
- E. int i=0, int j=3; i<weather.length; ++i
- F. int i=0; ++i<10 && i<weather.length;
- G. None of the above

8. What is the output of calling printType(11)?

```
31: void printType(Object o) {
32:     if (o instanceof Integer bat) {
33:         System.out.print("int");
34:     } else if (o instanceof Integer bat && bat < 10) {
35:         System.out.print("small int");
36:     } else if (o instanceof Long bat || bat <= 20) {
37:         System.out.print("long");
38:     } default {
39:         System.out.print("unknown");
40:     }
41: }
```

- A. int
- B. small int
- C. long
- D. unknown
- E. Nothing is printed.
- F. The code contains one line that does not compile.
- G. The code contains two lines that do not compile.

H. None of the above.

Explanation: Line 36 does not compile, though. Due to flow scoping, if o is not a Long, then bat is not in scope in the expression bat <= 20. Line 38 also does not compile as default cannot be used as part of an if/else statement. For these two reasons, option G is correct.

9. Which statements, when inserted independently into the following blank, will cause the code to print 2 at runtime? (Choose all that apply.)

```
int count = 0;
BUNNY: for (int row = 1; row <=3; row++)
    RABBIT: for (int col = 0; col <3 ; col++) {
        if ((col + row) % 2 == 0)
            ;
        count++;
    }
System.out.println(count);
```

- A. break BUNNY
- B. break RABBIT
- C. continue BUNNY
- D. continue RABBIT
- E. break
- F. continue
- G. None of the above, as the code contains a compiler error

Explanation: The code uses nested loops and a condition that runs when row + col is even; otherwise, count increases. Options E and F are equivalent to B and D since unlabeled statements apply to the innermost loop. The first true condition occurs on the second inner-loop iteration (row = 1, col = 1). Option A is wrong because it exits the loop with count = 1. In options B, C, and E, count becomes 1 in the first inner loop, exits, then increments to 2 in the third outer loop iteration before the program ends, making these options correct.

10. Given the following method, how many lines contain compilation errors?

```
8:  enum DayOfWeek {
9:      SUNDAY, MONDAY, TUESDAY, WEDNESDAY, THURSDAY, FRIDAY, SATURDAY;
10:     private DayOfWeek getWeekDay(int day, final int thursday) {
11:         int otherDay = day;
12:         int Sunday = 0;
13:         switch (otherDay) {
14:             default:
15:                 case 1: continue;
16:                 case thursday: return DayOfWeek.THURSDAY;
17:                 case 2,10: break;
18:                 case Sunday: return DayOfWeek.SUNDAY;
19:                 case DayOfWeek.MONDAY: return DayOfWeek.MONDAY;
20:         }
21:         return DayOfWeek.FRIDAY;
22:     } }
```

- A. None, the code compiles and runs without issue.

- B. 1.
- C. 2.
- D. 3.
- E. 4.
- F. 5.
- G. 6.
- H. The code compiles but may produce an error at runtime.

11. What is the output of calling `printLocation(Animal.MAMMAL)`?

```
10: class Zoo {  
11:     enum Animal {BIRD, FISH, MAMMAL}  
12:     void printLocation(Animal a) {  
13:         long type = switch (a) {  
14:             case BIRD -> 1;  
15:             case FISH -> 2;  
16:             case MAMMAL -> 3;  
17:             default -> 4;  
18:         };  
19:         System.out.print(type);  
20:     } }
```

- A. 3
- B. 4
- C. 34
- D. The code does not compile because of line 13.
- E. The code does not compile because of line 17.
- F. None of the above.

12. What is the result of the following code snippet?

```
3: int sing = 8, squawk = 2, notes = 0;  
4: while (sing > squawk) {  
5:     sing--;  
6:     squawk += 2;  
7:     notes += sing + squawk;  
8: }  
9: System.out.println(notes);
```

- A. 11
- B. 13
- C. 23
- D. 33
- E. 50
- F. The code will not compile because of line 7.

13. What is the result of calling `getHatSize(9f)` on the following code snippet?

```
10: int getHatSize(Number measurement) {  
11:     return switch (measurement) {  
12:         case Double d -> 1 + d.intValue();  
13:         case null      -> 11;  
14:         case !(Number n) -> 3 + n.intValue();  
15:         case Float f when f < 10 -> 4 + f.intValue();  
16:     };  
17: }
```

- A. 10
- B. 11
- C. 12
- D. 13
- E. The code does not compile because it is missing a default clause.
- F. The code does not compile for a different reason.

Explanation: The code does not compile because case clause uses the logical complement operator (!), which is not permitted with pattern matching.

14. What is the output of the following code snippet?

```
2: boolean keepGoing = true;  
3: int result = 15, meters = 10;  
4: do {  
5:     meters--;  
6:     if (meters==8) keepGoing = false;  
7:     result -= 2;  
8: } while keepGoing;  
9: System.out.println(result);
```

- A. 7
- B. 9
- C. 10
- D. 11
- E. 15
- F. The code will not compile because of line 6.
- G. The code does not compile for a different reason.

Explanation: Line 8 is missing the required parentheses around the boolean conditional expression.

15. Which statements about the following code snippet are correct? (Choose all that apply.)

```
for (var penguin : new int[2])  
    System.out.println(penguin);  
  
var ostrich = new Character[3];  
for (var emu : ostrich)  
    System.out.println(emu);  
  
List<Integer> parrots = new ArrayList<Integer>();  
for (var macaw : parrots)
```

```
System.out.println(macaw);
```

- A. The data type of penguin is Integer.
- B. The data type of penguin is int.
- C. The data type of emu is undefined.
- D. The data type of emu is Character.
- E. The data type of macaw is List.
- F. The data type of macaw is Integer.
- G. None of the above, as the code does not compile.

16. What is the result of the following code snippet?

```
final char a = 'A', e = 'E';
char grade = 'B';
switch (grade) {
    default:
    case a:
    case 'B': 'C': System.out.print("great ");
    case 'D':      System.out.print("good "); break;
    case e:
    case 'F':      System.out.print("not good ");
}
```

- A. great
- B. great good
- C. good
- D. not good
- E. The code does not compile because the data type of one or more case clauses does not match the data type of the switch variable.
- F. None of the above.

Explanation: The code does not compile, although not for the reason specified in option E. The second case clause contains invalid syntax. This line could have been fixed to say case 'B', 'C' or by adding the case keyword before 'C'.

17. Given the following array, which code snippets print the elements in reverse order from how they are declared? (Choose all that apply.)

A.

```
char[] wolf = {'W', 'e', 'b', 'b', 'y'};
int q = wolf.length;
for ( ; ; ) {
    System.out.print(wolf[--q]);
    if (q==0) break;
}
```

B.

```
for (int m=wolf.length-1; m>=0; --m)
    System.out.print(wolf[m]);
```

C.

```
for (int z=0; z<wolf.length; z++)  
    System.out.print(wolf[wolf.length-z]);
```

D.

```
int x = wolf.length-1;  
for (int j=0; x>=0 && j==0; x--)  
    System.out.print(wolf[x]);
```

E.

```
final int r = wolf.length;  
for (int w = r-1; r>-1; w = r-1)  
    System.out.print(wolf[w]);
```

F.

```
for (int i=wolf.length; i>0; --i)  
    System.out.print(wolf[i]);
```

G. None of the above

18. What distinct numbers are printed when the following method is executed? (Choose all that apply.)

```
private void countAttendees() {  
    int participants = 4, animals = 2, performers = -1;  
  
    while ((participants = participants + 1) < 10) {}  
    do {} while (animals++ <= 1);  
    for ( ; performers < 2; performers += 2) {}  
  
    System.out.println(participants);  
    System.out.println(animals);  
    System.out.println(performers);  
}
```

A. 6

B. 3

C. 4

D. 5

E. 10

F. 9

G. The code does not compile.

H. None of the above.

19. What is the output of the following code snippet?

```
2: double iguana = 0;  
3: do {  
4:     int snake = 1;  
5:     System.out.print(snake++ + " ");  
6:     iguana--;  
7: } while (snake <= 5);  
8: System.out.println(iguana);
```


- A. 1 2 3 4 -4.0
- B. 1 2 3 4 -5.0
- C. 1 2 3 4 5 -4.0
- D. 0 1 2 3 4 5 -5.0
- E. The code does not compile.
- F. The code compiles but produces an infinite loop at runtime.
- G. None of the above.

Explanation: The variable snake is declared within the body of the do/while statement, so it is out of scope on line 7

20. Which statements, when inserted into the following blanks, allow the code to compile and run without entering an infinite loop? (Choose all that apply.)

```

4:  int height = 1;
5:  L1: while (height++ <10) {
6:      long humidity = 12;
7:      L2: do {
8:          if (humidity-- % 12 == 0) ;
9:          int temperature = 30;
10:         L3: for ( ; ; ) {
11:             temperature++;
12:             if (temperature>50) ;
13:         }
14:     } while (humidity> 4);
15: }

```

- A. break L2 on line 8; continue L2 on line 12
- B. continue on line 8; continue on line 12
- C. break L3 on line 8; break L1 on line 12
- D. continue L2 on line 8; continue L3 on line 12
- E. continue L2 on line 8; continue L2 on line 12
- F. None of the above, as the code contains a compiler error

Explanation: The key point is that the innermost loop is infinite, so correct solutions must skip or exit it. Option A works because break L2 exits the second inner loop, avoiding the infinite loop. Option B skips the innermost loop only once, then runs it indefinitely, causing an infinite loop. Option C fails due to a compiler error — L3 is out of scope. Option D behaves like B, also looping infinitely. Option E is correct since continue L2 on line 12 exits the infinite loop, allowing the program to finish.

21. A minimum of how many lines need to be corrected before the following method will compile?

```

21: void findZookeeper(Integer id) {
22:     System.out.print(switch (id) {
23:         case 10 -> {"Jane";}
24:         case 20 -> {yield "Lisa";};
25:         case 30 -> "Kelly";
26:         case 30 -> "Sarah";
27:         default -> "Unassigned";
28:     });
29: }

```

- A. Zero
- B. One
- C. Two
- D. Three**
- E. Four
- F. Five

Explanation: Line 23 does not compile because it is missing a yield statement. Line 24 does not compile because it contains an extra semicolon at the end. Finally, lines 25 and 26 do not compile because they use the same case value. At least one of them would need to be changed for the code to compile.

22. What is the output of the following code snippet?

```
2: var tailFeathers = 3;
3: final var one = 1;
4: switch (tailFeathers) {
5:     case one: System.out.print(3 + " ");
6:     default: case 3: System.out.print(5 + " ");
7: }
8: while (tailFeathers > 1) {
9:     System.out.print(--tailFeathers + " "); }
```

- A. 3
- B. 5 1
- C. 5 2
- D. 3 5 1
- E. 5 2 1**
- F. The code will not compile because of lines 3–5.
- G. The code will not compile because of line 6.

23. What is the output of the following code snippet?

```
15: int penguin = 50, turtle = 75;
16: boolean older = penguin >= turtle;
17: if (older = true) System.out.println("Success");
18: else System.out.println("Failure");
19: else if (penguin != 50) System.out.println("Other");
```

- A. Success
- B. Failure
- C. Other
- D. The code will not compile because of line 17.
- E. The code compiles but throws an exception at runtime.
- F. None of the above.**

Explanation: Line 19 starts with an else statement, but there is no preceding if statement that it matches. For this reason, line 19 does not compile, making option F the correct answer.

24. What is the output of the following code snippet?

```
22: String zooStatus = "Closed";
23: int visitors = switch (zooStatus) {
24:     case String s when s.equals("Open") -> 10;
25:     case Object s when s != null && !s.equals("") -> 20;
26:     case null -> {yield 30;}
27:     default -> 40;
28: };
29: System.out.print(visitors);
```

- A. 10
- B. 20**
- C. 30
- D. 40
- E. Exactly one line does not compile.
- F. Exactly two lines do not compile.
- G. Three or more lines do not compile.

25. What is the output of the following code snippet?

```
6: String instrument = "violin";
7: final String CELLO = "cello";
8: String viola = "viola";
9: int p = -1;
10: switch (instrument) {
11:     case "bass" : break;
12:     case CELLO : p++;
13:     default: p++;
14:     case "VIOLIN": p++;
15:     case "viola" : ++p; break;
16: }
17: System.out.print(p);
```

- A. -1
- B. 0
- C. 1
- D. 2**
- E. 3
- F. The code does not compile.

Explanation: This execution path increments p a total of three times, bringing the final value of p to 2 and making option D the correct answer.

26. What is the output of the following code snippet?

```
9: int w = 0, r = 1;
10: String name = "";
11: while (w < 2) {
12:     name += "A";
```

```

13:    do {
14:        name += "B";
15:        if (name.length() > 0) name += "C";
16:        else break;
17:    } while (r <= 1);
18:    r++; w++; }
19: System.out.println(name);

```

- A. ABC
- B. ABCABC
- C. ABCABCABC
- D. Line 15 contains a compilation error.
- E. Line 18 contains a compilation error.
- F. The code compiles but never terminates at runtime.
- G. The code compiles but throws a NullPointerException at runtime.

Explanation: While it may seem complicated, the key is to notice that the variable `r` is updated outside of the `do/while` loop. This is allowed from a compilation standpoint, since it is defined before the loop, but it means the innermost loop never breaks the termination condition `r <= 1`. At runtime, this will produce an infinite loop the first time the innermost loop is entered, making option F the correct answer.

27. What is printed by the following code snippet?

```

23: byte amphibian = 2;
24: String name = "Salamander";
25: String color = switch (amphibian) {
26:     case 1 -> { yield "Red"; }
27:     case 2 -> { if (name.equals("Frog")) yield "Green";
28:                 yield "Blue"; }
29:     case 3 -> { yield "Purple"; }
30:     default -> throw new RuntimeException();
31: };
32: System.out.print(color);

```

- A. Red
- B. Green
- C. Purple
- D. Blue
- E. The code does not compile.
- F. An exception is thrown at runtime.

28. What is the output of calling `getFish("goldie")`?

```

40: void getFish(Object fish) {
41:     if (!(fish instanceof String guppy))
42:         System.out.print("Eat!");
43:     else if (!(fish instanceof String guppy)) {
44:         throw new RuntimeException();
45:     }
46:     System.out.print("Swim!");
47: }

```

- A. Eat!
- B. Swim!
- C. Eat! followed by an exception
- D. Eat!Swim!
- E. An exception is printed
- F. None of the above

Explanation: Based on flow scoping, guppy is in scope after lines 41–42 if the type is not a String. In this case, line 43 declares a variable guppy that is a duplicate of the previously defined local variable defined on line 41. For this reason, the code does not compile, and option F is correct.

29. What is the result of the following code?

```
1: public class PrintIntegers {
2:     public static void main(String[] args) {
3:         int y = -2;
4:         do System.out.print(++y + " ");
5:         while (y <= 5);
6:     } }
```

- A. -2 -1 0 1 2 3 4 5
- B. -2 -1 0 1 2 3 4
- C. -1 0 1 2 3 4 5 6
- D. -1 0 1 2 3 4 5
- E. The code will not compile because of line 5.
- F. The code contains an infinite loop and does not terminate.

30. What is the minimum number of lines that would need to be changed or removed for the following code to compile and return a value when called with dance(10)?

```
41: double dance(Object speed) {
42:     return switch (speed) {
43:         case 5 -> {yield 4};
44:         case 10 -> 8;
45:         case 15,20 -> 12;
46:         default -> 20;
47:         case null -> 16;
48:     }
49: }
```

- A. Zero, the code compiles and runs without issue
- B. One
- C. Two
- D. Three
- E. Four
- F. Five
- G. Six

Explanation: Line 43 needs the semicolon after yield, and line 48 needs one after the return statement. Lines 43–45 fail to compile because numeric values aren't compatible with Object; changing line 41 to use Integer fixes this. Also, default on line 46 makes case null on line 47 unreachable, so it should be removed. With these four fixes, the code compiles, making option E correct.

Chapter 4: Core APIs

1. What is output by the following code?

```
1: public class Fish {
2:     public static void main(String[] args) {
3:         int numFish = 4;
4:         String fishType = "tuna";
5:         String anotherFish = numFish + 1;
6:         System.out.println(anotherFish + " " + fishType);
7:         System.out.println(numFish + " " + 1);
8:     } }
```

- A. 4 1
- B. 5
- C. 5 tuna
- D. 5tuna
- E. 51tuna
- F. The code does not compile.

Explanation: The problem is that we can't store an int in a String variable (Line 5)

2. Which of these array declarations are not legal? (Choose all that apply.)

- A. `int[][] scores = new int[5][];`
- B. `Object[][][] cubbies = new Object[3][0][5];`
- C. `String beans[] = new beans[6];`
- D. `java.util.Date[] dates[] = new java.util.Date[2][];`
- E. `int[][] types = new int[];`
- F. `int[][] java = new int[][];`

3. Note that March 12, 2028, is the weekend when we spring forward, and November 5, 2028, is when we fall back for daylight saving time. Which of the following can fill in the blank without the code throwing an exception? (Choose all that apply.)

```
var zone = ZoneId.of("US/Eastern");
var date = _____;
var time = LocalTime.of(2, 15);
var z = ZonedDateTime.of(date, time, zone);
```

- A. `LocalDate.of(2028, 3, 12)`
- B. `LocalDate.of(2028, 3, 40)`
- C. `LocalDate.of(2028, 11, 5)`
- D. `LocalDate.of(2028, 11, 6)`
- E. `LocalDate.of(2029, 2, 29)`
- F. `LocalDate.of(2028, MonthEnum.MARCH, 12);`

Explanation: Option B throws an exception because there is no March 40. Option E also throws an exception because 2029 isn't a leap year and therefore has no February 29. Option F doesn't compile because the enum should be named Month, rather than MonthEnum.

4. Which of the following are output by this code? (Choose all that apply.)

```
3: var s = "Hello";
4: var t = new String(s);
5: if ("Hello".equals(s)) System.out.println("one");
6: if (t == s) System.out.println("two");
7: if (t.intern() == s) System.out.println("three");
8: if ("Hello" == s) System.out.println("four");
9: if ("Hello".intern() == t) System.out.println("five");
```

- A. one
- B. two
- C. three
- D. four
- E. five
- F. The code does not compile.
- G. None of the above.

5. What is the result of the following code?

```
7: var sb = new StringBuilder();
8: sb.append("aaa").insert(1, "bb").insert(4, "ccc");
9: System.out.println(sb);
```

- A. abbaaccc
- B. abbaccca
- C. bbaaaccc
- D. bbaaccca
- E. An empty line.
- F. The code does not compile.

6. How many of these lines contain a compiler error?

```
23: double one = Math.pow(1, 2);
24: int two = Math.round(1.0);
25: float three = Math.random();
26: var doubles = new double[] {one, two, three};
```

- A. 0
- B. 1
- C. 2
- D. 3
- E. 4

Explanation: The round() method returns an int when called with a float. However, we are calling it with a double, so it returns a long. The random() method returns a double. Since two lines have a compiler error, option C is the answer.

7. Which of these statements is true of the two values? (Choose all that apply.)

```
2025-08-28T05:00 GMT-04:00  
2025-08-28T09:00 GMT-06:00
```

- A. The first date/time is earlier.
- B. The second date/time is earlier.
- C. Both date/times are the same.
- D. The date/times are two hours apart.
- E. The date/times are six hours apart.
- F. The date/times are 10 hours apart.

Explanation: The first date/time is 9:00 GMT, and the second is 15:00 GMT. Therefore, the first one is earlier by six hours.

8. Which of the following return 5 when run independently? (Choose all that apply.)

```
var string = "12345";  
var builder = new StringBuilder("12345");
```

- A. builder.charAt(4)
- B. builder.replace(2, 4, "6").charAt(3)
- C. builder.replace(2, 5, "6").charAt(2)
- D. string.charAt(5)
- E. string.length
- F. string.replace("123", "1").charAt(2)
- G. None of the above

9. Which of the following are true about arrays? (Choose all that apply.)

- A. The first element is index 0.
- B. The first element is index 1.
- C. Arrays are fixed size.
- D. Arrays are immutable.
- E. Calling equals() on two different arrays containing the same primitive values always returns true.
- F. Calling equals() on two different arrays containing the same primitive values always returns false.
- G. Calling equals() on two different arrays containing the same primitive values can return true or false.

Explanation: An array does not override equals(), so it uses object equality. Since two different objects are not equal, option F is correct, and options E and G are incorrect.

10. How many of these lines contain a compiler error?

```
23: int one = Math.min(5, 3);  
24: long two = Math.round(5.5);  
25: double three = Math.floor(6.6);  
26: var doubles = new double[] {one, two, three};
```

A. 0

B. 1

C. 2

D. 3

E. 4

11. What is the output of the following code?

```
var date = LocalDate.of(2025, 4, 3);  
date.plusDays(2);  
date.plusHours(3);  
System.out.println(date.getYear() + " " + date.getMonth()  
    + " " + date.getDayOfMonth());
```

A. 2025 MARCH 4

B. 2025 MARCH 6

C. 2025 APRIL 3

D. 2025 APRIL 5

E. The code does not compile.

F. A runtime exception is thrown.

Explanation: A `LocalDate` does not have a time element. Therefore, there is no method to add hours, making option E the answer.

12. What is output by the following code ignoring any new lines in the output? (Choose all that apply.)

```
var numbers = "012345678".indent(1);  
numbers = numbers.stripLeading();  
System.out.println(numbers.substring(1, 3));  
System.out.println(numbers.substring(7, 7));  
System.out.println(numbers.substring(7));
```

A. 12

B. 123

C. 7

D. 78

E. A blank line.

F. An exception is thrown.

13. What is the result of the following code?

```
public class Lion {  
    public void roar(String roar1, StringBuilder roar2) {  
        roar1.concat("!!!");  
        roar2.append("!!!");  
    }  
    public static void main(String[] args) {  
        var roar1 = "roar";  
        var roar2 = new StringBuilder("roar");  
        new Lion().roar(roar1, roar2);  
        System.out.println(roar1 + " " + roar2);  
    }  
}
```

- A. roar roar
- B. roar roar!!!
- C. roar!!! Roar
- D. roar!!! Roar!!!
- E. An exception is thrown.
- F. The code does not compile.

Explanation: String is immutable. Calling concat() returns a new String but does not change the original. A StringBuilder is mutable. Calling append() adds characters to the existing character sequence along with returning a reference to the same object.

14. Given the following, which can correctly fill in the blank allowing the code to compile? (Choose all that apply.)

```
var date = LocalDate.now();
var time = LocalTime.now();
var dateTime = date._____(time);
var zoneId = ZoneId.systemDefault();
var zonedDateTime = ZonedDateTime.of(dateTime, zoneId);
Instant instant = _____;
```

- A. asTime()
- B. atTime()
- C. withTime()
- D. dateTime.toInstant()
- E. new Instant()
- F. zonedDateTime.toInstant()

15. What is the output of the following? (Choose all that apply.)

```
var arr = new String[] { "PIG", "pig", "123"};
Arrays.sort(arr);
System.out.println(Arrays.toString(arr));
System.out.println(Arrays.binarySearch(arr, "Pippa"));
```

- A. [pig, PIG, 123]
- B. [PIG, pig, 123]
- C. [123, PIG, pig]
- D. [123, pig, PIG]
- E. -3
- F. -2
- G. The results of binarySearch() are undefined in this example.

Explanation: Numbers sort before letters, and uppercase sorts before lowercase. This makes option C one of the answers. The binarySearch() method looks at where a value would be inserted, which is before the second element for Pippa. It then negates it and subtracts one, which is option E.

16. Which of these statements are true? (Choose all that apply.)

```
var letters = new StringBuilder("abcdefg");
```

- A. letters.substring(1, 2) returns a single-character String.
- B. letters.substring(2, 2) returns a single-character String.
- C. letters.substring(6, 5) returns a single-character String.
- D. letters.substring(6, 6) returns a single-character String.
- E. letters.substring(1, 2) throws an exception.
- F. letters.substring(2, 2) throws an exception.
- G. letters.substring(6, 5) throws an exception.
- H. letters.substring(6, 6) throws an exception.

17. What is the result of the following code? (Choose all that apply.)

```
13: String s1 = ""
14:     purr"";
15: String s2 = "";
16:
17: s1.toUpperCase();
18: s1.trim();
19: s1.substring(1, 3);
20: s1 += "two";
21:
22: s2 += 2;
23: s2 += 'c';
24: s2 += false;
25:
26: if ( s2 == "2cfalse") System.out.println("==");
27: if ( s2.equals("2cfalse")) System.out.println("equals");
28: System.out.println(s1.length());
```

- A. 2
- B. 4
- C. 7
- D. 10
- E. ==
- F. equals
- G. An exception is thrown.
- H. The code does not compile.

18. Which of the following fill in the blank to print a positive integer? (Choose all that apply.)

```
String[] s1 = { "Camel", "Peacock", "Llama"};
String[] s2 = { "Camel", "Llama", "Peacock"};
String[] s3 = { "Camel"};
String[] s4 = { "Camel", null};
System.out.println(Arrays._____);
```

- A. `compare(s1, s2)`
- B. `mismatch(s1, s2)`
- C. `compare(s3, s4)`
- D. `mismatch(s3, s4)`
- E. `compare(s4, s4)`
- F. `mismatch(s4, s4)`

Explanation: The `compare()` method returns a positive integer when the arrays are different and the first is larger. The `mismatch()` method returns a positive integer when the arrays are different in a position index 1 or greater.

19. Note that March 12, 2028 is the weekend that clocks spring ahead for daylight saving time. What is the output of the following? (Choose all that apply.)

```
var date = LocalDate.of(2028, Month.MARCH, 12);
var time = LocalTime.of(1, 30);
var zone = ZoneId.of("US/Eastern");
var dateTime1 = ZonedDateTime.of(date, time, zone);
var dateTime2 = dateTime1.plus(1, ChronoUnit.HOURS);

long diff = ChronoUnit.HOURS.between(dateTime1, dateTime2);
int hour = dateTime2.getHour();
boolean offset = dateTime1.getOffset()
    == dateTime2.getOffset();
System.out.println("diff = " + diff);
System.out.println("hour = " + hour);
System.out.println("offset = " + offset);
```

- A. `diff = 1`
- B. `diff = 2`
- C. `hour = 2`
- D. `hour = 3`
- E. `offset = true`
- F. The code does not compile.
- G. A runtime exception is thrown.

Explanation: The `dateTime1` object has a time of 1:30 per initialization. The `dateTime2` object is an hour later. However, there is no 2:30 when springing ahead, setting the time to 3:30. Option A is correct because it is an hour later. Option D is also correct because the hour of the new time is 3. Option E is not correct because we have changed the time zone offset due to daylight saving time.

20. Which of the following can fill in the blank to print `avaJ`? (Choose all that apply.)

```
3: var puzzle = new StringBuilder("Java");
4: puzzle._____;
5: System.out.println(puzzle);
```

- A. `reverse()`
- B. `append("vaJ$").substring(0, 4)`
- C. `append("vaJ$").delete(0, 3).deleteCharAt(puzzle.length() - 1)`
- D. `append("vaJ$").delete(0, 3).deleteCharAt(puzzle.length())`

E. None of the above

Explanation: Option C uses method chaining. First, it creates the value "JavavaJ\$". Then, it removes the first three characters, resulting in "avaJ\$". Finally, it removes the last character, resulting in "avaJ".

21. What is the output of the following code?

```
var date = LocalDate.of(2025, Month.APRIL, 30);  
date.plusDays(2);  
date.plusYears(3);  
System.out.println(date.getYear() + " " + date.getMonth()  
    + " " + date.getDayOfMonth());
```

- A. 2025 APRIL 30
- B. 2025 MAY 2
- C. 2028 APRIL 2
- D. 2028 APRIL 30
- E. 2028 MAY 2
- F. The code does not compile.
- G. A runtime exception is thrown.

Explanation: The date starts out as April 30, 2025. Since dates are immutable and the plus methods' return values are ignored, the result is unchanged. Therefore, option A is correct.

22. What is the output of the following?

```
var result = LocalDate.of(2025, Month.OCTOBER, 31)  
    .plusYears(1)  
    .plusMonths(-5)  
    .plusMonths(1)  
    .withYear(2026)  
    .atTime(LocalTime.of(13, 4));  
System.out.println(result);
```

- A. 2025-06-30T13:04
- B. 2026-04-304
- C. 2026-04-30T13:04
- D. 2026-06-30T
- E. 2026-06-30T13:04
- F. The code does not compile.
- G. A runtime exception is thrown.

Chapter 5: Methods

1. Which statements about the final modifier are correct? (Choose all that apply.)

- A. Instance and static variables can be marked final.
- B. A variable is effectively final only if it is marked final.
- C. An object that is marked final cannot be modified.
- D. Local variables cannot be declared with type var and the final modifier.
- E. A primitive that is marked final cannot be modified.

2. Which of the following can fill in the blank in this code to make it compile? (Choose all that apply.)

```
public class Ant {  
    _____ void method() {}  
}
```

- A. default
- B. final
- C. private
- D. Public
- E. String
- F. zzz:

3. Which of the following methods compile? (Choose all that apply.)

- A. final static void rain() {}
- B. public final int void snow() {}
- C. private void int hail() {}
- D. static final void sleet() {}
- E. void final ice() {}
- F. void public slush() {}

4. Which of the following can fill in the blank and allow the code to compile? (Choose all that apply.)

```
final _____ song = 6;
```

- A. int
- B. Integer
- C. long
- D. Long
- E. double
- F. Double

Explanation: The value 6 can be implicitly promoted to any of the primitive types, making options A, C, and E correct. It can also be autoboxed to Integer, making option B correct. It cannot be both promoted and autoboxed, making options D and F incorrect.

5. Which of the following methods compile? (Choose all that apply.)

- A. `public void january() { return; }`
- B. `public int february() { return null; }`
- C. `public void march() {}`
- D. `public int april() { return 9; }`
- E. `public int may() { return 9.0; }`
- F. `public int june() { return; }`

Explanation: Option E does not compile because it tries to return a double when the return type is int. Since a double cannot be assigned to an int, it cannot be returned as one either. Option F does not compile because no value is actually returned.

6. Which of the following methods compile? (Choose all that apply.)

- A. `public void violin(int... nums) {}`
- B. `public void viola(String values, int... nums) {}`
- C. `public void cello(int... nums, String values) {}`
- D. `public void bass(String... values, int... nums) {}`
- E. `public void flute(String[] values, ...int nums) {}`
- F. `public void oboe(String[] values, int[] nums) {}`

7. Given the following method, which of the method calls return 2? (Choose all that apply.)

```
public int juggle(boolean b, boolean... b2) {  
    return b2.length;  
}
```

- A. `juggle();`
- B. `juggle(true);`
- C. `juggle(true, true);`
- D. `juggle(true, true, true);`
- E. `juggle(true, {true, true});`
- F. `juggle(true, new boolean[2]);`

8. Which of the following statements is correct?

- A. Package access is more lenient than protected access.
- B. A public class that has private fields and package methods is not visible to classes outside the package.
- C. You can use access modifiers so only some of the classes in a package see a particular package class.
- D. You can use access modifiers to allow access to all methods and not any instance variables.
- E. You can use access modifiers to restrict access to all classes that begin with the word Test.

9. Given the following class definitions, which lines in the main() method generate a compiler error?
(Choose all that apply.)

```
// Classroom.java
package my.school;
public class Classroom {
    private int roomNumber;
    protected static String teacherName;
    static int globalKey = 54321;
    public static int floor = 3;
    Classroom(int r, String t) {
        roomNumber = r;
        teacherName = t; } }

// School.java
1: package my.city;
2: import my.school.*;
3: public class School {
4:     public static void main(String[] args) {
5:         System.out.println(Classroom.globalKey);
6:         Classroom room = new Classroom(101, "Mrs. Anderson");
7:         System.out.println(room.roomNumber);
8:         System.out.println(Classroom.floor);
9:         System.out.println(Classroom.teacherName); } }
```

A. None: the code compiles fine.

B. Line 5.

C. Line 6.

D. Line 7.

E. Line 8.

F. Line 9

10. What is the output of executing the Chimp program?

```
// Rope.java
1: package rope;
2: public class Rope {
3:     public static int LENGTH = 5;
4:     static {
5:         LENGTH = 10;
6:     }
7:     public static void swing() {
8:         System.out.print("swing ");
9:     } }

// Chimp.java
1: import rope.*;
2: import static rope.Rope.*;
3: public class Chimp {
4:     public static void main(String[] args) {
5:         Rope.swing();
6:         new Rope().swing(); }
```

```
7:      System.out.println(LENGTH) ;
8:    } }
```

- A. swing swing 5
- B. swing swing 10
- C. Compiler error on line 2 of Chimp
- D. Compiler error on line 5 of Chimp
- E. Compiler error on line 6 of Chimp
- F. Compiler error on line 7 of Chimp

11. Which statements are true of the following code? (Choose all that apply.)

```
1: public class Rope {
2:     public static void swing() {
3:         System.out.print("swing");
4:     }
5:     public void climb() {
6:         System.out.println("climb");
7:     }
8:     public static void play() {
9:         swing();
10:        climb();
11:    }
12:    public static void main(String[] args) {
13:        Rope rope = new Rope();
14:        rope.play();
15:        Rope rope2 = null;
16:        System.out.print("-");
17:        rope2.play();
18:    } }
```

- A. The code compiles as is.
- B. There is exactly one compiler error in the code.
- C. There are exactly two compiler errors in the code.
- D. If the line(s) with compiler errors are removed, the output is swing-climb.
- E. If the line(s) with compiler errors are removed, the output is swing-swing.
- F. If the line(s) with compile errors are removed, the code throws a NullPointerException

12. How many variables in the following method are effectively final?

```
10: public void feed() {
11:     int monkey = 0;
12:     if(monkey > 0) {
13:         var giraffe = monkey++;
14:         String name;
15:         name = "geoffrey";
16:     }
17:     String name = "milly";
18:     var food = 10;
19:     while(monkey <= 10) {
20:         food = 0;
```

```
21:    }
22:    name = null;
23: }
```

- A. 1.
- B. 2.
- C. 3.
- D. 4.
- E. 5.
- F. None of the above. The code does not compile.

Explanation: The test for effectively final is if the final modifier can be added to the local variable and the code still compiles. The monkey variable declared on line 11 is not effectively final because it is modified on line 13. The giraffe and name variables declared on lines 13 and 14, respectively, are effectively final and not modified after they are set. The name variable declared on line 17 is not effectively final since it is modified on line 22. Finally, the food variable on line 18 is not effectively final since it is modified on line 20.

13. What is the output of the following code?

```
// RopeSwing.java
import rope.*;
import static rope.Rope.*;
public class RopeSwing {
    private static Rope ropel = new Rope();
    private static Rope rope2 = new Rope();
    {
        System.out.println(ropel.length);
    }
    public static void main(String[] args) {
        ropel.length = 2;
        rope2.length = 8;
        System.out.println(ropel.length);
    }
}

// Rope.java
package rope;
public class Rope {
    public static int length = 0;
}
```

- A. 02
- B. 08
- C. 2
- D. 8
- E. The code does not compile.
- F. An exception is thrown.

Explanation: There are two details to notice in this code. First, note that RopeSwing has an instance initializer and not a static initializer. Since RopeSwing is never constructed, the instance initializer does not run. The other detail is that length is static. Changes from any object update this common static variable. The code prints 8, making option D correct.

14. How many lines in the following code have compiler errors?

```
1: public class RopeSwing {
2:     private static final String leftRope;
3:     private static final String rightRope;
4:     private static final String bench;
5:     private static final String name = "name";
6:     static {
7:         leftRope = "left";
8:         rightRope = "right";
9:     }
10:    static {
11:        name = "name";
12:        rightRope = "right";
13:    }
14:    public static void main(String[] args) {
15:        bench = "bench";
16:    }
17: }
```

- A. 0
- B. 1
- C. 2
- D. 3
- E. 4**
- F. 5

Explanation: If a variable is static final, it must be set exactly once, and it must be in the declaration line or in a static initialization block. Line 15 doesn't compile because final variables are not allowed to be set after that point.

15. Which of the following can replace line 2 to make this code compile?

```
1: import java.util.*;
2: // INSERT CODE HERE
3: public class Imports {
4:     public void method(ArrayList<String> list) {
5:         sort(list);
6:     }
7: }
```

- A. import static java.util.Collections;
- B. import static java.util.Collections.*;**
- C. import static java.util.Collections.sort(ArrayList<String>);
- D. static import java.util.Collections;
- E. static import java.util.Collections.*;
- F. static import java.util.Collections.sort(ArrayList<String>);

Explanation: The two valid ways to do this are import static java.util.Collections.*; and import static java.util.Collections.sort;; making option B correct. Option A is incorrect because you can do a static import only on static members. Classes such as Collections require a regular import

16. What is the result of the following statements?

```
1: public class Test {
2:     public void print(byte x) {
3:         System.out.print("byte-");
4:     }
5:     public void print(int x) {
6:         System.out.print("int-");
7:     }
8:     public void print(float x) {
9:         System.out.print("float-");
10:    }
11:    public void print(Object x) {
12:        System.out.print("Object-");
13:    }
14:    public static void main(String[] args) {
15:        Test t = new Test();
16:        short s = 123;
17:        t.print(s);
18:        t.print(true);
19:        t.print(6.789);
20:    }
21: }
```

- A. byte-float-Object-
- B. int-float-Object-
- C. byte-Object-float-
- D. int-Object-float-
- E. int-Object-Object-
- F. byte-Object-Object-

Explanation: The argument on line 17 is a short. It can be promoted to an int, so print() on line 5 is invoked. The argument on line 18 is a boolean. It can be autoboxed to a Boolean, so print() on line 11 is invoked. The argument on line 19 is a double. It can be autoboxed to a Double, so print() on line 11 is invoked.

17. What is the result of the following program?

```
1: public class Squares {
2:     public static long square(int x) {
3:         var y = x * (long) x;
4:         x = -1;
5:         return y;
6:     }
7:     public static void main(String[] args) {
8:         var value = 9;
9:         var result = square(value);
10:        System.out.println(value);
11:    }
12: }
```

```
11:    } }
```

- A. -1
- B. 9
- C. 81
- D. Compiler error on line 9
- E. Compiler error on a different line

18. Which of the following are output by the following code? (Choose all that apply.)

```
public class StringBuilders {  
    public static StringBuilder work(StringBuilder a,  
        StringBuilder b) {  
        a = new StringBuilder("a");  
        b.append("b");  
        return a;  
    }  
    public static void main(String[] args) {  
        var s1 = new StringBuilder("s1");  
        var s2 = new StringBuilder("s2");  
        var s3 = work(s1, s2);  
        System.out.println("s1 = " + s1);  
        System.out.println("s2 = " + s2);  
        System.out.println("s3 = " + s3);  
    }  
}
```

- A. s1 = a
- B. s1 = s1
- C. s2 = s2
- D. s2 = s2b
- E. s3 = a
- F. The code does not compile.

19. Which of the following will compile when independently inserted in the following code? (Choose all that apply.)

```
1: public class Order3 {  
2:     final String value1 = "red";  
3:     static String value2 = "blue";  
4:     String value3 = "yellow";  
5:     {  
6:         // CODE SNIPPET 1  
7:     }  
8:     static {  
9:         // CODE SNIPPET 2  
10:    } }
```

- A. Insert at line 6: value1 = "green";
- B. Insert at line 6: value2 = "purple";
- C. Insert at line 6: value3 = "orange";

- D. Insert at line 9: value1 = "magenta";
- E. Insert at line 9: value2 = "cyan";
- F. Insert at line 9: value3 = "turquoise";

Explanation: The variable value1 is a final instance variable. It can be set only once: in the variable declaration, an instance initializer, or a constructor. The variable value2 is a static variable. Both instance and static initializers are able to access static variables, making options B and E correct. The variable value3 is an instance variable, making option C correct.

20. Which of the following are true about the following code? (Choose all that apply.)

```
public class Run {
    static void execute() {
        System.out.print("1-");
    }
    static void execute(int num) {
        System.out.print("2-");
    }
    static void execute(Integer num) {
        System.out.print("3-");
    }
    static void execute(Object num) {
        System.out.print("4-");
    }
    static void execute(int... nums) {
        System.out.print("5-");
    }
    public static void main(String[] args) {
        Run.execute(100);
        Run.execute(100L);
    }
}
```

- A. The code prints out 2-4-.
- B. The code prints out 3-4-.
- C. The code prints out 4-2-.
- D. The code prints out 4-4-.
- E. The code prints 3-4- if you remove the method static void execute(int num).
- F. The code prints 4-4- if you remove the method static void execute(int num).

21. Which method signatures are valid overloads of the following method signature? (Choose all that apply.)

```
public void moo(int m, int... n)
```

- A. public void moo(int a, int... b)
- B. public int moo(char ch)
- C. public void moooo(int... z)
- D. private void moo(int... x)
- E. public void moooo(int y)
- F. public void moo(int... c, int d)
- G. public void moo(int... i, int j...)

Explanation: When overloading methods, the return type and access modifiers do not need to be the same.

Chapter 6: Class Design

1. Which code can be inserted to have the code print 2?

```
public class BirdSeed {  
    private int numberBags;  
    boolean call;  
  
    public BirdSeed() {  
        // LINE 1  
        call = false;  
        // LINE 2  
    }  
  
    public BirdSeed(int numberBags) {  
        this.numberBags = numberBags;  
    }  
  
    public static void main(String[] args) {  
        var seed = new BirdSeed();  
        System.out.print(seed.numberBags);  
    } }
```

- A. Replace line 1 with BirdSeed(2);
 - B. Replace line 2 with BirdSeed(2);
 - C. Replace line 1 with new BirdSeed(2);
 - D. Replace line 2 with new BirdSeed(2);
 - E. Replace line 1 with this(2);
 - F. Replace line 2 with this(2);
 - G. The code prints 2 without any changes.
2. Which modifier pairs can be used together in a method declaration? (Choose all that apply.)
- A. static and final
 - B. private and static
 - C. static and abstract
 - D. private and abstract
 - E. abstract and final
 - F. private and final
3. Which of the following statements about methods are true? (Choose all that apply.)
- A. Overloaded methods must have the same signature.
 - B. Overridden methods must have the same signature.
 - C. Hidden methods must have the same signature.
 - D. Overloaded methods must have the same return type.
 - E. Overridden methods must have the same return type.
 - F. Hidden methods must have the same return type.

4. What is the output of the following program?

```
1: class Mammal {
2:     private void sneeze() {}
3:     public Mammal(int age) {
4:         System.out.print("Mammal");
5:     } }
6: public class Platypus extends Mammal {
7:     int sneeze() { return 1; }
8:     public Platypus() {
9:         System.out.print("Platypus");
10:    }
11:    public static void main(String[] args) {
12:        new Mammal(5);
13:    } }
```

- A. Platypus
- B. Mammal
- C. PlatypusMammal
- D. MammalPlatypus
- E. The code will compile if line 7 is changed.
- F. The code will compile if line 9 is changed.

5. Which of the following completes the constructor so that this code prints out 50?

```
class Speedster {
    int numSpots;
}
public class Cheetah extends Speedster {
    int numSpots;

    public Cheetah(int numSpots) {
        // INSERT CODE HERE
    }

    public static void main(String[] args) {
        Speedster s = new Cheetah(50);
        System.out.print(s.numSpots);
    }
}
```

- A. numSpots = numSpots;
- B. numSpots = this.numSpots;
- C. this.numSpots = numSpots;
- D. numSpots = super.numSpots;
- E. super.numSpots = numSpots;
- F. The code does not compile regardless of the code inserted into the constructor.
- G. None of the above.

6. Which of the following declare immutable classes? (Choose all that apply.)

```
public final class Moose {  
    private final int antlers;  
}  
  
public class Caribou {  
    private int antlers = 10;  
}  
  
public class Reindeer {  
    private final int antlers = 5;  
}  
  
public final class Elk {}  
  
public final class Deer {  
    private final Object o = new Object();  
}
```

- A. Moose
- B. Caribou
- C. Reindeer
- D. Elk**
- E. Deer**
- F. None of the above

7. What is the output of the following code?

```
1: class Arthropod {  
2:     protected void printName(long input) {  
3:         System.out.print("Arthropod");  
4:     }  
5:     void printName(int input) {  
6:         System.out.print("Spooky");  
7:     } }  
8: public class Spider extends Arthropod {  
9:     protected void printName(int input) {  
10:         System.out.print("Spider");  
11:     }  
12:     public static void main(String[] args) {  
13:         Arthropod a = new Spider();  
14:         a.printName((short)4);  
15:         a.printName(4);  
16:         a.printName(5L);  
17:     } }
```

- A. SpiderSpiderArthropod**
- B. SpiderSpiderSpider
- C. SpiderSpookyArthropod
- D. SpookySpiderArthropod
- E. The code will not compile because of line 5.

- F. The code will not compile because of line 9.
- G. None of the above.

8. What is the result of the following code?

```
1:  abstract class Bird {
2:      private final void fly() { System.out.println("Bird"); }
3:      protected Bird() { System.out.print("Wow-"); }
4:  }
5:  public class Pelican extends Bird {
6:      public Pelican() { System.out.print("Oh-"); }
7:      protected void fly() { System.out.println("Pelican"); }
8:      public static void main(String[] args) {
9:          var chirp = new Pelican();
10:         chirp.fly();
11:     } }
```

- A. Oh-Bird
- B. Oh-Pelican
- C. Wow-Oh-Bird
- D. Wow-Oh-Pelican**
- E. The code contains a compilation error.
- F. None of the above.

9. Which of the following statements about overridden methods are true? (Choose all that apply.)

- A. An overridden method must contain method parameters that are the same or covariant with the method parameters in the inherited method.
- B. An overridden method may declare a new exception, provided it is not checked.**
- C. An overridden method must be more accessible than the method in the parent class.
- D. An overridden method may declare a broader checked exception than the method in the parent class.
- E. If an inherited method returns void, then the overridden version of the method must return void.**
- F. None of the above.

10. Which of the following pairs, when inserted into the blanks, allow the code to compile? (Choose all that apply.)

```
1:  public class Howler {
2:      public Howler(long shadow) {
3:          _____;
4:      }
5:      private Howler(int moon) {
6:          super();
7:      }
8:  }
9:  class Wolf extends Howler {
10:     protected Wolf(String stars) {
11:         super(2L);
12:     }
13:     public Wolf() {
```

```

14:      _____;
15:    }
16: }

```

- A. this(3) at line 3, this("") at line 14.
- B. this() at line 3, super(1) at line 14.
- C. this((short)1) at line 3, this(null) at line 14.
- D. super() at line 3, super() at line 14.
- E. this(2L) at line 3, super((short)2) at line 14.
- F. this(5) at line 3, super(null) at line 14.
- G. Remove lines 3 and 14.

11. What is the result of the following?

```

1:  public class PolarBear {
2:      StringBuilder value = new StringBuilder("t");
3:      { value.append("a"); }
4:      { value.append("c"); }
5:      private PolarBear() {
6:          value.append("b");
7:      }
8:      public PolarBear(String s) {
9:          this();
10:         value.append(s);
11:     }
12:     public PolarBear(CharSequence p) {
13:         value.append(p);
14:     }
15:     public static void main(String[] args) {
16:         Object bear = new PolarBear();
17:         bear = new PolarBear("f");
18:         System.out.println(((PolarBear)bear).value);
19:     } }

```

- A. tacb
- B. tacf
- C. tacbf
- D. tcafb
- E. taftacb
- F. The code does not compile.
- G. An exception is thrown.

12. How many lines of the following program contain a compilation error?

```

1:  public class Rodent {
2:      public Rodent(Integer x) {}
3:      protected static Integer chew() throws Exception {
4:          System.out.println("Rodent is chewing");
5:          return 1;
6:      }
7:  }

```

```
8:  class Beaver extends Rodent { //NOT COMPILE
9:      public Number chew() throws RuntimeException { //NOT COMPILE
10:          System.out.println("Beaver is chewing on wood");
11:          return 2;
12:      } }
```

- A. None
- B. 1
- C. 2**
- D. 3
- E. 4
- F. 5

13. Which of these classes compile and will include a default constructor created by the compiler? (Choose all that apply.)

A.

```
public class Bird {}
```

B.

```
public class Bird {
    public bird() {}
}
```

C.

```
public class Bird {
    public bird(String name) {}
}
```

D.

```
public class Bird {
    public Bird() {}
}
```

E.

```
public class Bird {
    Bird(String name) {}
}
```

F.

```
public class Bird {
    private Bird(int age) {}
}
```

G.

```
public class Bird {
    public Bird bird() { return null; }
}
```

14. Which of the following statements about inheritance are correct? (Choose all that apply.)

- A. A class can directly extend any number of classes.
- B. A class can implement any number of interfaces.
- C. All variables inherit java.lang.Object.
- D. If class A is extended by B, then B is a superclass of A.
- E. If class C implements interface D, then C is a subtype of D.
- F. Multiple inheritance is the property of a class to have multiple direct superclasses.

15. Which statement about the following program is correct?

```
1: abstract class Nocturnal {
2:     boolean isBlind();
3: }
4: public class Owl extends Nocturnal {
5:     public boolean isBlind() { return false; }
6:     public static void main(String[] args) {
7:         var nocturnal = (Nocturnal)new Owl();
8:         System.out.println(nocturnal.isBlind());
9:     } }
```

- A. It compiles and prints true.
- B. It compiles and prints false.
- C. The code will not compile because of line 2.
- D. The code will not compile because of line 5.
- E. The code will not compile because of line 7.
- F. The code will not compile because of line 8.
- G. None of the above.

16. What is the result of the following?

```
1: class Arachnid {
2:     static StringBuilder sb = new StringBuilder();
3:     { sb.append("c"); }
4:     static
5:     { sb.append("u"); }
6:     { sb.append("r"); }
7: }
8: public class Scorpion extends Arachnid {
9:     static
10:    { sb.append("q"); }
11:    { sb.append("m"); }
12:    public static void main(String[] args) {
13:        System.out.print(Scorpion.sb + " ");
14:        System.out.print(Scorpion.sb + " ");
15:        new Arachnid();
16:        new Scorpion();
17:        System.out.print(Scorpion.sb);
18:    } }
```

- A. qu qu qumrcrc
- B. u u ucrcrm
- C. uq uq uqmcrcr
- D. uq uq uqcrerm
- E. qu qu qumrcrc
- F. qu qu qucrerm
- G. The code does not compile.

17. Which of the following are true? (Choose all that apply.)

- A. this() can be called from anywhere in a constructor.
- B. this() can be called from anywhere in an instance method.
- C. this.variableName can be called from any instance method in the class.
- D. this.variableName can be called from any static method in the class.
- E. You can call the default constructor written by the compiler using this().
- F. You can access a private constructor with the main() method in the same class.

18. Which statements about the following classes are correct? (Choose all that apply.)

```

1: public class Mammal {
2:     private void eat() {}
3:     protected static void drink() {}
4:     public Integer dance(String p) { return null; }
5: }
6: class Primate extends Mammal {
7:     public void eat(String p) {}
8: }
9: class Monkey extends Primate {
10:    public static void drink() throws RuntimeException {}
11:    public Number dance(CharSequence p) { return null; }
12:    public int eat(String p) {}
13: }
```

- A. The eat() method in Mammal is correctly overridden on line 7.
- B. The eat() method in Mammal is correctly overloaded on line 7.
- C. The drink() method in Mammal is correctly overridden on line 10.
- D. The drink() method in Mammal is correctly hidden on line 10.
- E. The dance() method in Mammal is correctly overridden on line 11.
- F. The dance() method in Mammal is correctly overloaded on line 11.
- G. The eat() method in Primate is correctly hidden on line 12.
- H. The eat() method in Primate is correctly overloaded on line 12.

19. What is the output of the following code?

```

1: class Reptile {
2:     {System.out.print("A");}
3:     public Reptile(int hatch) {}
4:     void layEggs() {
5:         System.out.print("Reptile");
6:     } }
7: public class Lizard extends Reptile {
```



```

8:      static {System.out.print("B");}
9:      public Lizard(int hatch) {}
10:     public final void layEggs() {
11:         System.out.print("Lizard");
12:     }
13:     public static void main(String[] args) {
14:         var reptile = new Lizard(1);
15:         reptile.layEggs();
16:     } }

```

- A. AALizard
- B. BALizard
- C. BLizardA
- D. ALizard
- E. The code will not compile because of line 3.
- F. None of the above.

20. Which statement about the following program is correct?

```

1:  class Bird {
2:      int feathers = 0;
3:      Bird(int x) { this.feathers = x; }
4:      Bird fly() {
5:          return new Bird(1);
6:      } }
7:  class Parrot extends Bird {
8:      protected Parrot(int y) { super(y); }
9:      protected Parrot fly() {
10:         return new Parrot(2);
11:     } }
12: public class Macaw extends Parrot {
13:     public Macaw(int z) { super(z); }
14:     public Macaw fly() {
15:         return new Macaw(3);
16:     }
17:     public static void main(String... sing) {
18:         Bird p = new Macaw(4);
19:         System.out.print(((Parrot)p.fly()).feathers);
20:     } }

```

- A. One line contains a compiler error.
- B. Two lines contain compiler errors.
- C. Three lines contain compiler errors.
- D. The code compiles but throws a ClassCastException at runtime.
- E. The program compiles and prints 3.
- F. The program compiles and prints 0.

21. Which of the following are properties of immutable classes? (Choose all that apply.)

- A. The class can contain setter methods, provided they are marked final.
- B. The class must not be able to be extended outside the class declaration.
- C. The class may not contain any instance variables.

- D. The class must be marked static.
- E. The class may not contain any static variables.
- F. The class may only contain private constructors.
- G. The data for mutable instance variables may be read, provided they cannot be modified by the caller.

22. What does the following program print?

```
1: class Person {
2:     static String name;
3:     void setName(String q) { name = q; } }
4: public class Child extends Person {
5:     static String name;
6:     void setName(String w) { name = w; }
7:     public static void main(String[] p) {
8:         final Child m = new Child();
9:         final Person t = m;
10:        m.name = "Elysia";
11:        t.name = "Sophia";
12:        m.setName("Webby");
13:        t.setName("Olivia");
14:        System.out.println(m.name + " " + t.name);
15:    } }
```

- A. Elysia Sophia
- B. Webby Olivia
- C. Olivia Olivia
- D. Olivia Sophia
- E. The code does not compile.
- F. None of the above.

23. What is the output of the following program?

```
1: class Canine {
2:     public Canine(boolean t) { logger.append("a"); }
3:     public Canine() { logger.append("q"); }
4:
5:     private StringBuilder logger = new StringBuilder();
6:     protected void print(String v) { logger.append(v); }
7:     protected String view() { return logger.toString(); }
8: }
9:
10: class Fox extends Canine {
11:     public Fox(long x) { print("p"); }
12:     public Fox(String name) {
13:         this(2);
14:         print("z");
15:     }
16: }
17:
```

```

18: public class Fennec extends Fox {
19:     public Fennec(int e) {
20:         super("tails");
21:         print("j");
22:     }
23:     public Fennec(short f) {
24:         super("eevee");
25:         print("m");
26:     }
27:
28:     public static void main(String... unused) {
29:         System.out.println(new Fennec(1).view());
30:     } }

```

- A. qpz
- B. qpzj**
- C. jzpa
- D. apj
- E. apjm
- F. The code does not compile.
- G. None of the above.

24. What is printed by the following program?

```

1: class Antelope {
2:     public Antelope(int p) {
3:         System.out.print("4");
4:     }
5:     { System.out.print("2"); }
6:     static { System.out.print("1"); }
7: }
8: public class Gazelle extends Antelope {
9:     public Gazelle(int p) {
10:         super(6);
11:         System.out.print("3");
12:     }
13:     public static void main(String hopping[]) {
14:         new Gazelle(0);
15:     }
16:     static { System.out.print("8"); }
17:     { System.out.print("9"); }
18: }

```

- A. 182640
- B. 182943
- C. 182493**
- D. 421389
- E. The code does not compile.
- F. The output cannot be determined until runtime.

25. Which of the following are true about a concrete class? (Choose all that apply.)

- A. A concrete class can be declared as abstract.
- B. A concrete class must implement all inherited abstract methods.
- C. A concrete class can be marked as final.
- D. A concrete class must be immutable.
- E. A concrete method that implements an abstract method must match the method declaration of the abstract method exactly.

26. What is the output of the following code?

```
4: public abstract class Whale {
5:     public abstract void dive();
6:     public static void main(String[] args) {
7:         Whale whale = new Orca();
8:         whale.dive(3);
9:     }
10: }
11: class Orca extends Whale {
12:     static public int MAX = 3;
13:     public void dive() {
14:         System.out.println("Orca diving");
15:     }
16:     public void dive(int... depth) {
17:         System.out.println("Orca diving deeper "+MAX);
18: } }
```

- A. Orca diving
- B. Orca diving deeper 3
- C. The code will not compile because of line 4.
- D. The code will not compile because of line 8.
- E. The code will not compile because of line 11.
- F. The code will not compile because of line 12.
- G. The code will not compile because of line 17.
- H. None of the above.

Chapter 7: Beyond Classes

1. Which of the following are valid record declarations? (Choose all that apply.)

```
public record Iguana(int age) {
    private static final int age = 10; }

public final record Gecko() {}

public abstract record Chameleon() {
    private static String name; }

public record BeardedDragon(boolean fun) {
    @Override public boolean fun() { return false; } }

public record Reptile(long size) {
    public Reptile {
        if(size == 1) throw new IllegalArgumentException();
    } }

public record Newt(double age) extends Reptile {
    public Newt(double age) {
        age = this.age % 2 == 0 ? 5 : 10;
    } }
```

- A. Iguana
 - B. Gecko
 - C. Chameleon
 - D. BeardedDragon
 - E. Reptile
 - F. Newt
 - G. None of the above
2. Which of the following statements can be inserted in the blank line so that the code will compile successfully? (Choose all that apply.)

```
interface CanHop {}
public class Frog implements CanHop {
    public static void main(String[] args) {
        _____ frog = new TurtleFrog();
    }
}

class BrazilianHornedFrog extends Frog {}
class TurtleFrog extends Frog {}
```

- A. Frog
- B. TurtleFrog
- C. BrazilianHornedFrog
- D. CanHop

E. var

F. Long

G. None of the above; the code contains a compilation error.

3. What is the result of the following program?

```
11: public class Favorites {
12:     enum Flavors {
13:         VANILLA, CHOCOLATE, STRAWBERRY
14:         public Flavors() {}
15:     }
16:     public static void main(String[] args) {
17:         for(final var e : Flavors.values())
18:             System.out.print((e.ordinal() % 2) + " ");
19:     } }
```

A. 0 1 0

B. 1 0 1

C. Exactly one line of code does not compile.

D. More than one line of code does not compile.

E. The code compiles but produces an exception at runtime.

F. None of the above.

Explanation: Line 14 also does not compile, as enum constructors are implicitly private and cannot be declared as public or protected + semicolon on line 13

4. What is the output of the following program?

```
public sealed class ArmoredAnimal permits Armadillo {
    public ArmoredAnimal(int size) {}
    @Override public String toString() { return "Strong"; }
    public static void main(String[] a) {
        var c = new Armadillo(10, null);
        System.out.println(c);
    }
}
class Armadillo extends ArmoredAnimal {
    @Override public String toString() { return "Cute"; }
    public Armadillo(int size, String name) {
        super(size);
    }
}
```

A. Strong

B. Cute

C. The program does not compile.

D. The code compiles but produces an exception at runtime.

E. None of the above.

Explanation: A class extending a sealed class must be marked final, sealed, or non-sealed. Since Armadillo is missing a modifier, the code does not compile, and option C is correct.

5. Which statement about the following program is correct?

```
1: interface HasExoskeleton {
2:     double size = 2.0f;
3:     abstract int getNumberOfSections();
4: }
5: abstract class Insect implements HasExoskeleton {
6:     abstract int getNumberOfLegs();
7: }
8: public class Beetle extends Insect {
9:     int getNumberOfLegs() { return 6; }
10:    int getNumberOfSections(int count) { return 1; }
11: }
```

- A. It compiles without issue.
- B. The code will produce a ClassCastException if called at runtime.
- C. The code will not compile because of line 2.
- D. The code will not compile because of line 5.
- E. The code will not compile because of line 8.**
- F. The code will not compile because of line 10.

Explanation: The concrete class Beetle extends Insect and inherits two abstract methods, getNumberOfSections() and getNumberOfLegs(). The Beetle class includes an overloaded version of getNumberOfSections() that takes an int value but it does not satisfy the abstract method requirement inherited from HasExoskeleton. For this reason, only one of the two abstract methods is properly overridden.

6. Which statements about the following program are correct? (Choose all that apply.)

```
1: public abstract interface Herbivore {
2:     int amount = 10;
3:     public void eatGrass();
4:     public abstract int chew() { return 13; }
5: }
6:
7: abstract class IsAPlant extends Herbivore {
8:     Object eatGrass(int season) { return null; }
9: }
```

- A. It compiles and runs without issue.
- B. The code will not compile because of line 1.
- C. The code will not compile because of line 2.
- D. The code will not compile because of line 4.**
- E. The code will not compile because of line 7.**
- F. The code will not compile because line 8 contains an invalid method override.

Explanation: Line 4 does not compile, since an abstract method cannot include a body. Line 7 also does not compile because the wrong keyword is used. A class implements an interface; it does not extend it.

7. What is the output of the following program?

```
1: interface Aquatic {
2:     int getNumOfGills(int p);
3: }
4: public class ClownFish implements Aquatic {
5:     String getNumOfGills() { return "14"; }
6:     int getNumOfGills(int input) { return 15; }
7:     public static void main(String[] args) {
8:         System.out.println(new ClownFish().getNumOfGills(-1));
9:     } }
```

- A. 14
- B. 15
- C. The code will not compile because of line 4.
- D. The code will not compile because of line 5.
- E. The code will not compile because of line 6.**
- F. None of the above.

Explanation: The inherited interface method `getNumOfGills(int)` is implicitly public; therefore, it must be declared public in any concrete class that implements the interface.

8. Given the following, select the statements that can be inserted into the blank line so that the code will compile and print true at runtime? (Choose all that apply.)

```
record Walrus(List<String> diet) {}
record Exhibit(Walrus animal, String location) {}

var e = new Exhibit(new Walrus(List.of("Wally")), "Artic");
System.out.print(e instanceof _____);
```

- A. `Exhibit(Walrus(List<Integer> z), Object a)`
- B. `Exhibit(Walrus(List m), Object n)`**
- C. `Object w && w.animal().diet().size() == 0`
- D. `Exhibit(Walrus(var i), var i)`
- E. `Exhibit(var p, var q)`**
- F. `Exhibit(List<?> g, var h)`
- G. `Exhibit(var x, CharSequence y)`**
- H. `Exhibit(Walrus(null), var v)`
- I. None of the above

9. Which of the following statements can be inserted in the blank so that the code will compile successfully? (Choose all that apply.)

```
abstract class Snake {}
class Cobra extends Snake {}
class GardenSnake extends Cobra {}
public class SnakeHandler {
    private Snake snakey;
    public void setSnake(Snake mySnake) { this.snakey = mySnake; }
    public static void main(String[] args) {
```



```
new SnakeHandler().setSnake(____);  
} }
```

A. new Cobra()

B. new Snake()

C. new Object()

D. new String("Snake")

E. new GardenSnake()

F. null

G. None of the above. The class does not compile, regardless of the value inserted in the blank.

10. What types can be inserted in the blanks on the lines marked X and Z that allow the code to compile?
(Choose all that apply.)

```
interface Walk { private static List move() { return null; } }  
interface Run extends Walk { public ArrayList move(); }  
class Leopard implements Walk {  
    public _____ move() { // X  
        return null;  
    }  
}  
class Panther implements Run {  
    public move(_____) { // Z  
        return null;  
    }  
}
```

A. Integer on the line marked X

B. ArrayList on the line marked X

C. List on the line marked X

D. List on the line marked Z

E. ArrayList on the line marked Z

F. None of the above, since the Run interface does not compile.

G. Does not compile for a different reason.

11. What is the result of compiling and executing the following code?

```
1: public class Movie {  
2:     private int butter = 5;  
3:     private Movie() {}  
4:     protected class Popcorn {  
5:         private Popcorn() {}  
6:         public static int butter = 10;  
7:         public void startMovie() {  
8:             System.out.println(butter);  
9:         }  
10:    }  
11:    public static void main(String[] args) {  
12:        var movie = new Movie();  
13:        Movie.Popcorn in = new Movie().new Popcorn();  
14:        in.startMovie();  
15:    } }
```

- A. The output is 5.
- B. The output is 10.
- C. Line 6 generates a compiler error.
- D. Line 12 generates a compiler error.
- E. Line 13 generates a compiler error.
- F. The code compiles but produces an exception at runtime.

12. Which variables or members are accessible from within the hiss() method? (Choose all that apply.)

```

13: public class BoaConstrictor {
14:     private Body body;
15:     BoaConstrictor(Body b) { this.body = b; }
16:     private long tail = 10;
17:     record Body(int stripes) {
18:         private static int counter = 0;
19:         int counter() { return counter; }
20:         Body {
21:             stripes = stripes + counter++;
22:         }
23:         private void hiss() {} } }

```

- A. counter()
- B. tail
- C. body
- D. stripes()
- E. stripes
- F. counter
- G. Line 15 does not compile.
- H. Line 17 does not compile.
- I. Lines 20–22 do not compile.

13. What is the result of the following program?

```

public class Weather {
    enum Seasons {
        WINTER, SPRING, SUMMER, FALL
    }

    public static void main(String[] args) {
        Seasons v = null;
        switch (v) {
            case Seasons.SPRING -> System.out.print("s");
            case Seasons.WINTER -> System.out.print("w");
            case Seasons.SUMMER -> System.out.print("m");
            default -> System.out.println("missing data"); }
    } }

```

- A. s
- B. w
- C. m

- D. missing data
- E. Exactly one line of code does not compile.
- F. More than one line of code does not compile.
- G. The code compiles but produces an exception at runtime.

14. Which statements about sealed classes are correct? (Choose all that apply.)

- A. A sealed interface restricts which subinterfaces may extend it.
- B. A sealed class cannot be indirectly extended by a class that is not listed in its permits clause.
- C. A sealed class can be extended by an abstract class.
- D. A sealed class can be extended by a subclass that uses the nonsealed modifier.
- E. A sealed interface restricts which subclasses may implement it.
- F. A sealed class cannot contain any nested subclasses.
- G. None of the above.

15. Which line allows the code to print Not scared at runtime?

```
public class Ghost {
    public static void boo() {
        System.out.println("Not scared");
    }
    protected final class Spirit {
        public void boo() {
            System.out.println("Booo!!!");
        }
    }
    public static void main(String... haunt) {
        var g = new Ghost().new Spirit() {};
        _____;
    } }
}
```

- A. g.boo()
- B. g.super.boo()
- C. new Ghost().boo()
- D. g.Ghost.boo()
- E. new Spirit().boo()
- F. None of the above

Explanation: The Spirit class is marked final, so it cannot be extended. The main() method uses an anonymous class that inherits from Spirit, which is not allowed. If Spirit were not marked final, then option C would be correct.

16. The following code appears in a file named Ostrich.java. What is the result of compiling the source file?

```
1: public class Ostrich {
2:     private int count;
3:     static class OstrichWrangler {
4:         public int stampede() {
5:             return count;
6:         } } }
}
```

- A. The code compiles successfully, and one bytecode file is generated: Ostrich.class.
- B. The code compiles successfully, and two bytecode files are generated: Ostrich.class and OstrichWrangler.class.
- C. The code compiles successfully, and two bytecode files are generated: Ostrich.class and Ostrich\$OstrichWrangler.class.
- D. A compiler error occurs on line 3.
- E. A compiler error occurs on line 5.

Explanation: The OstrichWrangler class is a static nested class; therefore, it cannot access the instance member count.

17. Which lines of the following interface declarations do not compile? (Choose all that apply.)

```
1: public interface Omnivore {  
2:     int amount = 10;  
3:     static boolean gather = true;  
4:     static void eatGrass() {}  
5:     int findMore() { return 2; }  
6:     default float rest() { return 2; }  
7:     protected int chew() { return 13; }  
8:     private static void eatLeaves() {}  
9: }
```

- A. All of the lines compile without issue.
- B. Line 2.
- C. Line 3.
- D. Line 4.
- E. Line 5.
- F. Line 6.
- G. Line 7.
- H. Line 8.

Explanation: Non-static interface methods with a body must be explicitly marked private or default. Line 7 does not compile, as interfaces do not have protected members.

18. What is printed by the following program?

```
public class Deer {  
    enum Food {APPLES, BERRIES, GRASS}  
    protected class Diet {  
        private Food getFavorite() {  
            return Food.BERRIES;  
        }  
    }  
    public static void main(String[] seasons) {  
        System.out.print(switch(new Diet().getFavorite()) {  
            case APPLES -> "a";  
            case BERRIES -> "b";  
            default -> "c";  
        });  
    }  
}
```

- A. a
- B. b
- C. c
- D. The code declaration of the Diet class does not compile.
- E. The main() method does not compile.
- F. The code compiles but produces an exception at runtime.
- G. None of the above.

Explanation: Diet is an inner class, which requires an instance of Deer to instantiate. Since the main() method is static, there is no such instance.

19. Which of the following is printed by the Bear program?

```
public class Bear {
    enum FOOD {
        BERRIES, INSECTS {
            public boolean isHealthy() { return true; }},
        FISH, ROOTS, COOKIES, HONEY;
        public abstract boolean isHealthy();
    }
    public static void main(String[] args) {
        System.out.print(FOOD.INSECTS);
        System.out.print(FOOD.INSECTS.ordinal());
        System.out.print(FOOD.INSECTS.isHealthy());
        System.out.print(FOOD.COOKIES.isHealthy());
    } }
```

- A. insects
- B. Insects
- C. 0
- D. 1
- E. false
- F. The code does not compile.

Explanation: the isHealthy() method is marked abstract in the enum; therefore, it must be implemented in each enum value declaration. Since only INSECTS implements it, the code does not compile.

20. What is the output of this code?

```
13: record Gorilla(int x, Double y) {
14:     Gorilla {}
15:     Gorilla() { this(1,2.0); }
16: }
17: record Family(Gorilla parent1, Gorilla parent2) {}
18:
19: var family = new Family(
20:     new Gorilla(1, null), new Gorilla(0, 1.2));
21: System.out.print(switch (family) {
22:     case Family(var a, var b) -> "1";
23:     case Family(Gorilla c, Gorilla (int d, double e)) -> "2";
24:     case Family(Gorilla (int f, Double g), var h) -> "3";
25:     case Family(Gorilla i, Gorilla (int j, Double k)) -> "4";
```

```

26:     case Family(Object m, Object n) -> "5";
27:     case null -> "6";
28:     default -> "7";
29: });

```

- A. 1
- B. 2
- C. 3
- D. 4
- E. 5
- F. 6
- G. 7
- H. None of the above

Explanation: The record declarations compile but the switch expression does not. First, the second case statement does not compile, as double is not compatible with Double. Next, the pattern matching case statement on line 22 dominates the ones on lines 23–25.

21. Given the following record declaration, which line of code can fill in the blank and allow the code to compile?

```

public record RabbitFood(int size, String brand, LocalDate expires) {
    public static int MAX_STORAGE = 100;
    public RabbitFood() {
        _____;
    }
}

```

- A. size = MAX_STORAGE
- B. this.size = 10
- C. if(expires.isAfter(LocalDate.now())) throw new RuntimeException()
- D. if(brand==null) super.brand = "Unknown"
- E. throw new RuntimeException()
- F. None of the above

Explanation: The record defines an overloaded constructor using parentheses, not a compact one. For this reason, the first line must be a call to another constructor, such as this(500, "Acme", LocalDate.now()).

22. Which of the following can be inserted in the rest() method? (Choose all that apply.)

```

public class Lion {
    class Cub {}
    static class Den {}
    static void rest() {
        _____;
    }
}

```

- A. Cub a = Lion.new Cub()
- B. Lion.Cub b = new Lion().Cub()
- C. Lion.Cub c = new Lion().new Cub()
- D. var d = new Den()
- E. var e = Lion.new Cub()

- F. `Lion.Den f = Lion.new Den()`
- G. `Lion.Den g = new Lion.Den()`
- H. `var h = new Cub()`

23. Given the following program, what can be inserted into the blank line that would allow it to print Swim! at runtime?

```
interface Swim {
    default void perform() { System.out.print("Swim!"); }
}
interface Dance {
    default void perform() { System.out.print("Dance!"); }
}
public class Penguin implements Swim, Dance {
    public void perform() { System.out.print("Smile!"); }
    private void doShow() {
        _____;
    }
    public static void main(String[] eggs) {
        new Penguin().doShow();
    }
}
```

- A. `super.perform()`
- B. `Swim.perform()`
- C. `super.Swim.perform()`
- D. `Swim.super.perform()`
- E. The code does not compile regardless of what is inserted into the blank.
- F. The code compiles, but due to polymorphism, it is not possible to produce the requested output without creating a new object.

24. Which lines of the following interface do not compile? (Choose all that apply.)

```
1: public interface BigCat {
2:     abstract String getName();
3:     static int hunt() { getName(); return 5; }
4:     default void climb() { rest(); }
5:     private void roar() { getName(); climb(); hunt(); }
6:     private static boolean sneak() { roar(); return true; }
7:     private int rest() { return 2; };
8: }
```

- A. Line 2
- B. Line 3
- C. Line 4
- D. Line 5
- E. Line 6
- F. Line 7
- G. None of the above

Explanation: Line 3 does not compile because the static method `hunt()` cannot access an abstract instance method `getName()`. Line 6 does not compile because the private static method `sneak()` cannot access the private instance method `roar()`.

25. What does the following program print?

```
1: public class Zebra {
2:     private int x = 24;
3:     public int hunt() {
4:         String message = "x is ";
5:         abstract class Stripes {
6:             private int x = 0;
7:             public void print() {
8:                 System.out.print(message + Zebra.this.x);
9:             }
10:        }
11:        var s = new Stripes() {};
12:        s.print();
13:        return x;
14:    }
15:    public static void main(String[] args) {
16:        new Zebra().hunt();
17:    } }
```

- A. x is 0
- B. x is 24
- C. Line 6 generates a compiler error.
- D. Line 8 generates a compiler error.
- E. Line 11 generates a compiler error.
- F. None of the above.

26. What is the output of the following program?

```
20: public enum Animals {
21:     MAMMAL(List.of(2,4)),
22:     INVERTEBRATE(List.of(2, 4, 6, 8, 100)),
23:     BIRD(null) {
24:         public int stand() {
25:             return legs.get(0) + 4;
26:         }
27:     };
28:     List<Integer> legs;
29:     Animals(List<Integer> legs) {
30:         this.legs = legs;
31:     }
32:     public int stand() { return legs.get(0); }
33:     public static void main(String[] a) {
34:         Animals.BIRD.legs = List.of(-1);
35:         System.out.println(Animals.BIRD.stand());
36:     } }
```


- A. null
- B. -1
- C. 3
- D. 4
- E. Compiler error on lines 23.
- F. Compiler error on lines 24.
- G. Compiler error on line 34.
- H. The code compiles but produces a NullPointerException at runtime.
- I. None of the above.

27. Assuming a record is defined with at least one field, which components does the compiler always insert, each of which may be overridden or redeclared? (Choose all that apply.)

- A. A no-argument constructor
- B. An accessor method for each field
- C. The toString() method
- D. The equals() method
- E. A mutator method for each field
- F. A sort method for each field
- G. The hashCode() method

28. Which of the following classes and interfaces do not compile? (Choose all that apply.)

```
public abstract class Camel { void travel(); }

public interface EatsGrass { private abstract int chew(); }

public abstract class Elephant {
    abstract private class SleepsAlot {
        abstract int sleep();
    }
}

public class Eagle { abstract soar(); }

public interface Spider { default void crawl() {} }
```

- A. Camel
- B. EatsGrass
- C. Elephant
- D. Eagle
- E. Spider

29. How many lines of the following program contain a compilation error?

```
1: class Primate {
2:     protected int age = 2;
3:     { age = 1; }
4:     public Primate() {
5:         this().age = 3;
6:     }
7: }
```

```

8: public class Orangutan {
9:     protected int age = 4;
10:    { age = 5; }
11:    public Orangutan() {
12:        this().age = 6;
13:    }
14:    public static void main(String[] bananas) {
15:        final Primate x = (Primate)new Orangutan();
16:        System.out.println(x.age);
17:    }
18: }

```

- A. None, and the program prints 1 at runtime.
- B. None, and the program prints 3 at runtime.
- C. None, but it causes a ClassCastException at runtime.
- D. 1
- E. 2
- F. 3**
- G. 4

Explanation: Both lines 5 and 12 do not compile, as `this()` is used instead of `this`. Remember, `this()` refers to calling a constructor, whereas `this` is a reference to the current instance. Next, the compiler does not allow casting to an unrelated class type. Since `Orangutan` is not a subclass of `Primate`, the cast on line 15 is invalid, and the code does not compile.

30. Assuming the following classes are declared as top-level types in the same file, which classes contain compiler errors? (Choose all that apply.)

```

sealed class Bird {
    public final class Flamingo extends Bird {}
}

sealed class Monkey {}

class EmperorTamarin extends Monkey {}

non-sealed class Mandrill extends Monkey {}

sealed class Friendly extends Mandrill permits Silly {}

final class Silly {}

```

- A. Bird
- B. Monkey
- C. EmperorTamarin**
- D. Mandrill
- E. Friendly**
- F. Silly
- G. All of the classes compile without issue.

Explanation: EmperorTamarin does not compile, as it is missing a non-sealed, sealed, or final modifier, making option C correct. Friendly also does not compile, since it lists a subclass Silly that does not extend it. While the permits clause is optional, the extends clause is not. Silly compiles just fine. Even though it does not extend Friendly, the compiler error is in the sealed class.

Chapter 8: Lambdas and Functional Interfaces

1. What is the result of the following class?

```
1: import java.util.function.*;
2:
3: public class Panda {
4:     int age;
5:     public static void main(String[] args) {
6:         Panda p1 = new Panda();
7:         p1.age = 1;
8:         check(p1, p -> p.age < 5);
9:     }
10:    private static void check(Panda panda,
11:        Predicate<Panda> pred) {
12:        String result =
13:            pred.test(panda) ? "match" : "not match";
14:        System.out.print(result);
15:    } }
```

- A. match
- B. not match
- C. Compiler error on line 8
- D. Compiler error on lines 10 and 11
- E. Compiler error on lines 12 and 13
- F. A runtime exception

2. What is the result of the following code?

```
1: interface Climb {
2:     boolean isTooHigh(int height, int limit);
3: }
4:
5: public class Climber {
6:     public static void main(String[] args) {
7:         check((h, m) -> h.append(m).isEmpty(), 5);
8:     }
9:     private static void check(Climb climb, int height) {
10:         if (climb.isTooHigh(height, 10))
11:             System.out.println("too high");
12:         else
13:             System.out.println("ok");
14:     }
15: }
```

- A. ok
- B. too high
- C. Compiler error on line 7.
- D. Compiler error on line 10.
- E. Compiler error on a different line.
- F. A runtime exception is thrown.

Explanation: The interface takes two int parameters. The code on line 7 attempts to use them as if h is a StringBuilder

3. Which statements about functional interfaces are true? (Choose all that apply.)

- A. A functional interface can contain default and private methods.
- B. A functional interface can be defined as a class or an interface.
- C. Abstract methods with signatures that are contained in public methods of java.lang.Object do not count toward the abstract method count for a functional interface.
- D. A functional interface cannot contain static or private static methods.
- E. A functional interface must be marked with the @FunctionalInterface annotation.

4. Which lambda can replace the MySecret class to return the same value? (Choose all that apply.)

```
interface Secret {  
    String magic(double d);  
}  
  
class MySecret implements Secret {  
    public String magic(double d) {  
        return "Poof";  
    }  
}
```

- A. (e) -> "Poof"
- B. (e) -> {"Poof"}
- C. (e) -> { String e = ""; "Poof" }
- D. (e) -> { String e = ""; return "Poof"; }
- E. (e) -> { String e = ""; return "Poof" }
- F. (e) -> { String f = ""; return "Poof"; }

5. Which of the following functional interfaces contain an abstract method that returns a primitive value? (Choose all that apply.)

- A. BooleanSupplier
- B. CharSupplier
- C. DoubleSupplier
- D. FloatSupplier
- E. IntSupplier
- F. StringSupplier

6. Which of the following lambda expressions can be passed to a function of Predicate<String> type? (Choose all that apply.)

- A. s -> s.isEmpty()
- B. s --> s.isEmpty()
- C. (String s) -> s.isEmpty()
- D. (String s) --> s.isEmpty()
- E. (StringBuilder s) -> s.isEmpty()
- F. (StringBuilder s) --> s.isEmpty()

7. Which of these statements is true about the following code?

```
public void method() {  
    x((var x) -> {}, (var x, var y) -> false);  
}  
public void x(Consumer<String> x, BinaryOperator<Boolean> y) {}
```

- A. The code does not compile because of one of the variables named x.
- B. The code does not compile because of one of the variables named y.
- C. The code does not compile for another reason.
- D. The code compiles, and the x in each lambda refers to the same type.
- E. The code compiles, and the x in each lambda refers to a different type.

8. Which of the following is equivalent to this code?

```
UnaryOperator<Integer> u = x -> x * x;
```

- A. BiFunction<Integer> f = x -> x*x;
- B. BiFunction<Integer, Integer> f = x -> x*x;
- C. BinaryOperator<Integer, Integer> f = x -> x*x;
- D. Function<Integer> f = x -> x*x;
- E. Function<Integer, Integer> f = x -> x*x;
- F. None of the above

9. Which statements are true? (Choose all that apply.)

- A. The Consumer interface is good for printing out an existing value.
- B. The Supplier interface is good for printing out an existing value.
- C. The IntegerSupplier interface returns an int.
- D. The Predicate interface returns an int.
- E. The Function interface has a method named test().
- F. The Predicate interface has a method named test().

10. Which of the following can be inserted without causing a compilation error? (Choose all that apply.)

```
public void remove(List<Character> chars) {  
    char end = 'z';  
    Predicate<Character> predicate = c -> {  
        char start = 'a'; return start <= c && c <= end; };  
  
    // INSERT LINE HERE  
}
```

- A. char start = 'a';
- B. char c = 'x';
- C. chars = null;
- D. end = '1';
- E. None of the above

11. How many times is true printed out by this code?

```
import java.util.function.Predicate;
public class Fantasy {
    public static void scary(String animal) {
        var dino = s -> "dino".equals(animal);
        var dragon = s -> "dragon".equals(animal);
        var combined = dino.or(dragon);
        System.out.println(combined.test(animal));
    }
    public static void main(String[] args) {
        scary("dino");
        scary("dragon");
        scary("unicorn");
    }
}
```

- A. One.
- B. Two.
- C. Three.
- D. The code does not compile.
- E. A runtime exception is thrown.

Explanation: The code does not compile because the lambdas are assigned to var. The compiler does not have enough information to determine they are of type Predicate<String>.

12. What does the following code output?

```
Function<Integer, Integer> s = a -> a + 4;
Function<Integer, Integer> t = a -> a * 3;
Function<Integer, Integer> c = s.compose(t);
System.out.print(c.apply(1));
```

- A. 7
- B. 15
- C. The code does not compile because of the data types in the lambda expressions.
- D. The code does not compile because of the compose() call.
- E. The code does not compile for another reason.

13. Which is true of the following code?

```
int length = 3;

for (int i = 0; i<3; i++) {
    if (i%2 == 0) {
        Supplier<Integer> supplier = () -> length; // A
        System.out.println(supplier.get()); // B
    } else {
        int j = i;
        Supplier<Integer> supplier = () -> j; // C
        System.out.println(supplier.get()); // D
    }
}
```

- A. The first compiler error is on line A.
- B. The first compiler error is on line B.
- C. The first compiler error is on line C.
- D. The first compiler error is on line D.
- E. The code compiles successfully.

14. Which of the following are valid lambda expressions? (Choose all that apply.)

- A. (Wolf w, var c) -> 39
- B. (final Camel c) -> {}
- C. (a,b,c) -> {int b = 3; return 2;}
- D. (x,y) -> new RuntimeException()
- E. (var y) -> return 0;
- F. () -> {float r}
- G. (Cat a, b) -> {}

15. Which lambda expression, when entered into the blank line in the following code, causes the program to print hahaha? (Choose all that apply.)

```
import java.util.function.Predicate;
public class Hyena {
    private int age = 1;
    public static void main(String[] args) {
        var p = new Hyena();
        double height = 10;
        int age = 1;
        testLaugh(p, _____);
        age = 2;
    }
    static void testLaugh(Hyena panda, Predicate<Hyena> joke) {
        var r = joke.test(panda) ? "hahaha" : "silence";
        System.out.print(r);
    }
}
```

- A. var -> p.age <= 10
- B. shenzi -> age==1
- C. p -> true
- D. age==1
- E. shenzi -> age==2
- F. h -> h.age < 5
- G. None of the above, as the code does not compile

16. Which of the following can be inserted without causing a compilation error?

```
public void remove(List<Character> chars) {
    char end = 'z';

    // INSERT LINE HERE
}
```



```
Predicate<Character> predicate = c -> {
    char start = 'a'; return start <= c && c <= end; };
}
```

- A. char start = 'a';
- B. char c = 'x';
- C. chars = null;
- D. end = '1';
- E. None of the above

17. What is the result of running the following class?

```
1: import java.util.function.*;
2:
3: public class Panda {
4:     int age;
5:     public static void main(String[] args) {
6:         Panda p1 = new Panda();
7:         p1.age = 1;
8:         check(p1, p -> {p.age < 5});
9:     }
10:    private static void check(Panda panda,
11:        Predicate<Panda> pred) {
12:        String result = pred.test(panda)
13:            ? "match" : "not match";
14:        System.out.print(result);
15:    } }
```

- A. match
- B. not match
- C. Compiler error on line 8.
- D. Compiler error on line 10.
- E. Compiler error on line 12.
- F. A runtime exception is thrown.

Explanation: C. Line 8 uses braces around the body. This means the return keyword and semicolon are required. Since the code doesn't compile, option C is the answer.

18. Which functional interfaces complete the following code? For line 7, assume m and n are instances of functional interfaces that exist and have the same type as y. (Choose three.)

```
6: _____ x = String::new;
7: _____ y = m.andThen(n);
8: _____ z = a -> a + a;
```

- A. BinaryConsumer<String, String>
- B. BiConsumer<String, String>
- C. BinaryFunction<String, String>
- D. BiFunction<String, String>
- E. Predicate<String>
- F. Supplier<String>

G. UnaryOperator<String>

H. UnaryOperator<String, String>

19. Which of the following compiles and prints out the entire set?

```
Set<?> set = Set.of("lion", "tiger", "bear");
var s = Set.copyOf(set);
Consumer<Object> consumer = _____;
s.forEach(consumer);
```

A. () -> System.out.println(s)

B. s -> System.out.println(s)

C. (s) -> System.out.println(s)

D. System.out.println(s)

E. System::out::println

F. System.out::println

20. Which lambda can replace the new Sloth() call in the main() method and produce the same output at runtime?

```
import java.util.List;
interface Yawn {
    String yawn(double d, List<Integer> time);
}
class Sloth implements Yawn {
    public String yawn(double zzz, List<Integer> time) {
        return "Sleep: " + zzz;
    }
}
public class Vet {
    public static String takeNap(Yawn y) {
        return y.yawn(10, null);
    }
    public static void main(String... unused) {
        System.out.print(takeNap(new Sloth()));
    }
}
```

A. (z,f) -> { String x = ""; return "Sleep: " + x }

B. (t,s) -> { String t = ""; return "Sleep: " + t; }

C. (w,q) -> {"Sleep: " + w}

D. (e,u) -> { String g = ""; "Sleep: " + e }

E. (a,b) -> "Sleep: " + (double)(b==null ? a : a)

F. (r,k) -> { String g = ""; return "Sleep: "; }

G. None of the above, as the program does not compile

21. Which of the following are valid functional interfaces? (Choose all that apply.)

```
public interface Transport {
    public int go();
    public boolean equals(Object o);
}
```

```
public abstract class Car {  
    public abstract Object swim(double speed, int duration);  
}  
  
public interface Locomotive extends Train {  
    public int getSpeed();  
}  
  
public interface Train extends Transport {}  
  
abstract interface Spaceship extends Transport {  
    default int blastOff();  
}  
  
public interface Boat {  
    int hashCode();  
    int hashCode(String input);  
}
```

- A. Boat
- B. Car
- C. Locomotive
- D. Spaceship
- E. Transport
- F. Train
- G. None of these is a valid functional interface.

Chapter 9: Collections and Generics

1. Suppose you need to display a collection of products for sale, which may contain duplicates. Additionally, you have a collection of sales that you need to track, sorted by the natural order of the sale ID, and you need to retrieve the text of each. Which two of the following classes best suit your needs for each of these scenarios? (Choose two.)

- A. ArrayList
- B. HashMap
- C. HashSet
- D. LinkedList
- E. SequencedTreeSet
- F. TreeMap

2. Which of the following are true? (Choose all that apply.)

```
12: List<?> q = List.of("mouse", "parrot");
13: var v = List.of("mouse", "parrot");
14:
15: q.removeIf(String::isEmpty);
16: q.removeIf(s -> s.length() == 4);
17: v.removeIf(String::isEmpty);
18: v.removeIf(s -> s.length() == 4);
```

- A. This code compiles and runs without error.
- B. Exactly one of these lines contains a compiler error.
- C. Exactly two of these lines contain a compiler error.
- D. Exactly three of these lines contain a compiler error.
- E. Exactly four of these lines contain a compiler error.
- F. If any lines with compiler errors are removed, this code runs without throwing an exception.
- G. If any lines with compiler errors are removed, this code throws an exception.

3. What is the result of the following statements?

```
3: var greetings = new ArrayDeque<String>();
4: greetings.offerLast("hello");
5: greetings.offerLast("hi");
6: greetings.offerFirst("ola");
7: greetings.pop();
8: greetings.peek();
9: while (greetings.peek() != null)
10:     System.out.print(greetings.pop());
```

- A. hello
- B. hellohi
- C. hellohiola
- D. hiola
- E. The code does not compile.
- F. An exception is thrown.

4. Which of these statements compile? (Choose all that apply.)

- A. `HashSet<Number> hs = new HashSet<Integer>();`
- B. `HashSet<? super ClassCastException> set = new HashSet<Exception>();`
- C. `List<> list = new ArrayList<String>();`
- D. `List<Object> values = new HashSet<Object>();`
- E. `List<Object> objects = new ArrayList<? extends Object>();`
- F. `Map<String, ? extends Number> hm = new HashMap<String, Integer>();`

5. What is the result of the following code?

```
1: public record Hello<T>(T t) {  
2:     public Hello(T t) { this.t = t; }  
3:     private <T> void println(T message) {  
4:         System.out.print(t + "-" + message);  
5:     }  
6:     public static void main(String[] args) {  
7:         new Hello<String>("hi").println(1);  
8:         new Hello("hola").println(true);  
9:     } }
```

- A. hi followed by a runtime exception.
- B. hi-1hola-true
- C. The first compiler error is on line 1.
- D. The first compiler error is on line 3.
- E. The first compiler error is on line 8.
- F. The first compiler error is on another line.

6. Which of the following can fill in the blank to print [7, 5, 3]? (Choose all that apply.)

```
8: public record Platypus(String name, int beakLength) {  
9:     @Override public String toString() {return "" + beakLength;}  
10:  
11:     public static void main(String[] args) {  
12:         Platypus p1 = new Platypus("Paula", 3);  
13:         Platypus p2 = new Platypus("Peter", 5);  
14:         Platypus p3 = new Platypus("Peter", 7);  
15:  
16:         List<Platypus> list = Arrays.asList(p1, p2, p3);  
17:  
18:         Collections.sort(list, Comparator.comparing_____);  
19:  
20:         System.out.println(list);  
21:     }  
22: }
```

A.

`(Platypus::beakLength)`

B.

```
(Platypus::beakLength).reversed()
```

C.

```
(Platypus::name)  
  .thenComparing(Platypus::beakLength)
```

D.

```
(Platypus::name)  
  .thenComparing(  
    Comparator.comparing(Platypus::beakLength)  
    .reversed())
```

E.

```
(Platypus::name)  
  .thenComparingNumber(Platypus::beakLength)  
  .reversed()
```

F.

```
(Platypus::name)  
  .thenComparingInt(Platypus::beakLength)  
  .reversed()
```

7. Which of the following method signatures are valid overrides of the hairy() method in the Alpaca class?
(Choose all that apply.)

```
import java.util.*;  
  
public class Alpaca {  
    public List<String> hairy(List<String> list) { return null; }  
}
```

A.

```
public List<String> hairy(List<CharSequence> list) { return null; }
```

B.

```
public List<String> hairy(List<String> list) { return null; }
```

C.

```
public List<String> hairy(List<Integer> list) { return null; }
```

D.

```
public List<CharSequence> hairy(List<String> list) { return null; }
```

E.

```
public Object hairy(List<String> list) { return null; }
```

F.

```
public ArrayList<String> hairy(List<String> list) { return null; }
```

8. Which of the following fills in the blank, allowing the code to compile and run without issue?

```
11: SequencedCollection<String> animals = new _____<>();
12: animals.addFirst("lions");
13: animals.addLast("tigers");
14: for(var a : animals)
15:     System.out.println(a);
16: System.out.println(animals.get(0));
```

- A. HashSet
- B. LinkedList
- C. TreeSetMap
- D. HashMap
- E. None of the above

Explanation: There is no get(int) method defined in SequencedCollection, meaning line 16 does not compile.

9. What is the result of the following program?

```
3: public class MyComparator implements Comparator<String> {
4:     public int compare(String a, String b) {
5:         return b.toLowerCase().compareTo(a.toLowerCase());
6:     }
7:     public static void main(String[] args) {
8:         String[] values = { "123", "Abb", "aab" };
9:         Arrays.sort(values, new MyComparator());
10:        for (var s: values)
11:            System.out.print(s + " ");
12:    }
13: }
```

- A. Abb aab 123
- B. aab Abb 123
- C. 123 Abb aab
- D. 123 aab Abb
- E. The code does not compile.
- F. A runtime exception is thrown.

10. Which of these statements can fill in the blank so that the Helper class compiles successfully? (Choose all that apply.)

```
2: public class Helper {
3:     public static <U extends Exception>
4:         void printException(U u) {
```

```

5:
6:     System.out.println(u.getMessage());
7: }
8: public static void main(String[] args) {
9:     Helper. _____;
10: } }

```

- A. `printException(new FileNotFoundException("A"))`
- B. `printException(new Exception("B"))`
- C. `<Throwable>printException(new Exception("C"))`
- D. `<NullPointerException>printException(new NullPointerException("D"))`
- E. `printException(new Throwable("E"))`

11. Which of the following will compile when filling in the blank? (Choose all that apply.)

```

var list = List.of(1, 2, 3);
var set = Set.of(1, 2, 3);
var map = Map.of(1, 2, 3, 4);

_____.forEach(System.out::println);

```

- A. `list`
- B. `set`
- C. `map`
- D. `map.keys()`
- E. `map.keySet()`
- F. `map.values()`
- G. `map.valueSet()`

12. Which of these statements can fill in the blank so that the Wildcard class compiles successfully? (Choose all that apply.)

```

3: public class Wildcard {
4:     public void showSize(List<?> list) {
5:         System.out.println(list.size());
6:     }
7:     public static void main(String[] args) {
8:         Wildcard card = new Wildcard();
9:         _____;
10:        card.showSize(list);
11:    } }

```

- A. `List<?> list = new HashSet<String>()`
- B. `ArrayList<? super Date> list = new ArrayList<Date>()`
- C. `List<?> list = new ArrayList<?>()`
- D. `List<Exception> list = new LinkedList<java.io.IOException>()`
- E. `ArrayList<? extends Number> list = new ArrayList<Integer>()`
- F. None of the above

13. What is the result of the following program?

```
3: public record Sorted(int num, String text)
4:     implements Comparable<Sorted>, Comparator<Sorted> {
5:
6:     public String toString() { return "" + num; }
7:     public int compareTo(Sorted s) {
8:         return text.compareTo(s.text);
9:     }
10:    public int compare(Sorted s1, Sorted s2) {
11:        return s1.num - s2.num;
12:    }
13:    public static void main(String[] args) {
14:        var s1 = new Sorted(88, "a");
15:        var s2 = new Sorted(55, "b");
16:        SequencedSet<Sorted> t1 = new TreeSet<Sorted>();
17:        t1.add(s1); t1.add(s2);
18:        var t2 = new TreeSet<Sorted>(s1);
19:        t2.add(s1); t2.add(s2);
20:        System.out.println(t1 + " " + t2);
21:    } }
```

- A. [55, 88] [55, 88]
- B. [55, 88] [88, 55]
- C. [88, 55] [55, 88]
- D. [88, 55] [88, 55]
- E. The code does not compile.
- F. A runtime exception is thrown.

14. What is the result of the following code?

```
Comparator<Integer> c1 = (o1, o2) -> o2 - o1;
Comparator<Integer> c2 = Comparator.naturalOrder();
Comparator<Integer> c3 = Comparator.reverseOrder();

var list = Arrays.asList(5, 4, 7, 2);
Collections.sort(list, _____);
Collections.reverse(list);
Collections.reverse(list);
System.out.println(Collections.binarySearch(list, 2));
```

- A. One or more of the comparators can fill in the blank so that the code prints 0.
- B. One or more of the comparators can fill in the blank so that the code prints 1.
- C. One or more of the comparators can fill in the blank so that the code prints 2.
- D. The result is undefined regardless of which comparator is used.
- E. A runtime exception is thrown regardless of which comparator is used.
- F. The code does not compile.

15. Which of the following lines can be inserted to make the code compile? (Choose all that apply.)

```
class W {}
class X extends W {}
class Y extends X {}
class Z<Y> {
    // INSERT CODE HERE
}
```

- A. `W w1 = new W();`
- B. `W w2 = new X();`
- C. `W w3 = new Y();`
- D. `Y y1 = new W();`
- E. `Y y2 = new X();`
- F. `Y y3 = new Y();`

Explanation: Y is both a class and a type parameter. This means that within the class Z, when we refer to Y, it uses the type parameter. All of the choices that mention class Y are incorrect because it no longer means the class Y.

16. Which options are true of the following code? (Choose all that apply.)

```
_____ q = new LinkedList<>();
var u = Collections.unmodifiableCollection(q);
q.add(10);
q.add(12);
q.remove(1);
System.out.print(u);
```

- A. If we fill in the blank with `List<Integer>`, the output is `[10]`.
- B. If we fill in the blank with `Queue<Integer>`, the output is `[10]`.
- C. If we fill in the blank with `var`, the output is `[10]`.
- D. One or more of the scenarios does not compile.
- E. One or more of the scenarios throws a runtime exception.

17. What is the result of the following code?

```
4: Map m = new HashMap();
5: m.put(123, "456");
6: m.put("abc", "def");
7: System.out.println(m.contains("123"));
```

- A. `false`
- B. `true`
- C. Compiler error on line 4.
- D. Compiler error on line 5.
- E. Compiler error on line 7.
- F. A runtime exception is thrown.

Explanation: Map does not have a `contains()` method. It has `containsKey()` and `containsValue()` instead.

18. What is the result of the following code? (Choose all that apply.)

```
48: var map = Map.of(1,2, 3, 6);
49: var list = List.copyOf(map.entrySet());
50:
51: List<Integer> one = List.of(8, 16, 2);
52: var copy = List.copyOf(one);
53: var copyOfCopy = List.copyOf(copy);
54: var thirdCopy = new ArrayList<>(copyOfCopy);
55:
56: list.replaceAll(x -> x * 2);
57: one.replaceAll(x -> x * 2);
58: thirdCopy.replaceAll(x -> x * 2);
59:
60: System.out.println(thirdCopy);
```

- A. One line fails to compile.
- B. Two lines fail to compile.
- C. Three lines fail to compile.
- D. The code compiles but throws an exception at runtime.
- E. If any lines with compiler errors are removed, the code throws an exception at runtime.
- F. If any lines with compiler errors are removed, the code prints [16, 32, 4].
- G. The code compiles and prints [16, 32, 4] without any changes.

Explanation: Line 48 is a `Map<Integer, Integer>`. Line 49 builds a `List` out of a `Set` of `Entry` objects, giving us `List<Entry<Integer, Integer>>`. This causes a compiler error on line 56. Lines 51–54 are all of type `List<Integer>`. The first three are immutable, and the one on line 54 is mutable. This means line 57 throws an `UnsupportedOperationException` since we attempt to modify the list.

19. What code change is needed to make the method compile, assuming there is no class named `T`?

```
public static T identity(T t) {
    return t;
}
```

- A. Add `<T>` after the `public` keyword.
- B. Add `<T>` after the `static` keyword.
- C. Add `<T>` after `T`.
- D. Add `<?>` after the `public` keyword.
- E. Add `<?>` after the `static` keyword.
- F. No change is required. The code already compiles.

20. Assuming keys are printed in order, what is the result of the following?

```
var map = new HashMap<Integer, Integer>();
map.put(1, 10);
map.put(2, 20);
map.put(3, null);
map.merge(1, 3, (a,b) -> a + b);
map.merge(3, 3, (a,b) -> a + b);
System.out.println(map);
```

- A. {1=10, 2=20}
- B. {1=10, 2=20, 3=null}
- C. {1=10, 2=20, 3=3}
- D. {1=13, 2=20}
- E. {1=13, 2=20, 3=null}
- F. {1=13, 2=20, 3=3}
- G. The code does not compile.
- H. An exception is thrown.

21. Which of the following statements are true? (Choose all that apply.)

- A. Comparable is in the java.util package.
- B. Comparator is in the java.util package.
- C. compare() is in the Comparable interface.
- D. compare() is in the Comparator interface.
- E. compare() takes one method parameter.
- F. compare() takes two method parameters.

22. What is the output of the following code snippet?

```
21: SequencedMap<Integer, String> cats = new TreeMap<>();
22: cats.put(3, "Snowball");
23: cats.put(2, "Sugar");
24: cats.put(1, "Minnie Mouse");
25: cats.pollFirstEntry();
26: var id = cats.lastEntry().getKey();
27: cats.pollFirstEntry();
28: System.out.print(cats.firstEntry().getValue());
```

- A. Minnie Mouse
- B. Snowball
- C. Sugar
- D. The code does not compile.
- E. The code compiles, but an exception is thrown at runtime.

23. What is the output of the following code snippet?

```
var fishes = new TreeSet<String>();
fishes.add("Koi");
fishes.addFirst("clown");
fishes.add("carp");
for(var fish : fishes)
    System.out.print(fish + ", ");
```

- A. carp, clown, Koi,
- B. carp, Koi, clown,
- C. clown, carp, Koi,
- D. clown, Koi, carp,
- E. Koi, carp, clown,
- F. Koi, clown, carp,
- G. The code does not compile.
- H. The code compiles but throws an exception at runtime.

Chapter 10: Streams

1. What could be the output of the following?

```
var stream = Stream.iterate("", (s) -> s + "1");  
System.out.println(stream.limit(2).map(x -> x + "2"));
```

- A. 12112
- B. 212
- C. 212112
- D. java.util.stream.ReferencePipeline\$3@4517d9a3
- E. The code does not compile.
- F. An exception is thrown.
- G. The code hangs.

Explanation: No terminal operation is called, so the stream never executes.

2. What could be the output of the following?

```
Predicate<String> predicate = s -> s.startsWith("g");  
var stream1 = Stream.generate(() -> "growl!");  
var stream2 = Stream.generate(() -> "growl!");  
var b1 = stream1.anyMatch(predicate);  
var b2 = stream2.allMatch(predicate);  
System.out.println(b1 + " " + b2);
```

- A. true false
- B. true true
- C. java.util.stream.ReferencePipeline\$3@4517d9a3
- D. The code does not compile.
- E. An exception is thrown.
- F. The code hangs.

Explanation: Both streams created in this code snippet are infinite streams. The variable b1 is set to true since anyMatch() terminates. Even though the stream is infinite, Java finds a match on the first element and stops looking. However, when allMatch() runs, it needs to keep going until the end of the stream since it keeps finding matches.

3. What could be the output of the following?

```
Predicate<String> predicate = s -> s.length() > 3;  
var stream = Stream.iterate("-",  
    s -> ! s.isEmpty(), (s) -> s + s);  
var b1 = stream.noneMatch(predicate);  
var b2 = stream.anyMatch(predicate);  
System.out.println(b1 + " " + b2);
```

- A. false false
- B. false true
- C. java.util.stream.ReferencePipeline\$3@4517d9a3
- D. The code does not compile.
- E. An exception is thrown.

F. The code hangs.

Explanation: An infinite stream is generated where each element is twice as long as the previous one. While this code uses the three-parameter `iterate()` method, the condition is never false. The variable `b1` is set to false because Java finds an element that matches when it gets to the element of length 4. However, the next line tries to operate on the same stream. Since streams can be used only once, this throws an exception that the “stream has already been operated upon or closed”.

4. Which are true statements about terminal operations in a stream that runs successfully? (Choose all that apply.)

A. At most one terminal operation can exist in a stream pipeline.

B. Terminal operations are a required part of the stream pipeline in order to get a result.

C. Terminal operations have `Stream` as the return type.

D. The `peek()` method is an example of a terminal operation.

E. The referenced `Stream` may be used after calling a terminal operation.

5. Which of the following sets result to 8.0? (Choose all that apply.)

A.

```
double result = LongStream.of(6L, 8L, 10L)
    .mapToInt(x -> (int) x)
    .collect(Collectors.groupingBy(x -> x))
    .keySet()
    .stream()
    .collect(Collectors.averagingInt(x -> x));
```

B.

```
double result = LongStream.of(6L, 8L, 10L)
    .mapToInt(x -> x)
    .boxed()
    .collect(Collectors.groupingBy(x -> x))
    .keySet()
    .stream()
    .collect(Collectors.averagingInt(x -> x));
```

C.

```
double result = LongStream.of(6L, 8L, 10L)
    .mapToInt(x -> (int) x)
    .boxed()
    .collect(Collectors.groupingBy(x -> x))
    .keySet()
    .stream()
    .collect(Collectors.averagingInt(x -> x));
```

D.

```
double result = LongStream.of(6L, 8L, 10L)
    .mapToInt(x -> (int) x)
    .collect(Collectors.groupingBy(x -> x, Collectors.toSet()))
    .keySet()
    .stream()
    .collect(Collectors.averagingInt(x -> x));
```

E.

```
double result = LongStream.of(6L, 8L, 10L)
    .mapToInt(x -> x)
    .boxed()
    .collect(Collectors.groupingBy(x -> x, Collectors.toSet()))
    .keySet()
    .stream()
    .collect(Collectors.averagingInt(x -> x));
```

F.

```
double result = LongStream.of(6L, 8L, 10L)
    .mapToInt(x -> (int) x)
    .boxed()
    .collect(Collectors.groupingBy(x -> x, Collectors.toSet()))
    .keySet()
    .stream()
    .collect(Collectors.averagingInt(x -> x));
```

Explanation: options B and E are incorrect because they do not cast the long to an int, resulting in a compiler error on the mapToInt() calls. Options A and D are incorrect because they are missing boxed() before the collect() call. Since groupingBy() is creating a Collection, we need a nonprimitive Stream.

6. Which of the following methods can fill in the blank so that the code prints out false?

```
var s = Stream.generate(() -> "meow");
var match = s._____ (String::isEmpty);
System.out.println(match);
```

- A. Only allMatch
- B. Only anyMatch
- C. Only noneMatch
- D. Both allMatch and anyMatch
- E. Both allMatch and noneMatch
- F. None of the above

Explanation: The anyMatch() and noneMatch() methods run infinitely. The stream has no way to know that a match won't show up later. Option A is correct because only allMatch() makes it safe to return false as soon as one element passes through the stream that doesn't match.

7. We have a method that returns a sorted list without changing the original. We want to rewrite it. Which of the following pairs can fill in the blanks in `refactored()` to do the same with streams?

```
private static List<String> sort(List<String> list) {
    var copy = new ArrayList<String>(list);
    Collections.sort(copy, (a, b) -> b.compareTo(a));
    return copy;
}

private static List<String> refactored(List<String> list) {
    return list.stream()
        ._____((a, b) -> b.compareTo(a))
        ._____;
}
```

- A. `compare` and `toList()`
- B. `compare` and `sort()`
- C. `compareTo` and `toList()`
- D. `compareTo` and `sort()`
- E. `sorted` and `collect()`
- F. `sorted` and `collect(Collectors.toList())`

Explanation: There is no `Stream<T>` method called `compare()` or `compareTo()`, so options A through D can be eliminated. The `sorted()` method is correct to use in a stream pipeline to return a sorted `Stream` and fills in the first blank. Either `toList()` or `collect(Collectors.toList())` can fill in the second blank.

8. Which of the following are true given this declaration? (Choose all that apply.)

```
var is = IntStream.empty();
```

- A. `is.average()` returns the type `int`.
- B. `is.average()` returns the type `OptionalInt`.
- C. `is.findAny()` returns the type `int`.
- D. `is.findAny()` returns the type `OptionalInt`.
- E. `is.sum()` returns the type `int`.
- F. `is.sum()` returns the type `OptionalInt`.

Explanation: The `average()` method returns an `OptionalDouble` since averages of any type can result in a fraction. The `findAny()` method returns an `OptionalInt` because there might not be any elements to find. The `sum()` method returns an `int` rather than an `OptionalInt` because the sum of an empty stream is zero.

9. Which of the following can we add after line 6 for the code to run without error and not produce any output? (Choose all that apply.)

```
4: var stream = LongStream.of(1, 2, 3);
5: var opt = stream.map(n -> n * 10)
6:     .filter(n -> n < 5).findFirst();
```

A.

```
if (opt.isPresent())
    System.out.println(opt.get());
```


B.

```
if (opt.isPresent())  
    System.out.println(opt.getAsLong());
```

C.

```
opt.ifPresent(System.out::println);
```

D.

```
opt.ifPresent(System.out::println);
```

- E. None of these; the code does not compile.
F. None of these; line 6 throws an exception at runtime.

10. Given the four statements (L, M, N, O), select the order that would cause the code to output 10 lines.

```
Stream.generate(() -> "1")  
  L: .filter(x -> x.length() > 1)  
  M: .forEach(System.out::println)  
  N: .limit(10)  
  O: .peek(System.out::println);
```

- A. L, N
B. L, N, O
C. L, N, M
D. L, N, M, O
E. L, O, M
F. N, M
G. N, O

Explanation: Only one of the method calls, `forEach()`, is a terminal operation, so any answer in which M is not the last line will not execute the pipeline.

11. What changes need to be made together for this code to print the string 12345? (Choose all that apply.)

```
Stream.iterate(1, x -> x++)  
  .limit(5).map(x -> x)  
  .collect(Collectors.joining());
```

- A. Changing `Collectors.joining()` to `Collectors.joining(",")`
B. Changing `map(x -> x)` to `map(x -> "" + x)`
C. Changing `x -> x++` to `x -> ++x`
D. Adding `.forEach(System.out::print)` after the call to `collect()`
E. Wrapping the entire line in a `System.out.print` statement
F. None of the above; the code already prints 12345

Explanation: As written, the code doesn't compile because the `Collectors.joining()` expects to get a `Stream<String>`. Option B fixes this, at which point nothing is output because the collector creates a String without outputting the result. Option E fixes this and causes the output to be 11111. Since the post-increment operator is used, the stream contains an infinite number of the character 1. Option C fixes this and causes the stream to contain increasing numbers.

12. Which is true of the following code?

```
Set<String> birds = Set.of("oriole", "flamingo");
Stream.concat(birds.stream(), birds.stream(), birds.stream())
    .sorted()           // line X
    .distinct()
    .findAny()
    .ifPresent(System.out::println);
```

- A. It is guaranteed to print flamingo as is and when line X is removed.
- B. It is guaranteed to print oriole as is and when line X is removed.
- C. It is guaranteed to print flamingo as is, but not when line X is removed.
- D. It is guaranteed to print oriole as is, but not when line X is removed.
- E. The output may vary as is.
- F. The code does not compile.
- G. It throws an exception because the same list is used as the source for multiple streams.

Explanation: The code does not compile because Stream.concat() takes two parameters, not the three provided.

13. Which of the following is true?

```
List<Integer> x1 = List.of(1, 2, 3);
List<Integer> x2 = List.of(4, 5, 6);
List<Integer> x3 = List.of();
Stream.of(x1, x2, x3).map(x -> x + 1)
    .flatMap(x -> x.stream())
    .forEach(System.out::print);
```

- A. The code compiles and prints 123456.
- B. The code compiles and prints 234567.
- C. The code compiles but does not print anything.
- D. The code compiles but prints stream references.
- E. The code runs infinitely.
- F. The code does not compile.
- G. The code throws an exception.

Explanation: If the map() and flatMap() calls were reversed, option B would be correct. In this case, the Stream created from the source is of type Stream<List>. Trying to use the addition operator (+) on a List is not supported in Java.

14. Which of the following are true? (Choose all that apply.)

```
4: Stream<Integer> s = Stream.of(1);
5: IntStream is = s.boxed();
6: DoubleStream ds = s.mapToDouble(x -> x);
7: Stream<Integer> s2 = ds.mapToInt(x -> x);
8: s2.forEach(System.out::print);
```

- A. Line 4 causes a compiler error.
- B. Line 5 causes a compiler error.
- C. Line 6 causes a compiler error.
- D. Line 7 causes a compiler error.

- E. Line 8 causes a compiler error.
- F. The code compiles but throws an exception at runtime.
- G. The code compiles and prints 1.

Explanation: `boxed()` is available only on primitive streams like `IntStream`, not `Stream<Integer>`. Line 7 does not compile for two reasons. First, converting from a double to an int would require an explicit cast. Also, `mapToInt()` returns an `IntStream`, so the data type of `s2` is incorrect.

15. Given the generic type `String`, the `partitioningBy()` collector creates a `Map<Boolean, List<String>>` when passed to `collect()` by default. When a downstream collector is passed to `partitioningBy()`, which return types can be created? (Choose all that apply.)
- A. `Map<boolean, List<String>>`
 - B. `Map<Boolean, List<String>>`
 - C. `Map<Boolean, Map<String>>`
 - D. `Map<Boolean, Set<String>>`
 - E. `Map<Long, TreeSet<String>>`
 - F. None of the above
16. Which of the following statements are true about this code? (Choose all that apply.)

```
20: Predicate<String> empty = String::isEmpty;
21: Predicate<String> notEmpty = empty.negate();
22:
23: var result = Stream.generate(() -> "")
24:     .limit(10)
25:     .filter(notEmpty)
26:     .collect(Collectors.groupingBy(k -> k))
27:     .entrySet()
28:     .stream()
29:     .map(Entry::getValue)
30:     .flatMap(Collection::stream)
31:     .collect(Collectors.partitioningBy(notEmpty));
32: System.out.println(result);
```

- A. It outputs `{}`.
 - B. It outputs `{false=[], true=[]}`.
 - C. If we changed line 31 from `partitioningBy(notEmpty)` to `groupingBy(n -> n)`, it would output `{}`.
 - D. If we changed line 31 from `partitioningBy(notEmpty)` to `groupingBy(n -> n)`, it would output `{false=[], true=[]}`.
 - E. The code does not compile.
 - F. The code compiles but does not terminate at runtime.
17. What is the result of the following?

```
var s = DoubleStream.of(1.2, 2.4);
s.peek(System.out::println).filter(x -> x > 2).count();
```

- A. 1
- B. 2
- C. 2.4
- D. 1.2 and 2.4
- E. There is no output.

- F. The code does not compile.
- G. An exception is thrown.

Explanation: The terminal operation is count(). Since there is a terminal operation, the intermediate operations run. The peek() operation comes before the filter(), so both numbers are printed.

18. What is the output of the following?

```
11: public class Paging {
12:     record Sesame(String name, boolean human) {
13:         @Override public String toString() {
14:             return name();
15:         }
16:     }
17:     record Page(List<Sesame> list, long count) {}
18:
19:     public static void main(String[] args) {
20:         var monsters = Stream.of(new Sesame("Elmo", false));
21:         var people = Stream.of(new Sesame("Abby", true));
22:         printPage(monsters, people);
23:     }
24:
25:     private static void printPage(Stream<Sesame> monsters,
26:         Stream<Sesame> people) {
27:         Page page = Stream.concat(monsters, people)
28:             .collect(Collectors.teeing(
29:                 Collectors.filtering(s -> s.name().startsWith("E"),
30:                     Collectors.toList()),
31:                 Collectors.counting(),
32:                 (l, c) -> new Page(l, c)));
33:         System.out.println(page);
34:     } }
```

- A. Page[list=[Abby], count=1]
- B. Page[list=[Abby], count=2]
- C. Page[list=[Elmo], count=1]
- D. Page[list=[Elmo], count=2]**
- E. The code does not compile due to Stream.concat().
- F. The code does not compile due to Collectors.teeing().
- G. The code does not compile for another reason.

Explanation: Since line 29 filters by names starting with E, that rules out options A and B. Finally, line 31 counts the entire list, which is of size 2, giving us option D as the answer.

19. What is the simplest way of rewriting this code?

```
List<Integer> x = IntStream.range(1, 6)
    .mapToObj(i -> i)
    .collect(Collectors.toList());
x.forEach(System.out::println);
```

A.

```
IntStream.range(1, 6);
```

B.

```
IntStream.range(1, 6)
    .forEach(System.out::println);
```

C.

```
IntStream.range(1, 6)
```

D.

```
.mapToObj(i -> i)
    .forEach(System.out::println);
```

E. None of the above is equivalent.

F. The provided code does not compile.

20. Which of the following throw an exception when an Optional is empty? (Choose all that apply.)

A. `opt.orElse("")`;

B. `opt.orElseGet(() -> "")`;

C. `opt.orElseThrow()`;

D. `opt.orElseThrow(() -> throw new Exception())`;

E. `opt.orElseThrow(RuntimeException::new)`;

F. `opt.get()`;

G. `opt.get("")`;

Explanation: Option D looks correct but will compile only if the throw is removed. Remember, the `orElseThrow()` should get a lambda expression or method reference that returns an exception, not one that throws an exception.

21. What is the output of the following?

```
var spliterator = Stream.generate(() -> "x")
    .spliterator();

spliterator.tryAdvance(System.out::print);
var split = spliterator.trySplit();
split.tryAdvance(System.out::print);
```

A. x

B. xx

C. A long list of x's.

D. There is no output.

E. The code does not compile.

F. The code compiles but does not terminate at runtime.

Chapter 11: Exceptions and Localization

1. Which of the following can be inserted on line 8 to make this code compile? (Choose all that apply.)

```
7: public void whatHappensNext() throws IOException {
8:     // INSERT CODE HERE
9: }
```

- A. `System.out.println("it's ok");`
- B. `throw new Exception();`
- C. `throw new IllegalArgumentException();`
- D. `throw new java.io.IOException();`
- E. `throw new RuntimeException();`
- F. None of the above

Explanation: Unchecked exceptions can be thrown in any method. Option B is incorrect because a broader exception is not allowed.

2. Which statement about the following class is correct?

```
1: class Problem extends Exception {
2:     public Problem() {}
3: }
4: class YesProblem extends Problem {}
5: public class MyDatabase {
6:     public static void connectToDatabase() throw Problem {
7:         throws new YesProblem();
8:     }
9:     public static void main(String[] c) throw Exception {
10:         connectToDatabase();
11:     }
12: }
```

- A. The code compiles and prints a stack trace for YesProblem at runtime.
- B. The code compiles and prints a stack trace for Problem at runtime.
- C. The code does not compile because Problem defines a constructor.
- D. The code does not compile because YesProblem does not define a constructor.
- E. The code does not compile but would if Problem and YesProblem were switched on lines 6 and 7.
- F. None of the above.

Explanation: The code does not compile because the `throw` and `throws` keywords are incorrectly used on lines 6, 7, and 9.

3. Which of the following are common types to localize? (Choose all that apply.)

- A. Dates
- B. Lambda expressions
- C. Class names
- D. Currency
- E. Numbers
- F. Variable names

4. What is the output of the following snippet, assuming a and b are both 0?

```
3:  try {
4:      System.out.print(a / b);
5:  } catch (RuntimeException e) {
6:      System.out.print(-1);
7:  } catch (ArithmeticException e) {
8:      System.out.print(0);
9:  } finally {
10:     System.out.print("done");
11: }
```

- A. -1
- B. 0
- C. done-1
- D. done0
- E. The code does not compile.
- F. An uncaught exception is thrown.
- G. None of the above.

Explanation: The order of catch blocks is important because they're checked in the order they appear after the try block. Because ArithmeticException is a child class of RuntimeException, the catch block on line 7 is unreachable.

5. Assuming the current locale uses dollars (\$) and the following method is called with a double value of 100_102.2, which of the following values are printed? (Choose all that apply.)

```
public void print(double t) {

    System.out.print(NumberFormat.getCompactNumberInstance().format(t));

    System.out.print(
        NumberFormat.getCompactNumberInstance(
            Locale.getDefault(), Style.SHORT).format(t));

    System.out.print(NumberFormat.getCurrencyInstance().format(t));
}
```

- A. 100
- B. \$100,000.00
- C. 100K
- D. 100 thousand
- E. 100M
- F. \$100,102.20
- G. None of the above

Explanation: When a CompactNumberFormat instance is requested without a style, it uses the SHORT style by default. This results in both of the first two statements printing 100K. Option F is also correct, as the full value is printed with a currency formatter.

6. What is the output of the following code?

```
LocalDate date = LocalDate.parse("2025-04-30",  
    DateTimeFormatter.ISO_LOCAL_DATE_TIME);  
System.out.println(date.getYear() + " "  
    + date.getMonth() + " " + date.getDayOfMonth());
```

- A. 2025 APRIL 2
- B. 2025 APRIL 30
- C. 2025 MAY 2
- D. The code does not compile.
- E. A runtime exception is thrown.

Explanation: A `LocalDate` does not have a time element. Therefore, a date/time formatter is not appropriate. If `ISO_LOCAL_DATE` were used, the code would print 2025 APRIL 30.

7. What does the following method print?

```
11: public void tryAgain(String s) {  
12:     try (FileReader r = null, p = new FileReader("")) {  
13:         System.out.print("X");  
14:         throw new IllegalArgumentException();  
15:     } catch (Exception s) {  
16:         System.out.print("A");  
17:         throw new FileNotFoundException();  
18:     } finally {  
19:         System.out.print("O");  
20:     }  
21: }
```

- A. XAO
- B. XOA
- C. One line of this method contains a compiler error.
- D. Two lines of this method contain compiler errors.
- E. Three or more lines of this method contain compiler errors.
- F. The code compiles, but a `NullPointerException` is thrown at runtime.
- G. None of the above.

Explanation: The first compiler error is on line 12 because each resource in a try-with-resources statement must have its own data type and be separated by a semicolon (;). Line 15 does not compile because the variable `s` is already declared in the method. Line 17 also does not compile. The `FileNotFoundException`, which inherits from `IOException` and `Exception`, is a checked exception, so it must be handled or declared by the method.

8. Assume that all of the files mentioned in the answer choices exist and define the same keys. Which one will be used to find the key in line 8?

```
6: Locale.setDefault(Locale.of("en", "US"));  
7: var b = ResourceBundle.getBundle("Dolphins");  
8: System.out.println(b.getString("name"));
```

- A. `Dolphins.properties`
- B. `Dolphins_US.properties`

C. Dolphins_en.properties

D. Whales.properties

E. Whales_en_US.properties

F. The code does not compile.

9. For what value of pattern will the following print <005.21> <008.49> <1,234.0>?

```
String pattern = "_____";  
var message = DoubleStream.of(5.21, 8.49, 1234)  
    .mapToObj(v -> new DecimalFormat(pattern).format(v))  
    .collect(Collectors.joining("> <"));  
System.out.println("<"+message+">");
```

A. ##.#

B. 0,000.0#

C. #,###.0

D. #,###,000.0#

E. The code does not compile regardless of what is placed in the blank.

F. None of the above.

10. Which scenario is the best use of an exception?

A. An element is not found when searching a list.

B. An unexpected parameter is passed into a method.

C. The computer caught fire.

D. You want to loop through a list.

E. You don't know how to code a method.

11. Which of the following exceptions must be handled or declared in the method in which they are thrown?
(Choose all that apply.)

```
class Apple extends RuntimeException {}  
class Orange extends Exception {}  
class Banana extends Error {}  
class Pear extends Apple {}  
class Tomato extends Orange {}  
class Peach extends Throwable {}
```

A. Apple

B. Orange

C. Banana

D. Pear

E. Tomato

F. Peach

Explanation: An exception that must be handled or declared is a checked exception. A checked exception inherits Exception but not RuntimeException. Option F is also correct, as a class that inherits Throwable but not RuntimeException or Error is also checked.

12. Which of the following changes, when made independently, would make this code compile? (Choose all that apply.)

```
1:  import java.io.*;
2:  public class StuckTurkeyCage implements AutoCloseable {
3:      public void close() throws IOException {
4:          throw new FileNotFoundException("Cage not closed");
5:      }
6:      public static void main(String[] args) {
7:          try (StuckTurkeyCage t = new StuckTurkeyCage()) {
8:              System.out.println("put turkeys in");
9:          }
10:     } }
```

- A. Remove throws IOException from the declaration on line 3.
 - B. Add throws Exception to the declaration on line 6.
 - C. Change line 9 to } catch (Exception e) {}.
 - D. Change line 9 to } finally {}.
 - E. The code compiles as is.
 - F. None of the above.
13. Which of the following are true statements about exception handling in Java? (Choose all that apply.)
- A. A traditional try statement without a catch block requires a finally block.
 - B. A traditional try statement without a finally block requires a catch block.
 - C. A traditional try statement with only one statement can omit the {}.
 - D. A try-with-resources statement without a catch block requires a finally block.
 - E. A try-with-resources statement without a finally block requires a catch block.
 - F. A try-with-resources statement with only one statement can omit the {}.

Explanation: A try-with-resources statement does not require a catch or finally block. A traditional try statement requires at least one of the two.

14. Assuming -g:vars is used when the code is compiled to include debug information, what is the output of the following code snippet?

```
var huey = (String)null;
Integer dewey = null;
Object louie = null;
if(louie == huey.substring(dewey.intValue())) {
    System.out.println("Quack!");
}
```

- A. A NullPointerException that does not include any variable names in the stack trace
- B. A NullPointerException naming huey in the stack trace
- C. A NullPointerException naming dewey in the stack trace
- D. A NullPointerException naming louie in the stack trace
- E. A NullPointerException naming huey and louie in the stack trace
- F. A NullPointerException naming huey and dewey in the stack trace
- G. None of the above

15. Which of the following, when inserted independently in the blank, use locale parameters that are properly formatted? (Choose all that apply.)

```
import java.util.Locale;
public class ReadMap implements AutoCloseable {
    private Locale locale;
    private boolean closed = false;
    @Override public void close() {
        System.out.println("Folding map");
        locale = null;
        closed = true;
    }
    public void open() {
        this.locale = _____;
    }
    public void use() {
        // Implementation omitted
    }
}
```

- A. `Locale.of("xM")`
- B. `Locale.of("MQ", "ks")`
- C. `Locale.of("qw")`
- D. `Locale.of("wp", "VW")`
- E. `Locale.create("zp")`
- F. `new Locale.Builder().setLanguage("yw").setRegion("PM")`
- G. The code does not compile regardless of what is placed in the blank.

16. Which of the following can be inserted into the blank to allow the code to compile and run without throwing an exception?

```
var f = DateTimeFormatter.ofPattern("hh o'clock");
System.out.println(f.format(_____.now()));
```

- A. `ZonedDateTime`
- B. `LocalDate`
- C. `LocalTimestamp`
- D. `LocalTime`
- E. The code does not compile regardless of what is placed in the blank.
- F. `None of the above`

17. Which of the following statements about resource bundles are correct? (Choose all that apply.)

- A. All keys must be in the same resource bundle to be used.
- B. A resource bundle is loaded by calling the new `ResourceBundle()` constructor.
- C. Resource bundle values are always read using the `Properties` class.
- D. Changing the default locale lasts for only a single run of the program.
- E. If a resource bundle for a specific locale is requested, then the resource bundle for the default locale will not be used.
- F. It is possible to use a resource bundle for a locale without specifying a default locale.

Explanation: Option A is incorrect because Java will look at parent bundles if a key is not found in a specified resource bundle. Option B is incorrect because resource bundles are loaded from static factory methods. Option C is incorrect, as resource bundle values are read from the ResourceBundle object directly. Option E is incorrect, as the resource bundle for the default locale may be used if there is no resource bundle for the specified locale (or its locale without a country code). Finally, option F is correct. The JVM will set a default locale automatically.

18. What is the output of the following code?

```
import java.io.*;
public class FamilyCar {
    static class Door implements AutoCloseable {
        public void close() {
            System.out.print("D");
        }
    }
    static class Window implements Closeable {
        public void close() {
            System.out.print("W");
            throw new RuntimeException();
        }
    }
    public static void main(String[] args) {
        var d = new Door();
        try (d; var w = new Window()) {
            System.out.print("T");
        } catch (Exception e) {
            System.out.print("E");
        } finally {
            System.out.print("F");
        }
    }
}
```

- A. TWF
- B. TWDF
- C. TWDEF
- D. TWF followed by an exception
- E. TWDF followed by an exception
- F. TWEF followed by an exception
- G. The code does not compile.

19. Suppose that we have the following three properties files and code. Which bundles are used on lines 8 and 9, respectively?

```
Dolphins.properties
name=The Dolphin
age=0

Dolphins_en.properties
name=Dolly
age=4

Dolphins_fr.properties
name=Dolly
```

```

5: var fr = Locale.of("fr");
6: Locale.setDefault(Locale.of("en", "US"));
7: var b = ResourceBundle.getBundle("Dolphins", fr);
8: b.getString("name");
9: b.getString("age");

```

- A. Dolphins.properties and Dolphins.properties
- B. Dolphins.properties and Dolphins_en.properties
- C. Dolphins_en.properties and Dolphins_en.properties
- D. Dolphins_fr.properties and Dolphins.properties**
- E. Dolphins_fr.properties and Dolphins_en.properties
- F. The code does not compile.
- G. None of the above.

20. What is printed by the following program?

```

1: public class DriveBus {
2:     public void go() {
3:         System.out.print("A");
4:         try {
5:             stop();
6:         } catch (ArithmeticException e) {
7:             System.out.print("B");
8:         } finally {
9:             System.out.print("C");
10:        }
11:        System.out.print("D");
12:    }
13:    public void stop() {
14:        System.out.print("E");
15:        Object x = null;
16:        x.toString();
17:        System.out.print("F");
18:    }
19:    public static void main(String n[]) {
20:        new DriveBus().go();
21:    } }

```

- A. AE
- B. AEBCD
- C. AEC
- D. AECD
- E. AE followed by a stack trace
- F. AEBCD followed by a stack trace
- G. AEC followed by a stack trace**
- H. A stack trace with no other output

21. Which change allows the following program to compile?

```
1: public class AhChoo {
2:     static class SneezeException extends Exception {}
3:     static class SniffleException extends SneezeException {}
4:     public static void main(String[] args) {
5:         try {
6:             throw new SneezeException();
7:         } catch (SneezeException | SniffleException e) {
8:         } finally {}
9:     } }
```

- A. Add throws SneezeException to the declaration on line 4.
- B. Add throws Throwable to the declaration on line 4.
- C. Change line 7 to } catch (SneezeException e) {.
- D. Change line 7 to } catch (SniffleException e) {.
- E. Remove line 7.
- F. The code compiles correctly as is.
- G. None of the above.

Explanation: The code does not compile because the multi-catch block on line 7 cannot catch both a superclass and a related subclass.

22. What is the output of the following code?

```
try {
    LocalDateTime book = LocalDateTime.of(2025, 4, 5, 12, 30, 20);
    System.out.print(book.format(DateTimeFormatter.ofPattern("m")));
    System.out.print(book.format(DateTimeFormatter.ofPattern("z")));
    System.out.print(DateTimeFormatter.ofPattern("y").format(book));
} catch (Throwable e) {}
```

- A. 4
- B. 30
- C. 402
- D. 3002
- E. 3002025
- F. 402025
- G. None of the above

23. Fill in the blank: A class that implements _____ may be in a try-with-resources statement.
(Choose all that apply.)

- A. AutoCloseable
- B. Resource
- C. Exception
- D. AutomaticResource
- E. Closeable
- F. RuntimeException
- G. Serializable

24. What is the output of the following program?

```
public class SnowStorm {
    static class WalkToSchool implements AutoCloseable {
        public void close() {
            throw new RuntimeException("flurry");
        }
    }
    public static void main(String[] args) {
        WalkToSchool walk1 = new WalkToSchool();
        try (walk1; WalkToSchool walk2 = new WalkToSchool()) {
            throw new RuntimeException("blizzard");
        } catch (Exception e) {
            System.out.println(e.getMessage()
                + " " + e.getSuppressed().length);
        }
        walk1 = null;
    }
}
```

- A. blizzard 0
- B. blizzard 1
- C. blizzard 2
- D. flurry 0
- E. flurry 1
- F. flurry 2
- G. None of the above

Explanation: The code does not compile because the resource walk1 is not final or effectively final and cannot be used in the declaration of a try-with-resources statement.

25. Assuming U.S. currency is in dollars (\$) and German currency is in euros (€), what is the output of the following program?

```
import java.text.NumberFormat;
import java.util.Locale;
import java.util.Locale.Category;
public record Wallet(double money) {
    private String openWallet() {
        Locale.setDefault(Category.DISPLAY,
            new Locale.Builder().setRegion("us").build());
        Locale.setDefault(Category.FORMAT,
            new Locale.Builder().setLanguage("en").build());
        return NumberFormat.getCurrencyInstance(Locale.GERMANY)
            .format(money);
    }
    public void printBalance() {
        System.out.println(openWallet());
    }
    public static void main(String... unused) {
        new Wallet(2.4).printBalance();
    }
}
```

- A. 2,40 €
- B. \$2.40
- C. 2.4
- D. The code does not compile.
- E. None of the above.

26. Which lines can fill in the blank to make the following code compile? (Choose all that apply.)

```
void rollOut() throws ClassCastException {}

public void transform(String c) {
    try {
        rollOut();
    } catch (IllegalArgumentException | _____) {
    }
}
```

- A. IOException a
- B. Error b
- C. NullPointerException c
- D. RuntimeException d
- E. NumberFormatException e
- F. ClassCastException f
- G. None of the above. The code contains a compiler error regardless of what is inserted into the blank.

Chapter 12: Modules

1. Which statement is true of the following module?

```
|---zoo
  |-- staff
    |-- Vet.java
```

- A. The directory structure shown is a valid module.
 - B. The directory structure would be a valid module if module.java were added directly underneath zoo/staff.
 - C. The directory structure would be a valid module if module.java were added directly underneath zoo.
 - D. The directory structure would be a valid module if module-info.java were added directly underneath zoo/staff.
 - E. The directory structure would be a valid module if module-info.java were added directly underneath zoo.
 - F. None of these changes would make this directory structure a valid module
2. Suppose module puppy depends on module dog and module dog depends on module animal. Fill in the blank so that code in module dog can access the animal.behavior package in module animal.

```
module animal {
    _____ animal.behavior;
}
```

- A. export
 - B. exports
 - C. require
 - D. requires
 - E. require transitive
 - F. requires transitive
 - G. None of the above
3. Fill in the blanks so this command to run the program is correct:

```
java
_____ modules
_____ zoo.animal.talks/zoo/animal/talks/Peacocks
```

- A. -d and -m
- B. -d and -p
- C. -m and -d
- D. -m and -p
- E. -p and -d
- F. -p and -m
- G. None of the above

Explanation: The -p or --module-path option is used to specify the location of the modules. The -m or --module option is used to specify the module and class name. However, running a program requires the package name to be specified with periods (.) instead of slashes.

4. Which of the following pairs make up a service?
- A. Consumer and service locator
 - B. Consumer and service provider interface
 - C. Service locator and service provider
 - D. Service locator and service provider interface
 - E. Service provider and service provider interface

Explanation: A service consists of the service provider interface and logic to look up implementations using a service locator.

5. A(n) _____ module is on the classpath while a(n) _____ module is on the module path. (Choose all that apply.)
- A. automatic, named
 - B. automatic, unnamed
 - C. named, automatic
 - D. named, unnamed
 - E. unnamed, automatic
 - F. unnamed, named
 - G. None of the above

Explanation: Automatic modules are on the module path but do not have a module-info.java file. Named modules are on the module path and do have a module-info.java. Unnamed modules are on the classpath.

6. Which of the following statements are true in a module-info.java file? (Choose all that apply.)
- A. The opens directive allows the use of reflection.
 - B. The opens directive declares that an API is called.
 - C. The use directive allows the use of reflection.
 - D. The use directive declares that an API is called.
 - E. The uses directive allows the use of reflection.
 - F. The uses directive declares that an API is called.

Explanation: Options C and D are incorrect because there is no use directive. Options A and F are correct because opens is for reflection and uses declares that an API consumes a service.

7. An automatic module name is generated if one is not supplied. Which of the following JAR filenames and generated automatic module name pairs are correct? (Choose all that apply.)
- A. emily-1.0.0.jar and emily
 - B. emily-1.0.0-SNAPSHOT.jar and emily
 - C. emily_the_cat-1.0.0.jar and emily_the_cat
 - D. emily_the_cat-1.0.0.jar and emily-the-cat
 - E. emily\$.jar and emily
 - F. emily\$.jar and emily.
 - G. emily\$.jar and emily..

Explanation: Any version information at the end of the JAR filename is removed, making options A and B correct. Underscores () are turned into dots (.). The special characters like a dollar sign (\$) are also turned into dots. However, adjacent dots are merged, and leading/trailing dots are removed.

8. Which of the following statements are true? (Choose all that apply.)

- A. Modules with cyclic dependencies will not compile.
- B. Packages with a cyclic dependency will not compile.
- C. A cyclic dependency always involves exactly two modules.
- D. A cyclic dependency always involves at least two requires statements.
- E. An unnamed module can be involved in a cyclic dependency with an automatic module.

Explanation: A cyclic dependency is when a module graph forms a circle. Option A is correct because the Java Platform Module System does not allow cyclic dependencies between modules. No such restriction exists for packages, making option B incorrect. A cyclic dependency can involve two or more modules that require each other, making option D correct, while option C is incorrect. Finally, option E is incorrect because unnamed modules cannot be referenced from an automatic module.

9. Suppose you are creating a service provider that contains the following class. Which line of code needs to be in your module-info.java?

```
package dragon;  
import magic.*;  
public class Dragon implements Magic {  
    public String getPower() {  
        return "breathe fire";  
    }  
}
```

- A. provides dragon.Dragon by magic.Magic;
- B. provides dragon.Dragon using magic.Magic;
- C. provides dragon.Dragon with magic.Magic;
- D. provides magic.Magic by dragon.Dragon;
- E. provides magic.Magic using dragon.Dragon;
- F. provides magic.Magic with dragon.Dragon;

Explanation: The provides directive takes the interface name first and the implementing class name second and also uses with

10. What is true of a module containing a file named module-info.java with the following contents? (Choose all that apply.)

```
module com.food.supplier {}
```

- A. All packages inside the module are automatically exported.
- B. No packages inside the module are automatically exported.
- C. A main method inside the module can be run.
- D. A main method inside the module cannot be run since the class is not exposed.
- E. The module-info.java file contains a compiler error.
- F. The module-info.java filename is incorrect.

Explanation: Packages inside a module are not exported by default, making option B correct and option A incorrect. Exporting is necessary for other code to use the packages; it is not necessary to call the main() method at the command line, making option C correct and option D incorrect. The module-info.java file has the correct name and compiles, making options E and F incorrect.

11. Suppose module `puppy` depends on module `dog` and module `dog` depends on module `animal`. Which lines allow module `puppy` to access the `animal.behavior` package in module `animal`? (Choose all that apply.)

```
module animal {
    exports animal.behavior;
}
module dog {
    _____ animal; // line S
}
module puppy {
    _____ dog;      // line T
}
```

- A. require on line S
- B. require on line T
- C. requires on line S
- D. requires on line T
- E. require transitive on line S
- F. require transitive on line T
- G. requires transitive on line S
- H. requires transitive on line T

Explanation: Because a simple **requires** only gives *dog* access to *animal*, but it doesn't *re-export* that access. This means **puppy**—which depends only on *dog*—cannot see the `animal.behavior` package unless *dog* declares **requires transitive** `animal`; which makes *animal*'s exports visible to any module that depends on *dog*.

12. Which of the following modules are provided by the JDK? (Choose all that apply.)

- A. `java.base`
- B. `java.desktop`
- C. `java.logging`
- D. `java.util`
- E. `jdk.base`
- F. `jdk.compiler`
- G. `jdk.xerces`

13. Which of the following compiles and is equivalent to this loop?

A.

```
List<Unicorn> all = new ArrayList<>();
for (Unicorn current : ServiceLoader.load(Unicorn.class))
    all.add(current);
```

B.

```
List<Unicorn> all = ServiceLoader.load(Unicorn.class)
    .getStream()
    .toList();
```

C.

```
List<Unicorn> all = ServiceLoader.load(Unicorn.class)
    .stream()
    .toList();
```

D.

```
List<Unicorn> all = ServiceLoader.load(Unicorn.class)
    .getStream()
    .map(Provider::get)
    .toList();
```

E.

```
List<Unicorn> all = ServiceLoader.load(Unicorn.class)
    .stream()
    .map(Provider::get)
    .toList();
```

F. None of the above

Explanation: There is no `getStream()` method on a `ServiceLoader`, making options A and C incorrect. Option B does not compile because the `stream()` method returns a list of `Provider` interfaces and needs to be converted to the `Unicorn` interface we are interested in.

14. Which of the following is a legal command to run a modular program where `n` is the module name and `c` is the fully qualified class name?

- A. `java --module-path x -m n.c`
- B. `java --module-path x -p n.c`
- C. `java --module-path x-x -m n/c`
- D. `java --module-path x -p n/c`
- E. `java --module-path x-x -m n-c`
- F. `java --module-path x -p n-c`
- G. None of the above

Explanation: The module name and class name are separated with a slash, making option C the answer. Note that `x-x` is legal because the module path is a folder name, so dashes are allowed.

15. For a top-down migration, all modules other than named modules are _____ modules and are on the _____.

- A. automatic, classpath
- B. automatic, module path
- C. unnamed, classpath
- D. unnamed, module path
- E. None of the above

16. Suppose you have separate modules for a service provider interface, service provider, service locator, and consumer. If you add a second service provider module, how many of the existing modules do you need to recompile?

- A. Zero
- B. One
- C. Two
- D. Three
- E. Four

Explanation: Since this is a new module, you need to compile it. However, none of the existing modules needs to be recompiled, making option A correct. The service locator will see the new service provider simply by having that new service provider on the module path.

17. Suppose we have a JAR file named `cat-1.2.3-RC1.jar`, and `Automatic-Module-Name` in the `MANIFEST.MF` is set to `dog`. What should an unnamed module referencing this automatic module include in `module-info.java`?
- A. `requires cat;`
 - B. `requires cat.RC;`
 - C. `requires cat-RC;`
 - D. `requires dog;`
 - E. None of the above

Explanation: Trick question! An unnamed module doesn't use a `module-info.java` file.

18. Two commands create artifacts that include smaller versions of the JDK. Which are used to create an `.exe` file and a directory, respectively?
- A. `jimage` and `jlink`
 - B. `jimage` and `jpackage`
 - C. `jlink` and `jimage`
 - D. `jlink` and `jpackage`
 - E. `jpackage` and `jimage`
 - F. `jpackage` and `jlink`

Explanation: The `jpackage` command creates self-contained application such as an `.exe`. The `jlink` command creates a directory with a smaller Java runtime containing just what is needed. The `jimage` command is used to inspect a Java image file.

19. Which is a true statement about the following module?

```
class dragon {  
    exports com.dragon.fire;  
    exports com.dragon.scales to castle;  
}
```

- A. All modules can reference the `com.dragon.fire` package.
- B. All modules can reference the `com.dragon.scales` package.
- C. Only the `castle` module can reference the `com.dragon.fire` package.
- D. Only the `castle` module can reference the `com.dragon.scales` package.
- E. None of the above.

20. Which would you expect to see when describing any module?

- A. requires java.base mandated
- B. requires java.core mandated
- C. requires java.lang mandated
- D. requires mandated java.base
- E. requires mandated java.core
- F. requires mandated java.lang
- G. None of the above

Explanation: When running java with the -d option, all the required modules are listed. Additionally, the java.base module is listed since it is included automatically. The line ends with mandated, making option A correct.

21. Suppose you have separate modules for a service provider interface, service provider, service locator, and consumer. Which module(s) need to specify a requires directive on the service provider?

- A. Service locator
- B. Service provider interface
- C. Consumer
- D. Consumer and service locator
- E. Consumer and service provider
- F. Service locator and service provider interface
- G. Consumer, service locator, and service provider interface
- H. None of the above

Explanation: This question is tricky. The service locator must have a uses directive, but that is on the service provider interface. No modules need to specify requires on the service provider since that is the implementation.

22. Which are true statements? (Choose all that apply.)

- A. An automatic module exports all packages to named modules.
- B. An automatic module exports only the specified packages to named modules.
- C. An automatic module exports no packages to named modules.
- D. An unnamed module exports only the named packages to named modules.
- E. An unnamed module exports all packages to named modules.
- F. An unnamed module exports no packages to named modules.

Explanation: An automatic module exports all packages, making option A correct. An unnamed module is not available to any modules on the module path. Therefore, it doesn't export any packages, and option F is correct.

23. Which is the first line to contain a compiler error?

```
1: module snake {  
2:     exports com.snake.tail;  
3:     exports com.snake.fangs to bird;  
4:     requires skin;  
5:     requires transitive skin;  
6: }
```

- A. Line 1.
- B. Line 2.
- C. Line 3.

D. Line 4.

E. Line 5.

F. The code does not contain any compiler errors.

Explanation: The module name is valid, as are the exports statements. Lines 4 and 5 are tricky because each is valid independently. However, the same module name is not allowed to be used in two requires statements. The second one fails to compile on line 5.

24. Which is a true statement about a package in a JAR on the classpath containing a module-info.java file?

A. It is possible to make the package available to all other modules on the classpath.

B. It is possible to make the package available to all other modules on the module path.

C. It is possible to make the package available to exactly one other specific module on the classpath.

D. It is possible to make the package available to exactly one other specific module on the module path.

E. It is possible to make sure the package is not available to any other modules on the classpath.

Explanation: Since the JAR is on the classpath, it is treated as a regular unnamed module even though it has a module-info.java file inside. Remember from learning about top-down migration that modules on the module path are not allowed to refer to the classpath, making options B and D incorrect. The classpath does not have a facility to restrict packages, making option A correct and options C and E incorrect.

25. Suppose you have separate modules for a service provider interface, service provider, service locator, and consumer. Which statements are true about the directives you need to specify? (Choose all that apply.)

A. The consumer must use the requires directive.

B. The consumer must use the uses directive.

C. The service locator must use the requires directive.

D. The service locator must use the uses directive.

E. None of the above.

Explanation: Options A and C are correct because both the consumer and the service locator depend on the service provider interface. Additionally, option D is correct because the service locator must specify that it uses the service provider interface to look it up.

Chapter 1: Building Blocks

1. Which of the following are legal entry point methods that can be run from the command line? (Choose all that apply.)
 - A. `private static void main(String[] args)`
 - B. `public static final main(String[] args)`
 - C. `public void main(String[] args)`
 - D. `public static final void main(String[] args)`
 - E. `public static void main(String[] args)`
 - F. `public static main(String[] args)`
2. Which answer options represent the order in which the following statements can be assembled into a program that will compile successfully? (Choose all that apply.)

```
X: class Rabbit {}  
Y: import java.util.*;  
Z: package animals;
```

- A. X, Y, Z
- B. Y, Z, X
- C. Z, Y, X
- D. Y, X
- E. Z, X
- F. X, Z

Explanation: The package and import statements are both optional. If both are present, the order must be package, then import, and then class.

3. Which of the following are true? (Choose all that apply.)

```
public class Bunny {  
    public static void main(String[] x) {  
        Bunny bun = new Bunny();  
    }  
}
```

- A. Bunny is a class.
- B. bun is a class.
- C. main is a class.
- D. Bunny is a reference to an object.
- E. bun is a reference to an object.
- F. main is a reference to an object.
- G. The main() method doesn't run because the parameter name is incorrect.

4. Which of the following are valid Java identifiers? (Choose all that apply.)

- A. `_`
- B. `_helloWorld$`
- C. `true`
- D. `java.lang`
- E. `Public`

F. 1980_s

G. Q2

5. Which statements about the following program are correct? (Choose all that apply.)

```
2: public class Bear {
3:     private Bear pandaBear;
4:     private void roar(Bear b) {
5:         System.out.println("Roar!");
6:         pandaBear = b;
7:     }
8:     public static void main(String[] args) {
9:         Bear brownBear = new Bear();
10:        Bear polarBear = new Bear();
11:        brownBear.roar(polarBear);
12:        polarBear = null;
13:        brownBear = null;
14:        System.gc(); } }
```

- A. The object created on line 9 is first eligible for garbage collection after line 13.
- B. The object created on line 9 is first eligible for garbage collection after line 14.
- C. The object created on line 10 is first eligible for garbage collection after line 12.
- D. The object created on line 10 is first eligible for garbage collection after line 13.
- E. Garbage collection is guaranteed to run.
- F. Garbage collection might or might not run.
- G. The code does not compile.

Explanation: Garbage collection **is never guaranteed to run**, making option F correct and option E incorrect. The Bear object created on line 10 is accessible via both the polarBear reference and the brownBear.pandaBear reference. After line 12, the object is still accessible via brownBear.pandaBear. After line 13, though, it is no longer accessible since brownBear.

6. Assuming the following class compiles, how many variables defined in the class or method are in scope on the line marked on line 14?

```
1: public class Camel {
2:     { int hairs = 3_000_0; }
3:     long water, air=2;
4:     boolean twoHumps = true;
5:     public void spit(float distance) {
6:         var path = "";
7:         { double teeth = 32 + distance++; }
8:         while(water> 0) {
9:             int age = twoHumps ? 1 : 2;
10:            short i=-1;
11:            for(i=0; i<10; i++) {
12:                var Private = 2;
13:            }
14:            // SCOPE
15:        }
16:    }
17: }
```

- A. 2
- B. 3
- C. 4
- D. 5
- E. 6
- F. 7**
- G. None of the above

Explanation: water, air, twoHumps, distance(from method), path, age and i

7. Which are true about this code? (Choose all that apply.)

```
public class KitchenSink {  
    private int numForks;  
  
    public static void main(String[] args) {  
        int numKnives;  
        System.out.print("""  
            "# forks = " + numForks +  
            " # knives = " + numKnives +  
            # cups = 0""");  
        }  
    }  
}
```

- A. The output includes # forks = 0.
- B. The output includes # knives = 0.
- C. The output includes # cups = 0.**
- D. The output includes a blank line.
- E. The output includes one or more lines that begin with whitespace.**
- F. The code does not compile.

Explanation: The first thing to recognize is that this is a text block and the code inside the """ is just text. Option E is also an answer since " # knives is indented.

8. Which of the following code snippets about var compile without issue when used in a method? (Choose all that apply.)

- A. var spring = null;
- B. var fall = "leaves";**
- C. var evening = 2; evening = null;
- D. var night = Integer.valueOf(3);**
- E. var day = 1/0;**
- F. var winter = 12, cold;
- G. var fall = 2, autumn = 2;
- H. var morning = ""; morning = null;**

Explanation: Option E is correct as this is a valid numeric expression. You might know that dividing by zero produces a runtime exception, but the question was only about whether the code compiled.

9. Which of the following is correct?

- A. An instance variable of type float defaults to 0.
- B. An instance variable of type char defaults to null.
- C. A local variable of type double defaults to 0.0.
- D. A local variable of type int defaults to null.
- E. A class variable of type String defaults to null.
- F. A class variable of type String defaults to the empty string "".
- G. None of the above.

10. Which of the following expressions, when inserted independently into the blank line, allow the code to compile? (Choose all that apply.)

```
public void printMagicData() {  
    var magic = ;  
    System.out.println(magic);  
}
```

- A. 3_1
- B. 1_329_.0
- C. 3_13.0_
- D. 5_291._2
- E. 2_234.0_0
- F. 9_6
- G. _1_3_5_0

11. Given the following two class files, what is the maximum number of imports that can be removed and have the code still compile?

```
// Water.java  
package aquarium;  
public class Water { }  
  
// Tank.java  
package aquarium;  
import java.lang.*;  
import java.lang.System;  
import aquarium.Water;  
import aquarium.*;  
public class Tank {  
    public void print(Water water) {  
        System.out.println(water); } } }
```

- A. 0
- B. 1
- C. 2
- D. 3
- E. 4
- F. Does not compile

Explanation: The first two imports can be removed because `java.lang` is automatically imported. The following two imports can be removed because Tank and Water are in the same package.

12. Which statements about the following class are correct? (Choose all that apply.)

```
1: public class ClownFish {
2:     int gills = 0, double weight=2;
3:     { int fins = gills; }
4:     void print(int length = 3) {
5:         System.out.println(gills);
6:         System.out.println(weight);
7:         System.out.println(fins);
8:         System.out.println(length);
9: } }
```

- A. Line 2 generates a compiler error.
- B. Line 3 generates a compiler error.
- C. Line 4 generates a compiler error.
- D. Line 7 generates a compiler error.
- E. The code prints 0.
- F. The code prints 2.0.
- G. The code prints 2.
- H. The code prints 3.

Explanation: Line 4 does not compile because `Java does not support setting default method parameter values`, making option C correct.

13. Given the following classes, which of the following snippets can independently be inserted in place of INSERT IMPORTS HERE and have the code compile? (Choose all that apply.)

```
package aquarium;
public class Water {
    boolean salty = false;
}

package aquarium.jellies;
public class Water {
    boolean salty = true;
}

package employee;
INSERT IMPORTS HERE
public class WaterFiller {
    Water water;
}
```

- A. `import aquarium.*;`
- B. `import aquarium.Water;`
- C. `import aquarium.jellies.*;`
- D. `import aquarium.*;`
- E. `import aquarium.jellies.Water;`
- F. `import aquarium.*;`

- G. import aquarium.jellies.*;
- H. import aquarium.Water;
- I. import aquarium.jellies.Water;
- J. None of the above

Explanation: Option A is correct because it imports all the classes in the aquarium package including aquarium.Water. Options B and C are correct because they import Water by class name. Since importing by class name takes precedence over wildcards, these compile.

14. Which of the following statements about the code snippet are true? (Choose all that apply.)

```
3: short numPets = 5L;  
4: int numGrains = 2.0;  
5: String name = "Scruffy";  
6: int d = numPets.length();  
7: int e = numGrains.length;  
8: int f = name.length();
```

- A. Line 3 generates a compiler error.
- B. Line 4 generates a compiler error.
- C. Line 5 generates a compiler error.
- D. Line 6 generates a compiler error.
- E. Line 7 generates a compiler error.
- F. Line 8 generates a compiler error.

15. Which of the following statements about garbage collection are correct? (Choose all that apply.)

- A. Calling System.gc() is guaranteed to free up memory by destroying objects eligible for garbage collection.
- B. Garbage collection runs on a set schedule.
- C. Garbage collection allows the JVM to reclaim memory for other objects.
- D. Garbage collection runs when your program has used up half the available memory.
- E. An object may be eligible for garbage collection but never removed from the heap.
- F. An object is eligible for garbage collection once no references to it are accessible in the program.
- G. Marking a variable final means its associated object will never be garbage collected.

Explanation: In Java, there are no guarantees about when garbage collection will run. The JVM is free to ignore calls to System.gc(). For this reason, options A, B, and D are incorrect.

16. Which are true about this code? (Choose all that apply.)

```
var blocky = "  
  squirrel \s  
  pigeon  \  
  termite""";  
System.out.print(blocky);
```

- A. It outputs two lines.
- B. It outputs three lines.
- C. It outputs four lines.
- D. There is one line with trailing whitespace.

- E. There are two lines with trailing whitespace.
- F. If we indented each line five characters, it would change the output.

Explanation: Remember that a backslash means to skip the line break. Option D is also correct as `\s` means to keep whitespace. In a text block, incidental indentation is ignored, making option F incorrect.

17. What lines are printed by the following program? (Choose all that apply.)

```
1: public class WaterBottle {
2:     private String brand;
3:     private boolean empty;
4:     public static float code;
5:     public static void main(String[] args) {
6:         WaterBottle wb = new WaterBottle();
7:         System.out.println("Empty = " + wb.empty);
8:         System.out.println("Brand = " + wb.brand);
9:         System.out.println("Code = " + code);
10:    }
```

- A. Line 8 generates a compiler error.
- B. Line 9 generates a compiler error.
- C. Empty =
- D. Empty = false
- E. Brand =
- F. Brand = null
- G. Code = 0.0
- H. Code = 0f

18. Which of the following statements about var are true? (Choose all that apply.)

- A. A var can be used as a constructor parameter.
- B. The type of a var is known at compile time.
- C. A var cannot be used as an instance variable.
- D. A var can be used in a multiple variable assignment statement.
- E. The value of a var cannot change at runtime.
- F. The type of a var cannot change at runtime.
- G. The word var is a reserved word in Java.

19. Which are true about the following code? (Choose all that apply.)

```
var num1 = Integer.parseInt("11");
var num2 = Integer.valueOf("B", 16);
System.out.println(Integer.max(num1, num2));
```

- A. The output is 11.
- B. The output is B.
- C. The code does not compile.
- D. num1 is a primitive.
- E. num2 is a primitive.
- F. A NumberFormatException is thrown.

20. Which statement about the following class is correct?

```
1: public class PoliceBox {
2:     String color;
3:     long age;
4:     public void PoliceBox() {
5:         color = "blue";
6:         age = 1200;
7:     }
8:     public static void main(String []time) {
9:         var p = new PoliceBox();
10:        var q = new PoliceBox();
11:        p.color = "green";
12:        p.age = 1400;
13:        p = q;
14:        System.out.println("Q1="+q.color);
15:        System.out.println("Q2="+q.age);
16:        System.out.println("P1="+p.color);
17:        System.out.println("P2="+p.age);
18: } }
```

- A. It prints Q1=blue.
- B. It prints Q2=1200.
- C. It prints P1=null.
- D. It prints P2=1400.
- E. Line 4 does not compile.
- F. Line 12 does not compile.
- G. Line 13 does not compile.
- H. None of the above.

21. What is the output of executing the following class?

```
1: public class Salmon {
2:     int count;
3:     { System.out.print(count+"-"); }
4:     { count++; }
5:     public Salmon() {
6:         count = 4;
7:         System.out.print(2+"-");
8:     }
9:     public static void main(String[] args) {
10:        System.out.print(7+"-");
11:        var s = new Salmon();
12:        System.out.print(s.count+"-"); } }
```

- A. 7-0-2-1-
- B. 7-0-1-
- C. 0-7-2-1-
- D. 7-0-2-4-
- E. 0-7-1-
- F. The class does not compile because of line 3.

G. The class does not compile because of line 4.

H. None of the above.

Explanation: The default value of an instance variable of type int is 0, so 0- is printed next, and count is assigned a value of 1. **Next, the constructor is called.**

22. Given the following class, which of the following lines of code can independently replace INSERT CODE HERE to make the code compile? (Choose all that apply.)

```
public class Price {  
    public void admission() {  
        INSERT CODE HERE  
        System.out.print(amount);  
    }  
}
```

A. int amount = 0b11;

B. int amount = 9L;

C. int amount = 0xE //14;

D. int amount = 1_2.0;

E. double amount = 1_0_.0;

F. int amount = 0b101 //5;

G. double amount = 9_2.1_2;

H. double amount = 1_2_.0_0;

Explanation: First, 0b is the prefix for a binary value, and 0x is the prefix for a hexadecimal value. These values can be assigned to many primitive types, including int and double, making options C and F correct.

23. Which statements about the following class are true? (Choose all that apply.)

```
1: public class River {  
2:     int Depth = 1;  
3:     float temp = 50.0;  
4:     public void flow() {  
5:         for (int i = 0; i < 1; i++) {  
6:             int depth = 2;  
7:             depth++;  
8:             temp--;  
9:         }  
10:        System.out.println(depth);  
11:        System.out.println(temp); }  
12:    public static void main(String... s) {  
13:        new River().flow();  
14:    } }
```

A. Line 3 generates a compiler error.

B. Line 6 generates a compiler error.

C. Line 7 generates a compiler error.

D. Line 10 generates a compiler error.

E. The program prints 3 on line 10.

F. The program prints 4 on line 10.

- G. The program prints 50.0 on line 11.
- H. The program prints 49.0 on line 11.

Chapter 2: Operators

1. Which of the following Java operators can be used with boolean variables? (Choose all that apply.)

- A. `==`
- B. `+`
- C. `--`
- D. `!`
- E. `%`
- F. `-`
- G. Cast with `(boolean)`

2. What data type (or types) will allow the following code snippet to compile? (Choose all that apply.)

```
byte apples = 5;
short oranges = 10;
_____ bananas = apples + oranges;
```

- A. `int`
- B. `long`
- C. `boolean`
- D. `double`
- E. `short`
- F. `byte`

Explanation: The expression `apples + oranges` is automatically promoted to `int`, so `int` and data types that can be promoted automatically from `int` will work, making options A, B, and D correct.

3. What change, when applied independently, would allow the following code snippet to compile? (Choose all that apply.)

```
3: long ear = 10;
4: int hearing = 2 * ear;
```

- A. No change; it compiles as is.
- B. Cast `ear` on line 4 to `int`.
- C. Change the data type of `ear` on line 3 to `short`.
- D. Cast `2 * ear` on line 4 to `int`.
- E. Change the data type of `hearing` on line 4 to `short`.
- F. Change the data type of `hearing` on line 4 to `long`.

Explanation: The code will not compile as is, so option A is not correct. The value `2 * ear` is automatically promoted to `long` and cannot be automatically stored in `hearing`, which is an `int` value.

4. What is the output of the following code snippet?

```
3: boolean canine = true, wolf = true;
4: int teeth = 20;
5: canine = (teeth != 10) ^ (wolf=false);
6: System.out.println(canine+"", "+teeth+", "+wolf);
```

- A. true, 20, true
- B. true, 20, false
- C. false, 10, true
- D. false, 20, false
- E. The code will not compile because of line 5.
- F. None of the above.

5. Which of the following operators are ranked in increasing or the same order of precedence? Assume the + operator is binary addition, not the unary form. (Choose all that apply.)

- A. +, *, %, --
- B. ++, (int), *
- C. =, ==, !
- D. (short), =, !, *
- E. *, /, %, +, ==
- F. !, ||, &
- G. ^, +, =, +=

Explanation: Options A and C are correct, as they show operators in increasing or the same order of precedence.

6. What is the output of the following program?

```
1: public class CandyCounter {
2:     static long addCandy(double fruit, float vegetables) {
3:         return (int)fruit+vegetables;
4:     }
5:
6:     public static void main(String[] args) {
7:         System.out.print(addCandy(1.4, 2.4f) + ", ");
8:         System.out.print(addCandy(1.9, (float)4) + ", ");
9:         System.out.print(addCandy((long)(int)(short)2, (float)4)); }
}
```

- A. 4, 6, 6.0
- B. 3, 5, 6
- C. 3, 6, 6
- D. 4, 5, 6
- E. The code does not compile because of line 9.
- F. None of the above.

Explanation: The code does not compile because line 3 contains a compilation error. The cast (int) is applied to fruit, not the expression fruit+vegetables. Since the cast operator has a higher operator precedence than the addition operator, it is applied to fruit, but the expression is promoted to a float, due to vegetables being float.

7. What is the output of the following code snippet?

```
int ph = 7, vis = 2;
boolean clear = vis > 1 & (vis < 9 || ph < 2);
boolean safe = (vis > 2) && (ph++ > 1);
boolean tasty = 7 <= --ph;
System.out.println(clear + "-" + safe + "-" + tasty);
```

- A. true-true-true
- B. true-true-false
- C. true-false-true
- D. true-false-false
- E. false-true-true
- F. false-true-false
- G. false-false-true
- H. false-false-false

8. What is the output of the following code snippet?

```
4: int pig = (short)4;
5: pig = pig++;
6: long goat = (int)2;
7: goat -= 1.0;
8: System.out.print(pig + " - " + goat);
```

- A. 4 - 1
- B. 4 - 2
- C. 5 - 1
- D. 5 - 2
- E. The code does not compile due to line 7.
- F. None of the above.

Explanation: Line 5 increments pig by 1, but it returns the original value of 4 since it is using the post-increment operator.

9. What are the unique outputs of the following code snippet? (Choose all that apply.)

```
int a = 2, b = 4, c = 2;
System.out.println(a > 2 ? --c : b++);
System.out.println(b = (a != c ? a : b++));
System.out.println(a > b ? b < c ? b : 2 : 1);
```

- A. 1
- B. 2
- C. 3
- D. 4
- E. 5
- F. 6
- G. The code does not compile.

10. Which is not an output of the following code snippet?

```
short height = 1, weight = 3;
short zebra = (byte) weight * (byte) height;
double ox = 1 + height * 2 + weight;
long giraffe = 1 + 9 % height + 1;
System.out.println(zebra);
System.out.println(ox);
System.out.println(giraffe);
```

- A. 2
- B. 3
- C. 6
- D. 6.0
- E. The code does not compile.

Explanation: The code does not compile due to an error on the second line, making option E correct. Even though both height and weight are cast to byte, the multiplication operator automatically promotes them to int, resulting in an attempt to store an int in a short variable.

11. What is the output of the following code?

```
11: int sample1 = (2 * 4) % 3;
12: int sample2 = 3 * 2 % -3;
13: int sample3 = 5 * (1 % 2);
14: System.out.println(sample1 + ", " + sample2 + ", " + sample3);
```

- A. 0, 0, 5
- B. 1, 2, 10
- C. 2, 1, 5
- D. 2, 0, 5
- E. 3, 1, 10
- F. 3, 2, 6
- G. The code does not compile.

12. The _____ operator increases a value and returns the original value, while the _____ operator decreases a value and returns the new value.

- A. post-increment, post-increment
- B. pre-decrement, post-decrement
- C. post-increment, post-decrement
- D. post-increment, pre-decrement
- E. pre-increment, pre-decrement
- F. pre-increment, post-decrement

13. What is the output of the following code snippet?

```
boolean sunny = true, raining = false, sunday = true;
boolean goingToTheStore = sunny & raining ^ sunday;
boolean goingToTheZoo = sunday && !raining;
boolean stayingHome = !(goingToTheStore && goingToTheZoo);
System.out.println(goingToTheStore + "-" + goingToTheZoo
    + "-" + stayingHome);
```

- A. true-false-false
- B. false-true-false
- C. true-true-true
- D. false-true-true
- E. false-false-false
- F. true-true-false
- G. None of the above

14. Which of the following statements are correct? (Choose all that apply.)

- A. The return value of an assignment operation expression can be void.
- B. The inequality operator (!=) can be used to compare objects.
- C. The equality operator (==) can be used to compare a boolean value with a numeric value.
- D. During runtime, the & and | operators may cause only the left side of the expression to be evaluated.
- E. The return value of an assignment operation expression is the value of the newly assigned variable.
- F. In Java, 0 and false may be used interchangeably.
- G. The logical complement operator (!) cannot be used to flip numeric values.

15. Which operator takes three operands or values?

- A. =
- B. &&
- C. *=
- D. ?:
- E. &
- F. ++
- G. /

16. How many lines of the following code contain compiler errors?

```
int note = 1 * 2 + (long)3;  
short melody = (byte)(double)(note *= 2);  
double song = melody;  
float symphony = (float)((song == 1_000f) ? song * 2L : song);
```

- A. 0
- B. 1
- C. 2
- D. 3
- E. 4

Explanation: The first line contains a compilation error. The value 3 is cast to long. The 1 * 2 value is evaluated as int but promoted to long when added to the 3. Trying to store a long value in an int variable triggers a compiler error.

17. Given the following code snippet, what are the values of the variables after it is executed? (Choose all that apply.)

```
int ticketsTaken = 1;
int ticketsSold = 3;
ticketsSold += 1 + ticketsTaken++;
ticketsTaken *= 2;
ticketsSold += (long)1;
```

- A. ticketsSold is 8.
- B. ticketsTaken is 2.
- C. ticketsSold is 6.
- D. ticketsTaken is 6.
- E. ticketsSold is 7.
- F. ticketsTaken is 4.
- G. The code does not compile.

18. Which of the following can be used to change the order of operation in an expression?

- A. []
- B. <>
- C. ()
- D. \/
- E. { }
- F. " "

19. What is the result of executing the following code snippet? (Choose all that apply.)

```
3: int start = 7;
4: int end = 4;
5: end += ++start;
6: start = (byte)(Byte.MAX_VALUE + 1);
```

- A. start is 0.
- B. start is -128.
- C. start is 127.
- D. end is 8.
- E. end is 11.
- F. end is 12.
- G. The code does not compile.
- H. The code compiles but throws an exception at runtime.

Explanation: On line 5, the pre-increment operator is executed first, so start is incremented to 8.

20. Which of the following statements about unary operators are true? (Choose all that apply.)

- A. Unary operators are always executed before any surrounding numeric binary or ternary operators.
- B. The - operator can be used to flip a boolean value.
- C. The pre-increment operator (++) returns the value of the variable before the increment is applied.
- D. The post-decrement operator (--) returns the value of the variable before the decrement is applied.
- E. The ! operator cannot be used on numeric values.
- F. None of the above.

Chapter 3: Making Decisions

1. What is the output of the following code snippet?

```
32: Object skips = 10;
33: switch (skips) {
34:     case a when a < 10 -> System.out.print(2);
35:     case b when b >= 10 -> System.out.print(4);
36:     case null -> System.out.print(6);
37:     default -> System.out.print(8);
38: }
```

- A. 2
 - B. 4
 - C. 6
 - D. 8
 - E. Exactly one line does not compile.
 - F. Exactly two lines do not compile.
 - G. None of the above.
2. Which of the following data types can be used in a switch expression? (Choose all that apply.)

- A. enum
- B. int
- C. Byte
- D. long
- E. boolean
- F. double

3. What is the output of the following code snippet?

```
3: int temperature = 4;
4: long humidity = -temperature + temperature * 3;
5: if (temperature >= 4)
6: if (humidity < 6) System.out.println("Too Low");
7: else System.out.println("Just Right");
8: else System.out.println("Too High");
```

- A. Too Low
- B. Just Right
- C. Too High
- D. A NullPointerException is thrown at runtime.
- E. The code will not compile because of line 7.
- F. The code will not compile because of line 8.

4. Which of the following data types are permitted on the right side of a for-each expression? (Choose all that apply.)

A. Double[][]
B. Object
C. Map
D. List
E. String
F. char[]
G. Exception

5. What is the output of calling printReptile(6)?

```
void printReptile(int category) {  
    var type = switch (category) {  
        case 1,2 -> "Snake";  
        case 3,4 -> "Lizard";  
        case 5,6 -> "Turtle";  
        case 7,8 -> "Alligator";  
    };  
    System.out.print(type);  
}
```

A. Snake
B. Lizard
C. Turtle
D. Alligator
E. TurtleAlligator
F. None of the above

6. What is the output of the following code snippet?

```
List<Integer> myFavoriteNumbers = new ArrayList<>();  
myFavoriteNumbers.add(10);  
myFavoriteNumbers.add(14);  
for (var a : myFavoriteNumbers) {  
    System.out.print(a + ", ");  
    break;  
}  
  
for (int b : myFavoriteNumbers) {  
    continue;  
    System.out.print(b + ", ");  
}  
  
for (Object c : myFavoriteNumbers)  
    System.out.print(c + ", ");
```

A. It compiles and runs without issue but does not produce any output.

- B. 10, 14,
- C. 10, 10, 14,
- D. 10, 10, 14, 10, 14,
- E. Exactly one line of code does not compile.
- F. Exactly two lines of code do not compile.
- G. Three or more lines of code do not compile.
- H. The code contains an infinite loop and does not terminate.

Explanation: The second for-each loop contains a continue followed by a print() statement. Because the continue is not conditional and always included as part of the body of the for-each loop, the print() statement is not reachable.

7. Assuming weather is a well-formed nonempty array, which code snippet, when inserted independently into the blank in the following code, prints all of the elements of weather? (Choose all that apply.)

```
private void print(int[] weather) {
    for () {
        System.out.println(weather[i]);
    }
}
```

- A. int i=weather.length; i>0; i--
- B. int i=0; i<=weather.length-1; ++i
- C. var w : weather
- D. int i=weather.length-1; i>=0; i--
- E. int i=0, int j=3; i<weather.length; ++i
- F. int i=0; ++i<10 && i<weather.length;
- G. None of the above

8. What is the output of calling printType(11)?

```
31: void printType(Object o) {
32:     if (o instanceof Integer bat) {
33:         System.out.print("int");
34:     } else if (o instanceof Integer bat && bat < 10) {
35:         System.out.print("small int");
36:     } else if (o instanceof Long bat || bat <= 20) {
37:         System.out.print("long");
38:     } default {
39:         System.out.print("unknown");
40:     }
41: }
```

- A. int
- B. small int
- C. long
- D. unknown
- E. Nothing is printed.
- F. The code contains one line that does not compile.
- G. The code contains two lines that do not compile.

H. None of the above.

Explanation: Line 36 does not compile, though. Due to flow scoping, if o is not a Long, then bat is not in scope in the expression bat <= 20. Line 38 also does not compile as default cannot be used as part of an if/else statement. For these two reasons, option G is correct.

9. Which statements, when inserted independently into the following blank, will cause the code to print 2 at runtime? (Choose all that apply.)

```
int count = 0;
BUNNY: for (int row = 1; row <=3; row++)
    RABBIT: for (int col = 0; col <3 ; col++) {
        if ((col + row) % 2 == 0)
            ;
        count++;
    }
System.out.println(count);
```

- A. break BUNNY
- B. break RABBIT
- C. continue BUNNY
- D. continue RABBIT
- E. break
- F. continue
- G. None of the above, as the code contains a compiler error

Explanation: The code uses nested loops and a condition that runs when row + col is even; otherwise, count increases. Options E and F are equivalent to B and D since unlabeled statements apply to the innermost loop. The first true condition occurs on the second inner-loop iteration (row = 1, col = 1). Option A is wrong because it exits the loop with count = 1. In options B, C, and E, count becomes 1 in the first inner loop, exits, then increments to 2 in the third outer loop iteration before the program ends, making these options correct.

10. Given the following method, how many lines contain compilation errors?

```
8:  enum DayOfWeek {
9:      SUNDAY, MONDAY, TUESDAY, WEDNESDAY, THURSDAY, FRIDAY, SATURDAY;
10:     private DayOfWeek getWeekDay(int day, final int thursday) {
11:         int otherDay = day;
12:         int Sunday = 0;
13:         switch (otherDay) {
14:             default:
15:                 case 1: continue;
16:                 case thursday: return DayOfWeek.THURSDAY;
17:                 case 2,10: break;
18:                 case Sunday: return DayOfWeek.SUNDAY;
19:                 case DayOfWeek.MONDAY: return DayOfWeek.MONDAY;
20:         }
21:         return DayOfWeek.FRIDAY;
22:     } }
```

- A. None, the code compiles and runs without issue.

- B. 1.
- C. 2.
- D. 3.
- E. 4.
- F. 5.
- G. 6.
- H. The code compiles but may produce an error at runtime.

11. What is the output of calling `printLocation(Animal.MAMMAL)`?

```
10: class Zoo {
11:     enum Animal {BIRD, FISH, MAMMAL}
12:     void printLocation(Animal a) {
13:         long type = switch (a) {
14:             case BIRD -> 1;
15:             case FISH -> 2;
16:             case MAMMAL -> 3;
17:             default -> 4;
18:         };
19:         System.out.print(type);
20:     } }
```

- A. 3
- B. 4
- C. 34
- D. The code does not compile because of line 13.
- E. The code does not compile because of line 17.
- F. None of the above.

12. What is the result of the following code snippet?

```
3: int sing = 8, squawk = 2, notes = 0;
4: while (sing > squawk) {
5:     sing--;
6:     squawk += 2;
7:     notes += sing + squawk;
8: }
9: System.out.println(notes);
```

- A. 11
- B. 13
- C. 23
- D. 33
- E. 50
- F. The code will not compile because of line 7.

13. What is the result of calling `getHatSize(9f)` on the following code snippet?

```
10: int getHatSize(Number measurement) {
11:     return switch (measurement) {
12:         case Double d -> 1 + d.intValue();
13:         case null      -> 11;
14:         case !(Number n) -> 3 + n.intValue();
15:         case Float f when f < 10 -> 4 + f.intValue();
16:     };
17: }
```

- A. 10
- B. 11
- C. 12
- D. 13
- E. The code does not compile because it is missing a default clause.
- F. The code does not compile for a different reason.

Explanation: The code does not compile because case clause uses the logical complement operator (!), which is not permitted with pattern matching.

14. What is the output of the following code snippet?

```
2: boolean keepGoing = true;
3: int result = 15, meters = 10;
4: do {
5:     meters--;
6:     if (meters==8) keepGoing = false;
7:     result -= 2;
8: } while keepGoing;
9: System.out.println(result);
```

- A. 7
- B. 9
- C. 10
- D. 11
- E. 15
- F. The code will not compile because of line 6.
- G. The code does not compile for a different reason.

Explanation: Line 8 is missing the required parentheses around the boolean conditional expression.

15. Which statements about the following code snippet are correct? (Choose all that apply.)

```
for (var penguin : new int[2])
    System.out.println(penguin);

var ostrich = new Character[3];
for (var emu : ostrich)
    System.out.println(emu);

List<Integer> parrots = new ArrayList<Integer>();
for (var macaw : parrots)
```

```
System.out.println(macaw);
```

- A. The data type of penguin is Integer.
- B. The data type of penguin is int.
- C. The data type of emu is undefined.
- D. The data type of emu is Character.
- E. The data type of macaw is List.
- F. The data type of macaw is Integer.
- G. None of the above, as the code does not compile.

16. What is the result of the following code snippet?

```
final char a = 'A', e = 'E';
char grade = 'B';
switch (grade) {
    default:
    case a:
    case 'B': 'C': System.out.print("great ");
    case 'D':      System.out.print("good "); break;
    case e:
    case 'F':      System.out.print("not good ");
}
```

- A. great
- B. great good
- C. good
- D. not good
- E. The code does not compile because the data type of one or more case clauses does not match the data type of the switch variable.
- F. None of the above.

Explanation: The code does not compile, although not for the reason specified in option E. The second case clause contains invalid syntax. This line could have been fixed to say case 'B', 'C' or by adding the case keyword before 'C'.

17. Given the following array, which code snippets print the elements in reverse order from how they are declared? (Choose all that apply.)

A.

```
char[] wolf = {'W', 'e', 'b', 'b', 'y'};
int q = wolf.length;
for ( ; ; ) {
    System.out.print(wolf[--q]);
    if (q==0) break;
}
```

B.

```
for (int m=wolf.length-1; m>=0; --m)
    System.out.print(wolf[m]);
```

C.

```
for (int z=0; z<wolf.length; z++)  
    System.out.print(wolf[wolf.length-z]);
```

D.

```
int x = wolf.length-1;  
for (int j=0; x>=0 && j==0; x--)  
    System.out.print(wolf[x]);
```

E.

```
final int r = wolf.length;  
for (int w = r-1; r>-1; w = r-1)  
    System.out.print(wolf[w]);
```

F.

```
for (int i=wolf.length; i>0; --i)  
    System.out.print(wolf[i]);
```

G. None of the above

18. What distinct numbers are printed when the following method is executed? (Choose all that apply.)

```
private void countAttendees() {  
    int participants = 4, animals = 2, performers = -1;  
  
    while ((participants = participants + 1) < 10) {}  
    do {} while (animals++ <= 1);  
    for ( ; performers < 2; performers += 2) {}  
  
    System.out.println(participants);  
    System.out.println(animals);  
    System.out.println(performers);  
}
```

A. 6

B. 3

C. 4

D. 5

E. 10

F. 9

G. The code does not compile.

H. None of the above.

19. What is the output of the following code snippet?

```
2: double iguana = 0;  
3: do {  
4:     int snake = 1;  
5:     System.out.print(snake++ + " ");  
6:     iguana--;  
7: } while (snake <= 5);  
8: System.out.println(iguana);
```


- A. 1 2 3 4 -4.0
- B. 1 2 3 4 -5.0
- C. 1 2 3 4 5 -4.0
- D. 0 1 2 3 4 5 -5.0
- E. The code does not compile.
- F. The code compiles but produces an infinite loop at runtime.
- G. None of the above.

Explanation: The variable snake is declared within the body of the do/while statement, so it is out of scope on line 7

20. Which statements, when inserted into the following blanks, allow the code to compile and run without entering an infinite loop? (Choose all that apply.)

```

4:  int height = 1;
5:  L1: while (height++ <10) {
6:      long humidity = 12;
7:      L2: do {
8:          if (humidity-- % 12 == 0) ;
9:          int temperature = 30;
10:         L3: for ( ; ; ) {
11:             temperature++;
12:             if (temperature>50) ;
13:         }
14:     } while (humidity> 4);
15: }

```

- A. break L2 on line 8; continue L2 on line 12
- B. continue on line 8; continue on line 12
- C. break L3 on line 8; break L1 on line 12
- D. continue L2 on line 8; continue L3 on line 12
- E. continue L2 on line 8; continue L2 on line 12
- F. None of the above, as the code contains a compiler error

Explanation: The key point is that the innermost loop is infinite, so correct solutions must skip or exit it. Option A works because break L2 exits the second inner loop, avoiding the infinite loop. Option B skips the innermost loop only once, then runs it indefinitely, causing an infinite loop. Option C fails due to a compiler error — L3 is out of scope. Option D behaves like B, also looping infinitely. Option E is correct since continue L2 on line 12 exits the infinite loop, allowing the program to finish.

21. A minimum of how many lines need to be corrected before the following method will compile?

```

21: void findZookeeper(Integer id) {
22:     System.out.print(switch (id) {
23:         case 10 -> {"Jane";}
24:         case 20 -> {yield "Lisa";};
25:         case 30 -> "Kelly";
26:         case 30 -> "Sarah";
27:         default -> "Unassigned";
28:     });
29: }

```

- A. Zero
- B. One
- C. Two
- D. Three**
- E. Four
- F. Five

Explanation: Line 23 does not compile because it is missing a yield statement. Line 24 does not compile because it contains an extra semicolon at the end. Finally, lines 25 and 26 do not compile because they use the same case value. At least one of them would need to be changed for the code to compile.

22. What is the output of the following code snippet?

```
2: var tailFeathers = 3;
3: final var one = 1;
4: switch (tailFeathers) {
5:     case one: System.out.print(3 + " ");
6:     default: case 3: System.out.print(5 + " ");
7: }
8: while (tailFeathers > 1) {
9:     System.out.print(--tailFeathers + " "); }
```

- A. 3
- B. 5 1
- C. 5 2
- D. 3 5 1
- E. 5 2 1**
- F. The code will not compile because of lines 3–5.
- G. The code will not compile because of line 6.

23. What is the output of the following code snippet?

```
15: int penguin = 50, turtle = 75;
16: boolean older = penguin >= turtle;
17: if (older = true) System.out.println("Success");
18: else System.out.println("Failure");
19: else if (penguin != 50) System.out.println("Other");
```

- A. Success
- B. Failure
- C. Other
- D. The code will not compile because of line 17.
- E. The code compiles but throws an exception at runtime.
- F. None of the above.**

Explanation: Line 19 starts with an else statement, but there is no preceding if statement that it matches. For this reason, line 19 does not compile, making option F the correct answer.

24. What is the output of the following code snippet?

```
22: String zooStatus = "Closed";
23: int visitors = switch (zooStatus) {
24:     case String s when s.equals("Open") -> 10;
25:     case Object s when s != null && !s.equals("") -> 20;
26:     case null -> {yield 30;}
27:     default -> 40;
28: };
29: System.out.print(visitors);
```

- A. 10
- B. 20**
- C. 30
- D. 40
- E. Exactly one line does not compile.
- F. Exactly two lines do not compile.
- G. Three or more lines do not compile.

25. What is the output of the following code snippet?

```
6: String instrument = "violin";
7: final String CELLO = "cello";
8: String viola = "viola";
9: int p = -1;
10: switch (instrument) {
11:     case "bass" : break;
12:     case CELLO : p++;
13:     default: p++;
14:     case "VIOLIN": p++;
15:     case "viola" : ++p; break;
16: }
17: System.out.print(p);
```

- A. -1
- B. 0
- C. 1
- D. 2**
- E. 3
- F. The code does not compile.

Explanation: This execution path increments p a total of three times, bringing the final value of p to 2 and making option D the correct answer.

26. What is the output of the following code snippet?

```
9: int w = 0, r = 1;
10: String name = "";
11: while (w < 2) {
12:     name += "A";
```

```

13:    do {
14:        name += "B";
15:        if (name.length()>0) name += "C";
16:        else break;
17:    } while (r <=1);
18:    r++; w++; }
19: System.out.println(name);

```

- A. ABC
- B. ABCABC
- C. ABCABCABC
- D. Line 15 contains a compilation error.
- E. Line 18 contains a compilation error.
- F. The code compiles but never terminates at runtime.
- G. The code compiles but throws a NullPointerException at runtime.

Explanation: While it may seem complicated, the key is to notice that the variable `r` is updated outside of the `do/while` loop. This is allowed from a compilation standpoint, since it is defined before the loop, but it means the innermost loop never breaks the termination condition `r <= 1`. At runtime, this will produce an infinite loop the first time the innermost loop is entered, making option F the correct answer.

27. What is printed by the following code snippet?

```

23: byte amphibian = 2;
24: String name = "Salamander";
25: String color = switch (amphibian) {
26:     case 1 -> { yield "Red"; }
27:     case 2 -> { if (name.equals("Frog")) yield "Green";
28:                 yield "Blue"; }
29:     case 3 -> { yield "Purple"; }
30:     default -> throw new RuntimeException();
31: };
32: System.out.print(color);

```

- A. Red
- B. Green
- C. Purple
- D. Blue
- E. The code does not compile.
- F. An exception is thrown at runtime.

28. What is the output of calling `getFish("goldie")`?

```

40: void getFish(Object fish) {
41:     if (!(fish instanceof String guppy))
42:         System.out.print("Eat!");
43:     else if (!(fish instanceof String guppy)) {
44:         throw new RuntimeException();
45:     }
46:     System.out.print("Swim!");
47: }

```

- A. Eat!
- B. Swim!
- C. Eat! followed by an exception
- D. Eat!Swim!
- E. An exception is printed
- F. None of the above

Explanation: Based on flow scoping, guppy is in scope after lines 41–42 if the type is not a String. In this case, line 43 declares a variable guppy that is a duplicate of the previously defined local variable defined on line 41. For this reason, the code does not compile, and option F is correct.

29. What is the result of the following code?

```
1: public class PrintIntegers {
2:     public static void main(String[] args) {
3:         int y = -2;
4:         do System.out.print(++y + " ");
5:         while (y <= 5);
6:     } }
```

- A. -2 -1 0 1 2 3 4 5
- B. -2 -1 0 1 2 3 4
- C. -1 0 1 2 3 4 5 6
- D. -1 0 1 2 3 4 5
- E. The code will not compile because of line 5.
- F. The code contains an infinite loop and does not terminate.

30. What is the minimum number of lines that would need to be changed or removed for the following code to compile and return a value when called with dance(10)?

```
41: double dance(Object speed) {
42:     return switch (speed) {
43:         case 5 -> {yield 4};
44:         case 10 -> 8;
45:         case 15,20 -> 12;
46:         default -> 20;
47:         case null -> 16;
48:     }
49: }
```

- A. Zero, the code compiles and runs without issue
- B. One
- C. Two
- D. Three
- E. Four
- F. Five
- G. Six

Explanation: Line 43 needs the semicolon after yield, and line 48 needs one after the return statement. Lines 43–45 fail to compile because numeric values aren't compatible with Object; changing line 41 to use Integer fixes this. Also, default on line 46 makes case null on line 47 unreachable, so it should be removed. With these four fixes, the code compiles, making option E correct.

Chapter 4: Core APIs

1. What is output by the following code?

```
1: public class Fish {
2:     public static void main(String[] args) {
3:         int numFish = 4;
4:         String fishType = "tuna";
5:         String anotherFish = numFish + 1;
6:         System.out.println(anotherFish + " " + fishType);
7:         System.out.println(numFish + " " + 1);
8:     } }
```

- A. 4 1
- B. 5
- C. 5 tuna
- D. 5tuna
- E. 51tuna
- F. The code does not compile.

Explanation: The problem is that we can't store an int in a String variable (Line 5)

2. Which of these array declarations are not legal? (Choose all that apply.)

- A. `int[][] scores = new int[5][];`
- B. `Object[][][] cubbies = new Object[3][0][5];`
- C. `String beans[] = new beans[6];`
- D. `java.util.Date[] dates[] = new java.util.Date[2][];`
- E. `int[][] types = new int[];`
- F. `int[][] java = new int[][];`

3. Note that March 12, 2028, is the weekend when we spring forward, and November 5, 2028, is when we fall back for daylight saving time. Which of the following can fill in the blank without the code throwing an exception? (Choose all that apply.)

```
var zone = ZoneId.of("US/Eastern");
var date = _____;
var time = LocalTime.of(2, 15);
var z = ZonedDateTime.of(date, time, zone);
```

- A. `LocalDate.of(2028, 3, 12)`
- B. `LocalDate.of(2028, 3, 40)`
- C. `LocalDate.of(2028, 11, 5)`
- D. `LocalDate.of(2028, 11, 6)`
- E. `LocalDate.of(2029, 2, 29)`
- F. `LocalDate.of(2028, MonthEnum.MARCH, 12);`

Explanation: Option B throws an exception because there is no March 40. Option E also throws an exception because 2029 isn't a leap year and therefore has no February 29. Option F doesn't compile because the enum should be named Month, rather than MonthEnum.

4. Which of the following are output by this code? (Choose all that apply.)

```
3: var s = "Hello";
4: var t = new String(s);
5: if ("Hello".equals(s)) System.out.println("one");
6: if (t == s) System.out.println("two");
7: if (t.intern() == s) System.out.println("three");
8: if ("Hello" == s) System.out.println("four");
9: if ("Hello".intern() == t) System.out.println("five");
```

- A. one
- B. two
- C. three
- D. four
- E. five
- F. The code does not compile.
- G. None of the above.

5. What is the result of the following code?

```
7: var sb = new StringBuilder();
8: sb.append("aaa").insert(1, "bb").insert(4, "ccc");
9: System.out.println(sb);
```

- A. abbaaccc
- B. abbaccca
- C. bbaaaccc
- D. bbaaccca
- E. An empty line.
- F. The code does not compile.

6. How many of these lines contain a compiler error?

```
23: double one = Math.pow(1, 2);
24: int two = Math.round(1.0);
25: float three = Math.random();
26: var doubles = new double[] {one, two, three};
```

- A. 0
- B. 1
- C. 2
- D. 3
- E. 4

Explanation: The round() method returns an int when called with a float. However, we are calling it with a double, so it returns a long. The random() method returns a double. Since two lines have a compiler error, option C is the answer.

7. Which of these statements is true of the two values? (Choose all that apply.)

```
2025-08-28T05:00 GMT-04:00  
2025-08-28T09:00 GMT-06:00
```

- A. The first date/time is earlier.
- B. The second date/time is earlier.
- C. Both date/times are the same.
- D. The date/times are two hours apart.
- E. The date/times are six hours apart.
- F. The date/times are 10 hours apart.

Explanation: The first date/time is 9:00 GMT, and the second is 15:00 GMT. Therefore, the first one is earlier by six hours.

8. Which of the following return 5 when run independently? (Choose all that apply.)

```
var string = "12345";  
var builder = new StringBuilder("12345");
```

- A. builder.charAt(4)
- B. builder.replace(2, 4, "6").charAt(3)
- C. builder.replace(2, 5, "6").charAt(2)
- D. string.charAt(5)
- E. string.length
- F. string.replace("123", "1").charAt(2)
- G. None of the above

9. Which of the following are true about arrays? (Choose all that apply.)

- A. The first element is index 0.
- B. The first element is index 1.
- C. Arrays are fixed size.
- D. Arrays are immutable.
- E. Calling equals() on two different arrays containing the same primitive values always returns true.
- F. Calling equals() on two different arrays containing the same primitive values always returns false.
- G. Calling equals() on two different arrays containing the same primitive values can return true or false.

Explanation: An array does not override equals(), so it uses object equality. Since two different objects are not equal, option F is correct, and options E and G are incorrect.

10. How many of these lines contain a compiler error?

```
23: int one = Math.min(5, 3);  
24: long two = Math.round(5.5);  
25: double three = Math.floor(6.6);  
26: var doubles = new double[] {one, two, three};
```

A. 0

B. 1

C. 2

D. 3

E. 4

11. What is the output of the following code?

```
var date = LocalDate.of(2025, 4, 3);
date.plusDays(2);
date.plusHours(3);
System.out.println(date.getYear() + " " + date.getMonth()
    + " " + date.getDayOfMonth());
```

A. 2025 MARCH 4

B. 2025 MARCH 6

C. 2025 APRIL 3

D. 2025 APRIL 5

E. The code does not compile.

F. A runtime exception is thrown.

Explanation: A `LocalDate` does not have a time element. Therefore, there is no method to add hours, making option E the answer.

12. What is output by the following code ignoring any new lines in the output? (Choose all that apply.)

```
var numbers = "012345678".indent(1);
numbers = numbers.stripLeading();
System.out.println(numbers.substring(1, 3));
System.out.println(numbers.substring(7, 7));
System.out.println(numbers.substring(7));
```

A. 12

B. 123

C. 7

D. 78

E. A blank line.

F. An exception is thrown.

13. What is the result of the following code?

```
public class Lion {
    public void roar(String roar1, StringBuilder roar2) {
        roar1.concat("!!!");
        roar2.append("!!!");
    }
    public static void main(String[] args) {
        var roar1 = "roar";
        var roar2 = new StringBuilder("roar");
        new Lion().roar(roar1, roar2);
        System.out.println(roar1 + " " + roar2);
    } }
```

- A. roar roar
- B. roar roar!!!
- C. roar!!! Roar
- D. roar!!! Roar!!!
- E. An exception is thrown.
- F. The code does not compile.

Explanation: String is immutable. Calling concat() returns a new String but does not change the original. A StringBuilder is mutable. Calling append() adds characters to the existing character sequence along with returning a reference to the same object.

14. Given the following, which can correctly fill in the blank allowing the code to compile? (Choose all that apply.)

```
var date = LocalDate.now();
var time = LocalTime.now();
var dateTime = date._____(time);
var zoneId = ZoneId.systemDefault();
var zonedDateTime = ZonedDateTime.of(dateTime, zoneId);
Instant instant = _____;
```

- A. asTime()
- B. atTime()
- C. withTime()
- D. dateTime.toInstant()
- E. new Instant()
- F. zonedDateTime.toInstant()

15. What is the output of the following? (Choose all that apply.)

```
var arr = new String[] { "PIG", "pig", "123"};
Arrays.sort(arr);
System.out.println(Arrays.toString(arr));
System.out.println(Arrays.binarySearch(arr, "Pippa"));
```

- A. [pig, PIG, 123]
- B. [PIG, pig, 123]
- C. [123, PIG, pig]
- D. [123, pig, PIG]
- E. -3
- F. -2
- G. The results of binarySearch() are undefined in this example.

Explanation: Numbers sort before letters, and uppercase sorts before lowercase. This makes option C one of the answers. The binarySearch() method looks at where a value would be inserted, which is before the second element for Pippa. It then negates it and subtracts one, which is option E.

16. Which of these statements are true? (Choose all that apply.)

```
var letters = new StringBuilder("abcdefg");
```

- A. letters.substring(1, 2) returns a single-character String.
- B. letters.substring(2, 2) returns a single-character String.
- C. letters.substring(6, 5) returns a single-character String.
- D. letters.substring(6, 6) returns a single-character String.
- E. letters.substring(1, 2) throws an exception.
- F. letters.substring(2, 2) throws an exception.
- G. letters.substring(6, 5) throws an exception.
- H. letters.substring(6, 6) throws an exception.

17. What is the result of the following code? (Choose all that apply.)

```
13: String s1 = ""
14:     purr"";
15: String s2 = "";
16:
17: s1.toUpperCase();
18: s1.trim();
19: s1.substring(1, 3);
20: s1 += "two";
21:
22: s2 += 2;
23: s2 += 'c';
24: s2 += false;
25:
26: if ( s2 == "2cfalse") System.out.println("==");
27: if ( s2.equals("2cfalse")) System.out.println("equals");
28: System.out.println(s1.length());
```

- A. 2
- B. 4
- C. 7
- D. 10
- E. ==
- F. equals
- G. An exception is thrown.
- H. The code does not compile.

18. Which of the following fill in the blank to print a positive integer? (Choose all that apply.)

```
String[] s1 = { "Camel", "Peacock", "Llama"};
String[] s2 = { "Camel", "Llama", "Peacock"};
String[] s3 = { "Camel"};
String[] s4 = { "Camel", null};
System.out.println(Arrays._____);
```

- A. compare(s1, s2)
- B. mismatch(s1, s2)
- C. compare(s3, s4)
- D. mismatch (s3, s4)
- E. compare(s4, s4)
- F. mismatch (s4, s4)

Explanation: The compare() method returns a positive integer when the arrays are different and the first is larger. The mismatch() method returns a positive integer when the arrays are different in a position index 1 or greater.

19. Note that March 12, 2028 is the weekend that clocks spring ahead for daylight saving time. What is the output of the following? (Choose all that apply.)

```
var date = LocalDate.of(2028, Month.MARCH, 12);
var time = LocalTime.of(1, 30);
var zone = ZoneId.of("US/Eastern");
var dateTime1 = ZonedDateTime.of(date, time, zone);
var dateTime2 = dateTime1.plus(1, ChronoUnit.HOURS);

long diff = ChronoUnit.HOURS.between(dateTime1, dateTime2);
int hour = dateTime2.getHour();
boolean offset = dateTime1.getOffset()
    == dateTime2.getOffset();
System.out.println("diff = " + diff);
System.out.println("hour = " + hour);
System.out.println("offset = " + offset);
```

- A. diff = 1
- B. diff = 2
- C. hour = 2
- D. hour = 3
- E. offset = true
- F. The code does not compile.
- G. A runtime exception is thrown.

Explanation: The dateTime1 object has a time of 1:30 per initialization. The dateTime2 object is an hour later. However, there is no 2:30 when springing ahead, setting the time to 3:30. Option A is correct because it is an hour later. Option D is also correct because the hour of the new time is 3. Option E is not correct because we have changed the time zone offset due to daylight saving time.

20. Which of the following can fill in the blank to print avaJ? (Choose all that apply.)

```
3: var puzzle = new StringBuilder("Java");
4: puzzle._____;
5: System.out.println(puzzle);
```

- A. reverse()
- B. append("vaJ\$").substring(0, 4)
- C. append("vaJ\$").delete(0, 3).deleteCharAt(puzzle.length() - 1)
- D. append("vaJ\$").delete(0, 3).deleteCharAt(puzzle.length())

E. None of the above

Explanation: Option C uses method chaining. First, it creates the value "JavavaJ\$". Then, it removes the first three characters, resulting in "avaJ\$". Finally, it removes the last character, resulting in "avaJ".

21. What is the output of the following code?

```
var date = LocalDate.of(2025, Month.APRIL, 30);  
date.plusDays(2);  
date.plusYears(3);  
System.out.println(date.getYear() + " " + date.getMonth()  
    + " " + date.getDayOfMonth());
```

- A. 2025 APRIL 30
- B. 2025 MAY 2
- C. 2028 APRIL 2
- D. 2028 APRIL 30
- E. 2028 MAY 2
- F. The code does not compile.
- G. A runtime exception is thrown.

Explanation: The date starts out as April 30, 2025. Since dates are immutable and the plus methods' return values are ignored, the result is unchanged. Therefore, option A is correct.

22. What is the output of the following?

```
var result = LocalDate.of(2025, Month.OCTOBER, 31)  
    .plusYears(1)  
    .plusMonths(-5)  
    .plusMonths(1)  
    .withYear(2026)  
    .atTime(LocalTime.of(13, 4));  
System.out.println(result);
```

- A. 2025-06-30T13:04
- B. 2026-04-304
- C. 2026-04-30T13:04
- D. 2026-06-30T
- E. 2026-06-30T13:04
- F. The code does not compile.
- G. A runtime exception is thrown.

Chapter 5: Methods

1. Which statements about the final modifier are correct? (Choose all that apply.)

- A. Instance and static variables can be marked final.
- B. A variable is effectively final only if it is marked final.
- C. An object that is marked final cannot be modified.
- D. Local variables cannot be declared with type var and the final modifier.
- E. A primitive that is marked final cannot be modified.

2. Which of the following can fill in the blank in this code to make it compile? (Choose all that apply.)

```
public class Ant {  
    _____ void method() {}  
}
```

- A. default
- B. final
- C. private
- D. Public
- E. String
- F. zzz:

3. Which of the following methods compile? (Choose all that apply.)

- A. final static void rain() {}
- B. public final int void snow() {}
- C. private void int hail() {}
- D. static final void sleet() {}
- E. void final ice() {}
- F. void public slush() {}

4. Which of the following can fill in the blank and allow the code to compile? (Choose all that apply.)

```
final _____ song = 6;
```

- A. int
- B. Integer
- C. long
- D. Long
- E. double
- F. Double

Explanation: The value 6 can be implicitly promoted to any of the primitive types, making options A, C, and E correct. It can also be autoboxed to Integer, making option B correct. It cannot be both promoted and autoboxed, making options D and F incorrect.

5. Which of the following methods compile? (Choose all that apply.)

- A. `public void january() { return; }`
- B. `public int february() { return null; }`
- C. `public void march() {}`
- D. `public int april() { return 9; }`
- E. `public int may() { return 9.0; }`
- F. `public int june() { return; }`

Explanation: Option E does not compile because it tries to return a double when the return type is int. Since a double cannot be assigned to an int, it cannot be returned as one either. Option F does not compile because no value is actually returned.

6. Which of the following methods compile? (Choose all that apply.)

- A. `public void violin(int... nums) {}`
- B. `public void viola(String values, int... nums) {}`
- C. `public void cello(int... nums, String values) {}`
- D. `public void bass(String... values, int... nums) {}`
- E. `public void flute(String[] values, ...int nums) {}`
- F. `public void oboe(String[] values, int[] nums) {}`

7. Given the following method, which of the method calls return 2? (Choose all that apply.)

```
public int juggle(boolean b, boolean... b2) {  
    return b2.length;  
}
```

- A. `juggle();`
- B. `juggle(true);`
- C. `juggle(true, true);`
- D. `juggle(true, true, true);`
- E. `juggle(true, {true, true});`
- F. `juggle(true, new boolean[2]);`

8. Which of the following statements is correct?

- A. Package access is more lenient than protected access.
- B. A public class that has private fields and package methods is not visible to classes outside the package.
- C. You can use access modifiers so only some of the classes in a package see a particular package class.
- D. You can use access modifiers to allow access to all methods and not any instance variables.
- E. You can use access modifiers to restrict access to all classes that begin with the word Test.

9. Given the following class definitions, which lines in the main() method generate a compiler error?
(Choose all that apply.)

```
// Classroom.java
package my.school;
public class Classroom {
    private int roomNumber;
    protected static String teacherName;
    static int globalKey = 54321;
    public static int floor = 3;
    Classroom(int r, String t) {
        roomNumber = r;
        teacherName = t; } }

// School.java
1: package my.city;
2: import my.school.*;
3: public class School {
4:     public static void main(String[] args) {
5:         System.out.println(Classroom.globalKey);
6:         Classroom room = new Classroom(101, "Mrs. Anderson");
7:         System.out.println(room.roomNumber);
8:         System.out.println(Classroom.floor);
9:         System.out.println(Classroom.teacherName); } }
```

A. None: the code compiles fine.

B. Line 5.

C. Line 6.

D. Line 7.

E. Line 8.

F. Line 9

10. What is the output of executing the Chimp program?

```
// Rope.java
1: package rope;
2: public class Rope {
3:     public static int LENGTH = 5;
4:     static {
5:         LENGTH = 10;
6:     }
7:     public static void swing() {
8:         System.out.print("swing ");
9:     } }

// Chimp.java
1: import rope.*;
2: import static rope.Rope.*;
3: public class Chimp {
4:     public static void main(String[] args) {
5:         Rope.swing();
6:         new Rope().swing(); }
```

```
7:      System.out.println(LENGTH) ;
8:    } }
```

- A. swing swing 5
- B. swing swing 10
- C. Compiler error on line 2 of Chimp
- D. Compiler error on line 5 of Chimp
- E. Compiler error on line 6 of Chimp
- F. Compiler error on line 7 of Chimp

11. Which statements are true of the following code? (Choose all that apply.)

```
1: public class Rope {
2:     public static void swing() {
3:         System.out.print("swing");
4:     }
5:     public void climb() {
6:         System.out.println("climb");
7:     }
8:     public static void play() {
9:         swing();
10:        climb();
11:    }
12:    public static void main(String[] args) {
13:        Rope rope = new Rope();
14:        rope.play();
15:        Rope rope2 = null;
16:        System.out.print("-");
17:        rope2.play();
18:    } }
```

- A. The code compiles as is.
- B. There is exactly one compiler error in the code.
- C. There are exactly two compiler errors in the code.
- D. If the line(s) with compiler errors are removed, the output is swing-climb.
- E. If the line(s) with compiler errors are removed, the output is swing-swing.
- F. If the line(s) with compile errors are removed, the code throws a NullPointerException

12. How many variables in the following method are effectively final?

```
10: public void feed() {
11:     int monkey = 0;
12:     if(monkey > 0) {
13:         var giraffe = monkey++;
14:         String name;
15:         name = "geoffrey";
16:     }
17:     String name = "milly";
18:     var food = 10;
19:     while(monkey <= 10) {
20:         food = 0;
```

```
21:    }
22:    name = null;
23: }
```

- A. 1.
- B. 2.
- C. 3.
- D. 4.
- E. 5.
- F. None of the above. The code does not compile.

Explanation: The test for effectively final is if the final modifier can be added to the local variable and the code still compiles. The monkey variable declared on line 11 is not effectively final because it is modified on line 13. The giraffe and name variables declared on lines 13 and 14, respectively, are effectively final and not modified after they are set. The name variable declared on line 17 is not effectively final since it is modified on line 22. Finally, the food variable on line 18 is not effectively final since it is modified on line 20.

13. What is the output of the following code?

```
// RopeSwing.java
import rope.*;
import static rope.Rope.*;
public class RopeSwing {
    private static Rope ropel = new Rope();
    private static Rope rope2 = new Rope();
    {
        System.out.println(ropel.length);
    }
    public static void main(String[] args) {
        ropel.length = 2;
        rope2.length = 8;
        System.out.println(ropel.length);
    }
}

// Rope.java
package rope;
public class Rope {
    public static int length = 0;
}
```

- A. 02
- B. 08
- C. 2
- D. 8
- E. The code does not compile.
- F. An exception is thrown.

Explanation: There are two details to notice in this code. First, note that RopeSwing has an instance initializer and not a static initializer. Since RopeSwing is never constructed, the instance initializer does not run. The other detail is that length is static. Changes from any object update this common static variable. The code prints 8, making option D correct.

14. How many lines in the following code have compiler errors?

```
1: public class RopeSwing {
2:     private static final String leftRope;
3:     private static final String rightRope;
4:     private static final String bench;
5:     private static final String name = "name";
6:     static {
7:         leftRope = "left";
8:         rightRope = "right";
9:     }
10:    static {
11:        name = "name";
12:        rightRope = "right";
13:    }
14:    public static void main(String[] args) {
15:        bench = "bench";
16:    }
17: }
```

- A. 0
- B. 1
- C. 2
- D. 3
- E. 4**
- F. 5

Explanation: If a variable is static final, it must be set exactly once, and it must be in the declaration line or in a static initialization block. Line 15 doesn't compile because final variables are not allowed to be set after that point.

15. Which of the following can replace line 2 to make this code compile?

```
1: import java.util.*;
2: // INSERT CODE HERE
3: public class Imports {
4:     public void method(ArrayList<String> list) {
5:         sort(list);
6:     }
7: }
```

- A. import static java.util.Collections;
- B. import static java.util.Collections.*;**
- C. import static java.util.Collections.sort(ArrayList<String>);
- D. static import java.util.Collections;
- E. static import java.util.Collections.*;
- F. static import java.util.Collections.sort(ArrayList<String>);

Explanation: The two valid ways to do this are import static java.util.Collections.*; and import static java.util.Collections.sort;; making option B correct. Option A is incorrect because you can do a static import only on static members. Classes such as Collections require a regular import

16. What is the result of the following statements?

```
1: public class Test {
2:     public void print(byte x) {
3:         System.out.print("byte-");
4:     }
5:     public void print(int x) {
6:         System.out.print("int-");
7:     }
8:     public void print(float x) {
9:         System.out.print("float-");
10:    }
11:    public void print(Object x) {
12:        System.out.print("Object-");
13:    }
14:    public static void main(String[] args) {
15:        Test t = new Test();
16:        short s = 123;
17:        t.print(s);
18:        t.print(true);
19:        t.print(6.789);
20:    }
21: }
```

- A. byte-float-Object-
- B. int-float-Object-
- C. byte-Object-float-
- D. int-Object-float-
- E. int-Object-Object-
- F. byte-Object-Object-

Explanation: The argument on line 17 is a short. It can be promoted to an int, so print() on line 5 is invoked. The argument on line 18 is a boolean. It can be autoboxed to a Boolean, so print() on line 11 is invoked. The argument on line 19 is a double. It can be autoboxed to a Double, so print() on line 11 is invoked.

17. What is the result of the following program?

```
1: public class Squares {
2:     public static long square(int x) {
3:         var y = x * (long) x;
4:         x = -1;
5:         return y;
6:     }
7:     public static void main(String[] args) {
8:         var value = 9;
9:         var result = square(value);
10:        System.out.println(value);
11:    }
12: }
```

```
11:    } }
```

- A. -1
- B. 9
- C. 81
- D. Compiler error on line 9
- E. Compiler error on a different line

18. Which of the following are output by the following code? (Choose all that apply.)

```
public class StringBuilders {  
    public static StringBuilder work(StringBuilder a,  
        StringBuilder b) {  
        a = new StringBuilder("a");  
        b.append("b");  
        return a;  
    }  
    public static void main(String[] args) {  
        var s1 = new StringBuilder("s1");  
        var s2 = new StringBuilder("s2");  
        var s3 = work(s1, s2);  
        System.out.println("s1 = " + s1);  
        System.out.println("s2 = " + s2);  
        System.out.println("s3 = " + s3);  
    }  
}
```

- A. s1 = a
- B. s1 = s1
- C. s2 = s2
- D. s2 = s2b
- E. s3 = a
- F. The code does not compile.

19. Which of the following will compile when independently inserted in the following code? (Choose all that apply.)

```
1:  public class Order3 {  
2:      final String value1 = "red";  
3:      static String value2 = "blue";  
4:      String value3 = "yellow";  
5:      {  
6:          // CODE SNIPPET 1  
7:      }  
8:      static {  
9:          // CODE SNIPPET 2  
10:     } }
```

- A. Insert at line 6: value1 = "green";
- B. Insert at line 6: value2 = "purple";
- C. Insert at line 6: value3 = "orange";

- D. Insert at line 9: value1 = "magenta";
- E. Insert at line 9: value2 = "cyan";
- F. Insert at line 9: value3 = "turquoise";

Explanation: The variable value1 is a final instance variable. It can be set only once: in the variable declaration, an instance initializer, or a constructor. The variable value2 is a static variable. Both instance and static initializers are able to access static variables, making options B and E correct. The variable value3 is an instance variable, making option C correct.

20. Which of the following are true about the following code? (Choose all that apply.)

```
public class Run {
    static void execute() {
        System.out.print("1-");
    }
    static void execute(int num) {
        System.out.print("2-");
    }
    static void execute(Integer num) {
        System.out.print("3-");
    }
    static void execute(Object num) {
        System.out.print("4-");
    }
    static void execute(int... nums) {
        System.out.print("5-");
    }
    public static void main(String[] args) {
        Run.execute(100);
        Run.execute(100L);
    }
}
```

- A. The code prints out 2-4-.
- B. The code prints out 3-4-.
- C. The code prints out 4-2-.
- D. The code prints out 4-4-.
- E. The code prints 3-4- if you remove the method static void execute(int num).
- F. The code prints 4-4- if you remove the method static void execute(int num).

21. Which method signatures are valid overloads of the following method signature? (Choose all that apply.)

```
public void moo(int m, int... n)
```

- A. public void moo(int a, int... b)
- B. public int moo(char ch)
- C. public void moooo(int... z)
- D. private void moo(int... x)
- E. public void moooo(int y)
- F. public void moo(int... c, int d)
- G. public void moo(int... i, int j...)

Explanation: When overloading methods, the return type and access modifiers do not need to be the same.

Chapter 6: Class Design

1. Which code can be inserted to have the code print 2?

```
public class BirdSeed {  
    private int numberBags;  
    boolean call;  
  
    public BirdSeed() {  
        // LINE 1  
        call = false;  
        // LINE 2  
    }  
  
    public BirdSeed(int numberBags) {  
        this.numberBags = numberBags;  
    }  
  
    public static void main(String[] args) {  
        var seed = new BirdSeed();  
        System.out.print(seed.numberBags);  
    } }
```

- A. Replace line 1 with BirdSeed(2);
 - B. Replace line 2 with BirdSeed(2);
 - C. Replace line 1 with new BirdSeed(2);
 - D. Replace line 2 with new BirdSeed(2);
 - E. Replace line 1 with this(2);
 - F. Replace line 2 with this(2);
 - G. The code prints 2 without any changes.
2. Which modifier pairs can be used together in a method declaration? (Choose all that apply.)
- A. static and final
 - B. private and static
 - C. static and abstract
 - D. private and abstract
 - E. abstract and final
 - F. private and final
3. Which of the following statements about methods are true? (Choose all that apply.)
- A. Overloaded methods must have the same signature.
 - B. Overridden methods must have the same signature.
 - C. Hidden methods must have the same signature.
 - D. Overloaded methods must have the same return type.
 - E. Overridden methods must have the same return type.
 - F. Hidden methods must have the same return type.

4. What is the output of the following program?

```
1:  class Mammal {
2:      private void sneeze() {}
3:      public Mammal(int age) {
4:          System.out.print("Mammal");
5:      } }
6:  public class Platypus extends Mammal {
7:      int sneeze() { return 1; }
8:      public Platypus() {
9:          System.out.print("Platypus");
10:     }
11:     public static void main(String[] args) {
12:         new Mammal(5);
13:     } }
```

- A. Platypus
- B. Mammal
- C. PlatypusMammal
- D. MammalPlatypus
- E. The code will compile if line 7 is changed.
- F. The code will compile if line 9 is changed.

5. Which of the following completes the constructor so that this code prints out 50?

```
class Speedster {
    int numSpots;
}
public class Cheetah extends Speedster {
    int numSpots;

    public Cheetah(int numSpots) {
        // INSERT CODE HERE
    }

    public static void main(String[] args) {
        Speedster s = new Cheetah(50);
        System.out.print(s.numSpots);
    }
}
```

- A. numSpots = numSpots;
- B. numSpots = this.numSpots;
- C. this.numSpots = numSpots;
- D. numSpots = super.numSpots;
- E. super.numSpots = numSpots;
- F. The code does not compile regardless of the code inserted into the constructor.
- G. None of the above.

6. Which of the following declare immutable classes? (Choose all that apply.)

```
public final class Moose {  
    private final int antlers;  
}  
  
public class Caribou {  
    private int antlers = 10;  
}  
  
public class Reindeer {  
    private final int antlers = 5;  
}  
  
public final class Elk {}  
  
public final class Deer {  
    private final Object o = new Object();  
}
```

- A. Moose
- B. Caribou
- C. Reindeer
- D. Elk**
- E. Deer**
- F. None of the above

7. What is the output of the following code?

```
1: class Arthropod {  
2:     protected void printName(long input) {  
3:         System.out.print("Arthropod");  
4:     }  
5:     void printName(int input) {  
6:         System.out.print("Spooky");  
7:     } }  
8: public class Spider extends Arthropod {  
9:     protected void printName(int input) {  
10:         System.out.print("Spider");  
11:     }  
12:     public static void main(String[] args) {  
13:         Arthropod a = new Spider();  
14:         a.printName((short)4);  
15:         a.printName(4);  
16:         a.printName(5L);  
17:     } }
```

- A. SpiderSpiderArthropod**
- B. SpiderSpiderSpider
- C. SpiderSpookyArthropod
- D. SpookySpiderArthropod
- E. The code will not compile because of line 5.

- F. The code will not compile because of line 9.
- G. None of the above.

8. What is the result of the following code?

```
1:  abstract class Bird {
2:      private final void fly() { System.out.println("Bird"); }
3:      protected Bird() { System.out.print("Wow-"); }
4:  }
5:  public class Pelican extends Bird {
6:      public Pelican() { System.out.print("Oh-"); }
7:      protected void fly() { System.out.println("Pelican"); }
8:      public static void main(String[] args) {
9:          var chirp = new Pelican();
10:         chirp.fly();
11:     } }
```

- A. Oh-Bird
- B. Oh-Pelican
- C. Wow-Oh-Bird
- D. Wow-Oh-Pelican**
- E. The code contains a compilation error.
- F. None of the above.

9. Which of the following statements about overridden methods are true? (Choose all that apply.)

- A. An overridden method must contain method parameters that are the same or covariant with the method parameters in the inherited method.
- B. An overridden method may declare a new exception, provided it is not checked.**
- C. An overridden method must be more accessible than the method in the parent class.
- D. An overridden method may declare a broader checked exception than the method in the parent class.
- E. If an inherited method returns void, then the overridden version of the method must return void.**
- F. None of the above.

10. Which of the following pairs, when inserted into the blanks, allow the code to compile? (Choose all that apply.)

```
1:  public class Howler {
2:      public Howler(long shadow) {
3:          _____;
4:      }
5:      private Howler(int moon) {
6:          super();
7:      }
8:  }
9:  class Wolf extends Howler {
10:     protected Wolf(String stars) {
11:         super(2L);
12:     }
13:     public Wolf() {
```

```

14:      _____;
15:    }
16: }

```

- A. this(3) at line 3, this("") at line 14.
- B. this() at line 3, super(1) at line 14.
- C. this((short)1) at line 3, this(null) at line 14.
- D. super() at line 3, super() at line 14.
- E. this(2L) at line 3, super((short)2) at line 14.
- F. this(5) at line 3, super(null) at line 14.
- G. Remove lines 3 and 14.

11. What is the result of the following?

```

1:  public class PolarBear {
2:      StringBuilder value = new StringBuilder("t");
3:      { value.append("a"); }
4:      { value.append("c"); }
5:      private PolarBear() {
6:          value.append("b");
7:      }
8:      public PolarBear(String s) {
9:          this();
10:         value.append(s);
11:     }
12:     public PolarBear(CharSequence p) {
13:         value.append(p);
14:     }
15:     public static void main(String[] args) {
16:         Object bear = new PolarBear();
17:         bear = new PolarBear("f");
18:         System.out.println(((PolarBear)bear).value);
19:     } }

```

- A. tacb
- B. tacf
- C. tacbf
- D. tcafb
- E. taftacb
- F. The code does not compile.
- G. An exception is thrown.

12. How many lines of the following program contain a compilation error?

```

1:  public class Rodent {
2:      public Rodent(Integer x) {}
3:      protected static Integer chew() throws Exception {
4:          System.out.println("Rodent is chewing");
5:          return 1;
6:      }
7:  }

```

```
8:  class Beaver extends Rodent { //NOT COMPILE
9:      public Number chew() throws RuntimeException { //NOT COMPILE
10:         System.out.println("Beaver is chewing on wood");
11:         return 2;
12:     } }
```

- A. None
- B. 1
- C. 2**
- D. 3
- E. 4
- F. 5

13. Which of these classes compile and will include a default constructor created by the compiler? (Choose all that apply.)

A.

```
public class Bird {}
```

B.

```
public class Bird {
    public bird() {}
}
```

C.

```
public class Bird {
    public bird(String name) {}
}
```

D.

```
public class Bird {
    public Bird() {}
}
```

E.

```
public class Bird {
    Bird(String name) {}
}
```

F.

```
public class Bird {
    private Bird(int age) {}
}
```

G.

```
public class Bird {
    public Bird bird() { return null; }
}
```

14. Which of the following statements about inheritance are correct? (Choose all that apply.)

- A. A class can directly extend any number of classes.
- B. A class can implement any number of interfaces.
- C. All variables inherit java.lang.Object.
- D. If class A is extended by B, then B is a superclass of A.
- E. If class C implements interface D, then C is a subtype of D.
- F. Multiple inheritance is the property of a class to have multiple direct superclasses.

15. Which statement about the following program is correct?

```
1: abstract class Nocturnal {
2:     boolean isBlind();
3: }
4: public class Owl extends Nocturnal {
5:     public boolean isBlind() { return false; }
6:     public static void main(String[] args) {
7:         var nocturnal = (Nocturnal)new Owl();
8:         System.out.println(nocturnal.isBlind());
9:     } }
```

- A. It compiles and prints true.
- B. It compiles and prints false.
- C. The code will not compile because of line 2.
- D. The code will not compile because of line 5.
- E. The code will not compile because of line 7.
- F. The code will not compile because of line 8.
- G. None of the above.

16. What is the result of the following?

```
1: class Arachnid {
2:     static StringBuilder sb = new StringBuilder();
3:     { sb.append("c"); }
4:     static
5:     { sb.append("u"); }
6:     { sb.append("r"); }
7: }
8: public class Scorpion extends Arachnid {
9:     static
10:    { sb.append("q"); }
11:    { sb.append("m"); }
12:    public static void main(String[] args) {
13:        System.out.print(Scorpion.sb + " ");
14:        System.out.print(Scorpion.sb + " ");
15:        new Arachnid();
16:        new Scorpion();
17:        System.out.print(Scorpion.sb);
18:    } }
```

- A. qu qu qumrcrc
- B. u u ucrcrm
- C. uq uq uqmcrcr
- D. uq uq uqcrerm
- E. qu qu qumrcrc
- F. qu qu qucrerm
- G. The code does not compile.

17. Which of the following are true? (Choose all that apply.)

- A. this() can be called from anywhere in a constructor.
- B. this() can be called from anywhere in an instance method.
- C. this.variableName can be called from any instance method in the class.
- D. this.variableName can be called from any static method in the class.
- E. You can call the default constructor written by the compiler using this().
- F. You can access a private constructor with the main() method in the same class.

18. Which statements about the following classes are correct? (Choose all that apply.)

```

1: public class Mammal {
2:     private void eat() {}
3:     protected static void drink() {}
4:     public Integer dance(String p) { return null; }
5: }
6: class Primate extends Mammal {
7:     public void eat(String p) {}
8: }
9: class Monkey extends Primate {
10:    public static void drink() throws RuntimeException {}
11:    public Number dance(CharSequence p) { return null; }
12:    public int eat(String p) {}
13: }

```

- A. The eat() method in Mammal is correctly overridden on line 7.
- B. The eat() method in Mammal is correctly overloaded on line 7.
- C. The drink() method in Mammal is correctly overridden on line 10.
- D. The drink() method in Mammal is correctly hidden on line 10.
- E. The dance() method in Mammal is correctly overridden on line 11.
- F. The dance() method in Mammal is correctly overloaded on line 11.
- G. The eat() method in Primate is correctly hidden on line 12.
- H. The eat() method in Primate is correctly overloaded on line 12.

19. What is the output of the following code?

```

1: class Reptile {
2:     {System.out.print("A");}
3:     public Reptile(int hatch) {}
4:     void layEggs() {
5:         System.out.print("Reptile");
6:     } }
7: public class Lizard extends Reptile {

```



```

8:      static {System.out.print("B");}
9:      public Lizard(int hatch) {}
10:     public final void layEggs() {
11:         System.out.print("Lizard");
12:     }
13:     public static void main(String[] args) {
14:         var reptile = new Lizard(1);
15:         reptile.layEggs();
16:     } }

```

- A. AALizard
- B. BALizard
- C. BLizardA
- D. ALizard
- E. The code will not compile because of line 3.
- F. None of the above.

20. Which statement about the following program is correct?

```

1:  class Bird {
2:      int feathers = 0;
3:      Bird(int x) { this.feathers = x; }
4:      Bird fly() {
5:          return new Bird(1);
6:      } }
7:  class Parrot extends Bird {
8:      protected Parrot(int y) { super(y); }
9:      protected Parrot fly() {
10:         return new Parrot(2);
11:     } }
12: public class Macaw extends Parrot {
13:     public Macaw(int z) { super(z); }
14:     public Macaw fly() {
15:         return new Macaw(3);
16:     }
17:     public static void main(String... sing) {
18:         Bird p = new Macaw(4);
19:         System.out.print(((Parrot)p.fly()).feathers);
20:     } }

```

- A. One line contains a compiler error.
- B. Two lines contain compiler errors.
- C. Three lines contain compiler errors.
- D. The code compiles but throws a ClassCastException at runtime.
- E. The program compiles and prints 3.
- F. The program compiles and prints 0.

21. Which of the following are properties of immutable classes? (Choose all that apply.)

- A. The class can contain setter methods, provided they are marked final.
- B. The class must not be able to be extended outside the class declaration.
- C. The class may not contain any instance variables.

- D. The class must be marked static.
- E. The class may not contain any static variables.
- F. The class may only contain private constructors.
- G. The data for mutable instance variables may be read, provided they cannot be modified by the caller.

22. What does the following program print?

```
1: class Person {
2:     static String name;
3:     void setName(String q) { name = q; } }
4: public class Child extends Person {
5:     static String name;
6:     void setName(String w) { name = w; }
7:     public static void main(String[] p) {
8:         final Child m = new Child();
9:         final Person t = m;
10:        m.name = "Elysia";
11:        t.name = "Sophia";
12:        m.setName("Webby");
13:        t.setName("Olivia");
14:        System.out.println(m.name + " " + t.name);
15:    } }
```

- A. Elysia Sophia
- B. Webby Olivia
- C. Olivia Olivia
- D. Olivia Sophia
- E. The code does not compile.
- F. None of the above.

23. What is the output of the following program?

```
1: class Canine {
2:     public Canine(boolean t) { logger.append("a"); }
3:     public Canine() { logger.append("q"); }
4:
5:     private StringBuilder logger = new StringBuilder();
6:     protected void print(String v) { logger.append(v); }
7:     protected String view() { return logger.toString(); }
8: }
9:
10: class Fox extends Canine {
11:     public Fox(long x) { print("p"); }
12:     public Fox(String name) {
13:         this(2);
14:         print("z");
15:     }
16: }
17:
```

```

18: public class Fennec extends Fox {
19:     public Fennec(int e) {
20:         super("tails");
21:         print("j");
22:     }
23:     public Fennec(short f) {
24:         super("eevee");
25:         print("m");
26:     }
27:
28:     public static void main(String... unused) {
29:         System.out.println(new Fennec(1).view());
30:     } }

```

- A. qpz
- B. qpzj**
- C. jzpa
- D. apj
- E. apjm
- F. The code does not compile.
- G. None of the above.

24. What is printed by the following program?

```

1: class Antelope {
2:     public Antelope(int p) {
3:         System.out.print("4");
4:     }
5:     { System.out.print("2"); }
6:     static { System.out.print("1"); }
7: }
8: public class Gazelle extends Antelope {
9:     public Gazelle(int p) {
10:         super(6);
11:         System.out.print("3");
12:     }
13:     public static void main(String hopping[]) {
14:         new Gazelle(0);
15:     }
16:     static { System.out.print("8"); }
17:     { System.out.print("9"); }
18: }

```

- A. 182640
- B. 182943
- C. 182493**
- D. 421389
- E. The code does not compile.
- F. The output cannot be determined until runtime.

25. Which of the following are true about a concrete class? (Choose all that apply.)

- A. A concrete class can be declared as abstract.
- B. A concrete class must implement all inherited abstract methods.
- C. A concrete class can be marked as final.
- D. A concrete class must be immutable.
- E. A concrete method that implements an abstract method must match the method declaration of the abstract method exactly.

26. What is the output of the following code?

```
4:  public abstract class Whale {
5:      public abstract void dive();
6:      public static void main(String[] args) {
7:          Whale whale = new Orca();
8:          whale.dive(3);
9:      }
10: }
11: class Orca extends Whale {
12:     static public int MAX = 3;
13:     public void dive() {
14:         System.out.println("Orca diving");
15:     }
16:     public void dive(int... depth) {
17:         System.out.println("Orca diving deeper "+MAX);
18: } }
```

- A. Orca diving
- B. Orca diving deeper 3
- C. The code will not compile because of line 4.
- D. The code will not compile because of line 8.
- E. The code will not compile because of line 11.
- F. The code will not compile because of line 12.
- G. The code will not compile because of line 17.
- H. None of the above.

Chapter 7: Beyond Classes

1. Which of the following are valid record declarations? (Choose all that apply.)

```
public record Iguana(int age) {
    private static final int age = 10; }

public final record Gecko() {}

public abstract record Chameleon() {
    private static String name; }

public record BeardedDragon(boolean fun) {
    @Override public boolean fun() { return false; } }

public record Reptile(long size) {
    public Reptile {
        if(size == 1) throw new IllegalArgumentException();
    } }

public record Newt(double age) extends Reptile {
    public Newt(double age) {
        age = this.age % 2 == 0 ? 5 : 10;
    } }
```

- A. Iguana
 - B. Gecko
 - C. Chameleon
 - D. BeardedDragon
 - E. Reptile
 - F. Newt
 - G. None of the above
2. Which of the following statements can be inserted in the blank line so that the code will compile successfully? (Choose all that apply.)

```
interface CanHop {}
public class Frog implements CanHop {
    public static void main(String[] args) {
        _____ frog = new TurtleFrog();
    }
}

class BrazilianHornedFrog extends Frog {}
class TurtleFrog extends Frog {}
```

- A. Frog
- B. TurtleFrog
- C. BrazilianHornedFrog
- D. CanHop

E. var

F. Long

G. None of the above; the code contains a compilation error.

3. What is the result of the following program?

```
11: public class Favorites {
12:     enum Flavors {
13:         VANILLA, CHOCOLATE, STRAWBERRY
14:         public Flavors() {}
15:     }
16:     public static void main(String[] args) {
17:         for(final var e : Flavors.values())
18:             System.out.print((e.ordinal() % 2) + " ");
19:     } }
```

A. 0 1 0

B. 1 0 1

C. Exactly one line of code does not compile.

D. More than one line of code does not compile.

E. The code compiles but produces an exception at runtime.

F. None of the above.

Explanation: Line 14 also does not compile, as enum constructors are implicitly private and cannot be declared as public or protected + semicolon on line 13

4. What is the output of the following program?

```
public sealed class ArmoredAnimal permits Armadillo {
    public ArmoredAnimal(int size) {}
    @Override public String toString() { return "Strong"; }
    public static void main(String[] a) {
        var c = new Armadillo(10, null);
        System.out.println(c);
    }
}
class Armadillo extends ArmoredAnimal {
    @Override public String toString() { return "Cute"; }
    public Armadillo(int size, String name) {
        super(size);
    }
}
```

A. Strong

B. Cute

C. The program does not compile.

D. The code compiles but produces an exception at runtime.

E. None of the above.

Explanation: A class extending a sealed class must be marked final, sealed, or non-sealed. Since Armadillo is missing a modifier, the code does not compile, and option C is correct.

5. Which statement about the following program is correct?

```
1: interface HasExoskeleton {
2:     double size = 2.0f;
3:     abstract int getNumberOfSections();
4: }
5: abstract class Insect implements HasExoskeleton {
6:     abstract int getNumberOfLegs();
7: }
8: public class Beetle extends Insect {
9:     int getNumberOfLegs() { return 6; }
10:    int getNumberOfSections(int count) { return 1; }
11: }
```

- A. It compiles without issue.
- B. The code will produce a ClassCastException if called at runtime.
- C. The code will not compile because of line 2.
- D. The code will not compile because of line 5.
- E. The code will not compile because of line 8.**
- F. The code will not compile because of line 10.

Explanation: The concrete class Beetle extends Insect and inherits two abstract methods, getNumberOfSections() and getNumberOfLegs(). The Beetle class includes an overloaded version of getNumberOfSections() that takes an int value but it does not satisfy the abstract method requirement inherited from HasExoskeleton. For this reason, only one of the two abstract methods is properly overridden.

6. Which statements about the following program are correct? (Choose all that apply.)

```
1: public abstract interface Herbivore {
2:     int amount = 10;
3:     public void eatGrass();
4:     public abstract int chew() { return 13; }
5: }
6:
7: abstract class IsAPlant extends Herbivore {
8:     Object eatGrass(int season) { return null; }
9: }
```

- A. It compiles and runs without issue.
- B. The code will not compile because of line 1.
- C. The code will not compile because of line 2.
- D. The code will not compile because of line 4.**
- E. The code will not compile because of line 7.**
- F. The code will not compile because line 8 contains an invalid method override.

Explanation: Line 4 does not compile, since an abstract method cannot include a body. Line 7 also does not compile because the wrong keyword is used. A class implements an interface; it does not extend it.

7. What is the output of the following program?

```
1: interface Aquatic {
2:     int getNumOfGills(int p);
3: }
4: public class ClownFish implements Aquatic {
5:     String getNumOfGills() { return "14"; }
6:     int getNumOfGills(int input) { return 15; }
7:     public static void main(String[] args) {
8:         System.out.println(new ClownFish().getNumOfGills(-1));
9:     } }
```

- A. 14
- B. 15
- C. The code will not compile because of line 4.
- D. The code will not compile because of line 5.
- E. The code will not compile because of line 6.
- F. None of the above.

Explanation: The inherited interface method `getNumOfGills(int)` is implicitly public; therefore, it must be declared public in any concrete class that implements the interface.

8. Given the following, select the statements that can be inserted into the blank line so that the code will compile and print true at runtime? (Choose all that apply.)

```
record Walrus(List<String> diet) {}
record Exhibit(Walrus animal, String location) {}

var e = new Exhibit(new Walrus(List.of("Wally")), "Artic");
System.out.print(e instanceof _____);
```

- A. `Exhibit(Walrus(List<Integer> z), Object a)`
- B. `Exhibit(Walrus(List m), Object n)`
- C. `Object w && w.animal().diet().size() == 0`
- D. `Exhibit(Walrus(var i), var i)`
- E. `Exhibit(var p, var q)`
- F. `Exhibit(List<?> g, var h)`
- G. `Exhibit(var x, CharSequence y)`
- H. `Exhibit(Walrus(null), var v)`
- I. None of the above

9. Which of the following statements can be inserted in the blank so that the code will compile successfully? (Choose all that apply.)

```
abstract class Snake {}
class Cobra extends Snake {}
class GardenSnake extends Cobra {}
public class SnakeHandler {
    private Snake snakey;
    public void setSnake(Snake mySnake) { this.snakey = mySnake; }
    public static void main(String[] args) {
```



```
new SnakeHandler().setSnake(____);  
} }
```

A. new Cobra()

B. new Snake()

C. new Object()

D. new String("Snake")

E. new GardenSnake()

F. null

G. None of the above. The class does not compile, regardless of the value inserted in the blank.

10. What types can be inserted in the blanks on the lines marked X and Z that allow the code to compile?
(Choose all that apply.)

```
interface Walk { private static List move() { return null; } }  
interface Run extends Walk { public ArrayList move(); }  
class Leopard implements Walk {  
    public _____ move() { // X  
        return null;  
    }  
}  
class Panther implements Run {  
    public move(_____) { // Z  
        return null;  
    }  
}
```

A. Integer on the line marked X

B. ArrayList on the line marked X

C. List on the line marked X

D. List on the line marked Z

E. ArrayList on the line marked Z

F. None of the above, since the Run interface does not compile.

G. Does not compile for a different reason.

11. What is the result of compiling and executing the following code?

```
1: public class Movie {  
2:     private int butter = 5;  
3:     private Movie() {}  
4:     protected class Popcorn {  
5:         private Popcorn() {}  
6:         public static int butter = 10;  
7:         public void startMovie() {  
8:             System.out.println(butter);  
9:         }  
10:    }  
11:    public static void main(String[] args) {  
12:        var movie = new Movie();  
13:        Movie.Popcorn in = new Movie().new Popcorn();  
14:        in.startMovie();  
15:    } }
```

- A. The output is 5.
- B. The output is 10.
- C. Line 6 generates a compiler error.
- D. Line 12 generates a compiler error.
- E. Line 13 generates a compiler error.
- F. The code compiles but produces an exception at runtime.

12. Which variables or members are accessible from within the hiss() method? (Choose all that apply.)

```

13: public class BoaConstrictor {
14:     private Body body;
15:     BoaConstrictor(Body b) { this.body = b; }
16:     private long tail = 10;
17:     record Body(int stripes) {
18:         private static int counter = 0;
19:         int counter() { return counter; }
20:         Body {
21:             stripes = stripes + counter++;
22:         }
23:         private void hiss() {} } }

```

- A. counter()
- B. tail
- C. body
- D. stripes()
- E. stripes
- F. counter
- G. Line 15 does not compile.
- H. Line 17 does not compile.
- I. Lines 20–22 do not compile.

13. What is the result of the following program?

```

public class Weather {
    enum Seasons {
        WINTER, SPRING, SUMMER, FALL
    }

    public static void main(String[] args) {
        Seasons v = null;
        switch (v) {
            case Seasons.SPRING -> System.out.print("s");
            case Seasons.WINTER -> System.out.print("w");
            case Seasons.SUMMER -> System.out.print("m");
            default -> System.out.println("missing data"); }
    } }

```

- A. s
- B. w
- C. m

- D. missing data
- E. Exactly one line of code does not compile.
- F. More than one line of code does not compile.
- G. The code compiles but produces an exception at runtime.

14. Which statements about sealed classes are correct? (Choose all that apply.)

- A. A sealed interface restricts which subinterfaces may extend it.
- B. A sealed class cannot be indirectly extended by a class that is not listed in its permits clause.
- C. A sealed class can be extended by an abstract class.
- D. A sealed class can be extended by a subclass that uses the nonsealed modifier.
- E. A sealed interface restricts which subclasses may implement it.
- F. A sealed class cannot contain any nested subclasses.
- G. None of the above.

15. Which line allows the code to print Not scared at runtime?

```
public class Ghost {
    public static void boo() {
        System.out.println("Not scared");
    }
    protected final class Spirit {
        public void boo() {
            System.out.println("Booo!!!");
        }
    }
    public static void main(String... haunt) {
        var g = new Ghost().new Spirit() {};
        _____;
    } }
}
```

- A. g.boo()
- B. g.super.boo()
- C. new Ghost().boo()
- D. g.Ghost.boo()
- E. new Spirit().boo()
- F. None of the above

Explanation: The Spirit class is marked final, so it cannot be extended. The main() method uses an anonymous class that inherits from Spirit, which is not allowed. If Spirit were not marked final, then option C would be correct.

16. The following code appears in a file named Ostrich.java. What is the result of compiling the source file?

```
1: public class Ostrich {
2:     private int count;
3:     static class OstrichWrangler {
4:         public int stampede() {
5:             return count;
6:         } } }
}
```

- A. The code compiles successfully, and one bytecode file is generated: Ostrich.class.
- B. The code compiles successfully, and two bytecode files are generated: Ostrich.class and OstrichWrangler.class.
- C. The code compiles successfully, and two bytecode files are generated: Ostrich.class and Ostrich\$OstrichWrangler.class.
- D. A compiler error occurs on line 3.
- E. A compiler error occurs on line 5.

Explanation: The OstrichWrangler class is a static nested class; therefore, it cannot access the instance member count.

17. Which lines of the following interface declarations do not compile? (Choose all that apply.)

```
1: public interface Omnivore {  
2:     int amount = 10;  
3:     static boolean gather = true;  
4:     static void eatGrass() {}  
5:     int findMore() { return 2; }  
6:     default float rest() { return 2; }  
7:     protected int chew() { return 13; }  
8:     private static void eatLeaves() {}  
9: }
```

- A. All of the lines compile without issue.
- B. Line 2.
- C. Line 3.
- D. Line 4.
- E. Line 5.
- F. Line 6.
- G. Line 7.
- H. Line 8.

Explanation: Non-static interface methods with a body must be explicitly marked private or default. Line 7 does not compile, as interfaces do not have protected members.

18. What is printed by the following program?

```
public class Deer {  
    enum Food {APPLES, BERRIES, GRASS}  
    protected class Diet {  
        private Food getFavorite() {  
            return Food.BERRIES;  
        }  
    }  
    public static void main(String[] seasons) {  
        System.out.print(switch(new Diet().getFavorite()) {  
            case APPLES -> "a";  
            case BERRIES -> "b";  
            default -> "c";  
        });  
    }  
}
```

- A. a
- B. b
- C. c
- D. The code declaration of the Diet class does not compile.
- E. The main() method does not compile.
- F. The code compiles but produces an exception at runtime.
- G. None of the above.

Explanation: Diet is an inner class, which requires an instance of Deer to instantiate. Since the main() method is static, there is no such instance.

19. Which of the following is printed by the Bear program?

```
public class Bear {
    enum FOOD {
        BERRIES, INSECTS {
            public boolean isHealthy() { return true; }},
        FISH, ROOTS, COOKIES, HONEY;
        public abstract boolean isHealthy();
    }
    public static void main(String[] args) {
        System.out.print(FOOD.INSECTS);
        System.out.print(FOOD.INSECTS.ordinal());
        System.out.print(FOOD.INSECTS.isHealthy());
        System.out.print(FOOD.COOKIES.isHealthy());
    } }
```

- A. insects
- B. Insects
- C. 0
- D. 1
- E. false
- F. The code does not compile.

Explanation: the isHealthy() method is marked abstract in the enum; therefore, it must be implemented in each enum value declaration. Since only INSECTS implements it, the code does not compile.

20. What is the output of this code?

```
13: record Gorilla(int x, Double y) {
14:     Gorilla {}
15:     Gorilla() { this(1,2.0); }
16: }
17: record Family(Gorilla parent1, Gorilla parent2) {}
18:
19: var family = new Family(
20:     new Gorilla(1, null), new Gorilla(0, 1.2));
21: System.out.print(switch (family) {
22:     case Family(var a, var b) -> "1";
23:     case Family(Gorilla c, Gorilla (int d, double e)) -> "2";
24:     case Family(Gorilla (int f, Double g), var h) -> "3";
25:     case Family(Gorilla i, Gorilla (int j, Double k)) -> "4";
```

```

26:     case Family(Object m, Object n) -> "5";
27:     case null -> "6";
28:     default -> "7";
29: });

```

- A. 1
- B. 2
- C. 3
- D. 4
- E. 5
- F. 6
- G. 7
- H. None of the above

Explanation: The record declarations compile but the switch expression does not. First, the second case statement does not compile, as double is not compatible with Double. Next, the pattern matching case statement on line 22 dominates the ones on lines 23–25.

21. Given the following record declaration, which line of code can fill in the blank and allow the code to compile?

```

public record RabbitFood(int size, String brand, LocalDate expires) {
    public static int MAX_STORAGE = 100;
    public RabbitFood() {
        _____;
    }
}

```

- A. size = MAX_STORAGE
- B. this.size = 10
- C. if(expires.isAfter(LocalDate.now())) throw new RuntimeException()
- D. if(brand==null) super.brand = "Unknown"
- E. throw new RuntimeException()
- F. None of the above

Explanation: The record defines an overloaded constructor using parentheses, not a compact one. For this reason, the first line must be a call to another constructor, such as this(500, "Acme", LocalDate.now()).

22. Which of the following can be inserted in the rest() method? (Choose all that apply.)

```

public class Lion {
    class Cub {}
    static class Den {}
    static void rest() {
        _____;
    }
}

```

- A. Cub a = Lion.new Cub()
- B. Lion.Cub b = new Lion().Cub()
- C. Lion.Cub c = new Lion().new Cub()
- D. var d = new Den()
- E. var e = Lion.new Cub()

- F. `Lion.Den f = Lion.new Den()`
- G. `Lion.Den g = new Lion.Den()`
- H. `var h = new Cub()`

23. Given the following program, what can be inserted into the blank line that would allow it to print Swim! at runtime?

```
interface Swim {
    default void perform() { System.out.print("Swim!"); }
}
interface Dance {
    default void perform() { System.out.print("Dance!"); }
}
public class Penguin implements Swim, Dance {
    public void perform() { System.out.print("Smile!"); }
    private void doShow() {
        _____;
    }
    public static void main(String[] eggs) {
        new Penguin().doShow();
    }
}
```

- A. `super.perform()`
- B. `Swim.perform()`
- C. `super.Swim.perform()`
- D. `Swim.super.perform()`
- E. The code does not compile regardless of what is inserted into the blank.
- F. The code compiles, but due to polymorphism, it is not possible to produce the requested output without creating a new object.

24. Which lines of the following interface do not compile? (Choose all that apply.)

```
1: public interface BigCat {
2:     abstract String getName();
3:     static int hunt() { getName(); return 5; }
4:     default void climb() { rest(); }
5:     private void roar() { getName(); climb(); hunt(); }
6:     private static boolean sneak() { roar(); return true; }
7:     private int rest() { return 2; };
8: }
```

- A. Line 2
- B. Line 3
- C. Line 4
- D. Line 5
- E. Line 6
- F. Line 7
- G. None of the above

Explanation: Line 3 does not compile because the static method `hunt()` cannot access an abstract instance method `getName()`. Line 6 does not compile because the private static method `sneak()` cannot access the private instance method `roar()`.

25. What does the following program print?

```
1: public class Zebra {
2:     private int x = 24;
3:     public int hunt() {
4:         String message = "x is ";
5:         abstract class Stripes {
6:             private int x = 0;
7:             public void print() {
8:                 System.out.print(message + Zebra.this.x);
9:             }
10:        }
11:        var s = new Stripes() {};
12:        s.print();
13:        return x;
14:    }
15:    public static void main(String[] args) {
16:        new Zebra().hunt();
17:    } }
```

- A. x is 0
- B. x is 24
- C. Line 6 generates a compiler error.
- D. Line 8 generates a compiler error.
- E. Line 11 generates a compiler error.
- F. None of the above.

26. What is the output of the following program?

```
20: public enum Animals {
21:     MAMMAL(List.of(2,4)),
22:     INVERTEBRATE(List.of(2, 4, 6, 8, 100)),
23:     BIRD(null) {
24:         public int stand() {
25:             return legs.get(0) + 4;
26:         }
27:     };
28:     List<Integer> legs;
29:     Animals(List<Integer> legs) {
30:         this.legs = legs;
31:     }
32:     public int stand() { return legs.get(0); }
33:     public static void main(String[] a) {
34:         Animals.BIRD.legs = List.of(-1);
35:         System.out.println(Animals.BIRD.stand());
36:     } }
```


- A. null
- B. -1
- C. 3
- D. 4
- E. Compiler error on lines 23.
- F. Compiler error on lines 24.
- G. Compiler error on line 34.
- H. The code compiles but produces a NullPointerException at runtime.
- I. None of the above.

27. Assuming a record is defined with at least one field, which components does the compiler always insert, each of which may be overridden or redeclared? (Choose all that apply.)

- A. A no-argument constructor
- B. An accessor method for each field
- C. The toString() method
- D. The equals() method
- E. A mutator method for each field
- F. A sort method for each field
- G. The hashCode() method

28. Which of the following classes and interfaces do not compile? (Choose all that apply.)

```
public abstract class Camel { void travel(); }

public interface EatsGrass { private abstract int chew(); }

public abstract class Elephant {
    abstract private class SleepsAlot {
        abstract int sleep();
    }
}

public class Eagle { abstract soar(); }

public interface Spider { default void crawl() {} }
```

- A. Camel
- B. EatsGrass
- C. Elephant
- D. Eagle
- E. Spider

29. How many lines of the following program contain a compilation error?

```
1: class Primate {
2:     protected int age = 2;
3:     { age = 1; }
4:     public Primate() {
5:         this().age = 3;
6:     }
7: }
```

```

8:  public class Orangutan {
9:      protected int age = 4;
10:     { age = 5; }
11:     public Orangutan() {
12:         this().age = 6;
13:     }
14:     public static void main(String[] bananas) {
15:         final Primate x = (Primate)new Orangutan();
16:         System.out.println(x.age);
17:     }
18: }

```

- A. None, and the program prints 1 at runtime.
- B. None, and the program prints 3 at runtime.
- C. None, but it causes a ClassCastException at runtime.
- D. 1
- E. 2
- F. 3**
- G. 4

Explanation: Both lines 5 and 12 do not compile, as `this()` is used instead of `this`. Remember, `this()` refers to calling a constructor, whereas `this` is a reference to the current instance. Next, the compiler does not allow casting to an unrelated class type. Since `Orangutan` is not a subclass of `Primate`, the cast on line 15 is invalid, and the code does not compile.

30. Assuming the following classes are declared as top-level types in the same file, which classes contain compiler errors? (Choose all that apply.)

```

sealed class Bird {
    public final class Flamingo extends Bird {}
}

sealed class Monkey {}

class EmperorTamarin extends Monkey {}

non-sealed class Mandrill extends Monkey {}

sealed class Friendly extends Mandrill permits Silly {}

final class Silly {}

```

- A. Bird
- B. Monkey
- C. EmperorTamarin**
- D. Mandrill
- E. Friendly**
- F. Silly
- G. All of the classes compile without issue.

Explanation: EmperorTamarin does not compile, as it is missing a non-sealed, sealed, or final modifier, making option C correct. Friendly also does not compile, since it lists a subclass Silly that does not extend it. While the permits clause is optional, the extends clause is not. Silly compiles just fine. Even though it does not extend Friendly, the compiler error is in the sealed class.

Chapter 8: Lambdas and Functional Interfaces

1. What is the result of the following class?

```
1: import java.util.function.*;
2:
3: public class Panda {
4:     int age;
5:     public static void main(String[] args) {
6:         Panda p1 = new Panda();
7:         p1.age = 1;
8:         check(p1, p -> p.age < 5);
9:     }
10:    private static void check(Panda panda,
11:        Predicate<Panda> pred) {
12:        String result =
13:            pred.test(panda) ? "match" : "not match";
14:        System.out.print(result);
15:    } }
```

A. match

B. not match

C. Compiler error on line 8

D. Compiler error on lines 10 and 11

E. Compiler error on lines 12 and 13

F. A runtime exception

2. What is the result of the following code?

```
1: interface Climb {
2:     boolean isTooHigh(int height, int limit);
3: }
4:
5: public class Climber {
6:     public static void main(String[] args) {
7:         check((h, m) -> h.append(m).isEmpty(), 5);
8:     }
9:     private static void check(Climb climb, int height) {
10:         if (climb.isTooHigh(height, 10))
11:             System.out.println("too high");
12:         else
13:             System.out.println("ok");
14:     }
15: }
```

A. ok

B. too high

C. Compiler error on line 7.

D. Compiler error on line 10.

E. Compiler error on a different line.

F. A runtime exception is thrown.

Explanation: The interface takes two int parameters. The code on line 7 attempts to use them as if h is a StringBuilder

3. Which statements about functional interfaces are true? (Choose all that apply.)

- A. A functional interface can contain default and private methods.
- B. A functional interface can be defined as a class or an interface.
- C. Abstract methods with signatures that are contained in public methods of java.lang.Object do not count toward the abstract method count for a functional interface.
- D. A functional interface cannot contain static or private static methods.
- E. A functional interface must be marked with the @FunctionalInterface annotation.

4. Which lambda can replace the MySecret class to return the same value? (Choose all that apply.)

```
interface Secret {  
    String magic(double d);  
}  
  
class MySecret implements Secret {  
    public String magic(double d) {  
        return "Poof";  
    }  
}
```

- A. (e) -> "Poof"
- B. (e) -> {"Poof"}
- C. (e) -> { String e = ""; "Poof" }
- D. (e) -> { String e = ""; return "Poof"; }
- E. (e) -> { String e = ""; return "Poof" }
- F. (e) -> { String f = ""; return "Poof"; }

5. Which of the following functional interfaces contain an abstract method that returns a primitive value? (Choose all that apply.)

- A. BooleanSupplier
- B. CharSupplier
- C. DoubleSupplier
- D. FloatSupplier
- E. IntSupplier
- F. StringSupplier

6. Which of the following lambda expressions can be passed to a function of Predicate<String> type? (Choose all that apply.)

- A. s -> s.isEmpty()
- B. s --> s.isEmpty()
- C. (String s) -> s.isEmpty()
- D. (String s) --> s.isEmpty()
- E. (StringBuilder s) -> s.isEmpty()
- F. (StringBuilder s) --> s.isEmpty()

7. Which of these statements is true about the following code?

```
public void method() {  
    x((var x) -> {}, (var x, var y) -> false);  
}  
public void x(Consumer<String> x, BinaryOperator<Boolean> y) {}
```

- A. The code does not compile because of one of the variables named x.
- B. The code does not compile because of one of the variables named y.
- C. The code does not compile for another reason.
- D. The code compiles, and the x in each lambda refers to the same type.
- E. The code compiles, and the x in each lambda refers to a different type.

8. Which of the following is equivalent to this code?

```
UnaryOperator<Integer> u = x -> x * x;
```

- A. BiFunction<Integer> f = x -> x*x;
- B. BiFunction<Integer, Integer> f = x -> x*x;
- C. BinaryOperator<Integer, Integer> f = x -> x*x;
- D. Function<Integer> f = x -> x*x;
- E. Function<Integer, Integer> f = x -> x*x;
- F. None of the above

9. Which statements are true? (Choose all that apply.)

- A. The Consumer interface is good for printing out an existing value.
- B. The Supplier interface is good for printing out an existing value.
- C. The IntegerSupplier interface returns an int.
- D. The Predicate interface returns an int.
- E. The Function interface has a method named test().
- F. The Predicate interface has a method named test().

10. Which of the following can be inserted without causing a compilation error? (Choose all that apply.)

```
public void remove(List<Character> chars) {  
    char end = 'z';  
    Predicate<Character> predicate = c -> {  
        char start = 'a'; return start <= c && c <= end; };  
  
    // INSERT LINE HERE  
}
```

- A. char start = 'a';
- B. char c = 'x';
- C. chars = null;
- D. end = '1';
- E. None of the above

11. How many times is true printed out by this code?

```
import java.util.function.Predicate;
public class Fantasy {
    public static void scary(String animal) {
        var dino = s -> "dino".equals(animal);
        var dragon = s -> "dragon".equals(animal);
        var combined = dino.or(dragon);
        System.out.println(combined.test(animal));
    }
    public static void main(String[] args) {
        scary("dino");
        scary("dragon");
        scary("unicorn");
    }
}
```

- A. One.
- B. Two.
- C. Three.
- D. The code does not compile.
- E. A runtime exception is thrown.

Explanation: The code does not compile because the lambdas are assigned to var. The compiler does not have enough information to determine they are of type Predicate<String>.

12. What does the following code output?

```
Function<Integer, Integer> s = a -> a + 4;
Function<Integer, Integer> t = a -> a * 3;
Function<Integer, Integer> c = s.compose(t);
System.out.print(c.apply(1));
```

- A. 7
- B. 15
- C. The code does not compile because of the data types in the lambda expressions.
- D. The code does not compile because of the compose() call.
- E. The code does not compile for another reason.

13. Which is true of the following code?

```
int length = 3;

for (int i = 0; i<3; i++) {
    if (i%2 == 0) {
        Supplier<Integer> supplier = () -> length; // A
        System.out.println(supplier.get()); // B
    } else {
        int j = i;
        Supplier<Integer> supplier = () -> j; // C
        System.out.println(supplier.get()); // D
    }
}
```

- A. The first compiler error is on line A.
- B. The first compiler error is on line B.
- C. The first compiler error is on line C.
- D. The first compiler error is on line D.
- E. The code compiles successfully.

14. Which of the following are valid lambda expressions? (Choose all that apply.)

- A. (Wolf w, var c) -> 39
- B. (final Camel c) -> {}
- C. (a,b,c) -> {int b = 3; return 2;}
- D. (x,y) -> new RuntimeException()
- E. (var y) -> return 0;
- F. () -> {float r}
- G. (Cat a, b) -> {}

15. Which lambda expression, when entered into the blank line in the following code, causes the program to print hahaha? (Choose all that apply.)

```
import java.util.function.Predicate;
public class Hyena {
    private int age = 1;
    public static void main(String[] args) {
        var p = new Hyena();
        double height = 10;
        int age = 1;
        testLaugh(p, _____);
        age = 2;
    }
    static void testLaugh(Hyena panda, Predicate<Hyena> joke) {
        var r = joke.test(panda) ? "hahaha" : "silence";
        System.out.print(r);
    }
}
```

- A. var -> p.age <= 10
- B. shenzi -> age==1
- C. p -> true
- D. age==1
- E. shenzi -> age==2
- F. h -> h.age < 5
- G. None of the above, as the code does not compile

16. Which of the following can be inserted without causing a compilation error?

```
public void remove(List<Character> chars) {
    char end = 'z';

    // INSERT LINE HERE
}
```



```
Predicate<Character> predicate = c -> {
    char start = 'a'; return start <= c && c <= end; };
}
```

- A. char start = 'a';
- B. char c = 'x';
- C. chars = null;
- D. end = '1';
- E. None of the above

17. What is the result of running the following class?

```
1: import java.util.function.*;
2:
3: public class Panda {
4:     int age;
5:     public static void main(String[] args) {
6:         Panda p1 = new Panda();
7:         p1.age = 1;
8:         check(p1, p -> {p.age < 5});
9:     }
10:    private static void check(Panda panda,
11:        Predicate<Panda> pred) {
12:        String result = pred.test(panda)
13:            ? "match" : "not match";
14:        System.out.print(result);
15:    } }
```

- A. match
- B. not match
- C. Compiler error on line 8.
- D. Compiler error on line 10.
- E. Compiler error on line 12.
- F. A runtime exception is thrown.

Explanation: C. Line 8 uses braces around the body. This means the return keyword and semicolon are required. Since the code doesn't compile, option C is the answer.

18. Which functional interfaces complete the following code? For line 7, assume m and n are instances of functional interfaces that exist and have the same type as y. (Choose three.)

```
6: _____ x = String::new;
7: _____ y = m.andThen(n);
8: _____ z = a -> a + a;
```

- A. BinaryConsumer<String, String>
- B. BiConsumer<String, String>
- C. BinaryFunction<String, String>
- D. BiFunction<String, String>
- E. Predicate<String>
- F. Supplier<String>

G. UnaryOperator<String>

H. UnaryOperator<String, String>

19. Which of the following compiles and prints out the entire set?

```
Set<?> set = Set.of("lion", "tiger", "bear");  
var s = Set.copyOf(set);  
Consumer<Object> consumer = _____;  
s.forEach(consumer);
```

A. () -> System.out.println(s)

B. s -> System.out.println(s)

C. (s) -> System.out.println(s)

D. System.out.println(s)

E. System::out::println

F. System.out::println

20. Which lambda can replace the new Sloth() call in the main() method and produce the same output at runtime?

```
import java.util.List;  
interface Yawn {  
    String yawn(double d, List<Integer> time);  
}  
class Sloth implements Yawn {  
    public String yawn(double zzz, List<Integer> time) {  
        return "Sleep: " + zzz;  
    }  
}  
public class Vet {  
    public static String takeNap(Yawn y) {  
        return y.yawn(10, null);  
    }  
    public static void main(String... unused) {  
        System.out.print(takeNap(new Sloth()));  
    }  
}
```

A. (z,f) -> { String x = ""; return "Sleep: " + x }

B. (t,s) -> { String t = ""; return "Sleep: " + t; }

C. (w,q) -> { "Sleep: " + w }

D. (e,u) -> { String g = ""; "Sleep: " + e }

E. (a,b) -> "Sleep: " + (double)(b==null ? a : a)

F. (r,k) -> { String g = ""; return "Sleep: "; }

G. None of the above, as the program does not compile

21. Which of the following are valid functional interfaces? (Choose all that apply.)

```
public interface Transport {  
    public int go();  
    public boolean equals(Object o);  
}
```

```
public abstract class Car {  
    public abstract Object swim(double speed, int duration);  
}  
  
public interface Locomotive extends Train {  
    public int getSpeed();  
}  
  
public interface Train extends Transport {}  
  
abstract interface Spaceship extends Transport {  
    default int blastOff();  
}  
  
public interface Boat {  
    int hashCode();  
    int hashCode(String input);  
}
```

- A. Boat
- B. Car
- C. Locomotive
- D. Spaceship
- E. Transport
- F. Train
- G. None of these is a valid functional interface.

Chapter 9: Collections and Generics

1. Suppose you need to display a collection of products for sale, which may contain duplicates. Additionally, you have a collection of sales that you need to track, sorted by the natural order of the sale ID, and you need to retrieve the text of each. Which two of the following classes best suit your needs for each of these scenarios? (Choose two.)

- A. ArrayList
- B. HashMap
- C. HashSet
- D. LinkedList
- E. SequencedTreeSet
- F. TreeMap

2. Which of the following are true? (Choose all that apply.)

```
12: List<?> q = List.of("mouse", "parrot");
13: var v = List.of("mouse", "parrot");
14:
15: q.removeIf(String::isEmpty);
16: q.removeIf(s -> s.length() == 4);
17: v.removeIf(String::isEmpty);
18: v.removeIf(s -> s.length() == 4);
```

- A. This code compiles and runs without error.
- B. Exactly one of these lines contains a compiler error.
- C. Exactly two of these lines contain a compiler error.
- D. Exactly three of these lines contain a compiler error.
- E. Exactly four of these lines contain a compiler error.
- F. If any lines with compiler errors are removed, this code runs without throwing an exception.
- G. If any lines with compiler errors are removed, this code throws an exception.

3. What is the result of the following statements?

```
3: var greetings = new ArrayDeque<String>();
4: greetings.offerLast("hello");
5: greetings.offerLast("hi");
6: greetings.offerFirst("ola");
7: greetings.pop();
8: greetings.peek();
9: while (greetings.peek() != null)
10:     System.out.print(greetings.pop());
```

- A. hello
- B. hellohi
- C. hellohiola
- D. hiola
- E. The code does not compile.
- F. An exception is thrown.

4. Which of these statements compile? (Choose all that apply.)

- A. `HashSet<Number> hs = new HashSet<Integer>();`
- B. `HashSet<? super ClassCastException> set = new HashSet<Exception>();`
- C. `List<> list = new ArrayList<String>();`
- D. `List<Object> values = new HashSet<Object>();`
- E. `List<Object> objects = new ArrayList<? extends Object>();`
- F. `Map<String, ? extends Number> hm = new HashMap<String, Integer>();`

5. What is the result of the following code?

```
1: public record Hello<T>(T t) {  
2:     public Hello(T t) { this.t = t; }  
3:     private <T> void println(T message) {  
4:         System.out.print(t + "-" + message);  
5:     }  
6:     public static void main(String[] args) {  
7:         new Hello<String>("hi").println(1);  
8:         new Hello("hola").println(true);  
9:     } }
```

- A. hi followed by a runtime exception.
- B. hi-1hola-true
- C. The first compiler error is on line 1.
- D. The first compiler error is on line 3.
- E. The first compiler error is on line 8.
- F. The first compiler error is on another line.

6. Which of the following can fill in the blank to print [7, 5, 3]? (Choose all that apply.)

```
8: public record Platypus(String name, int beakLength) {  
9:     @Override public String toString() {return "" + beakLength;}  
10:  
11:     public static void main(String[] args) {  
12:         Platypus p1 = new Platypus("Paula", 3);  
13:         Platypus p2 = new Platypus("Peter", 5);  
14:         Platypus p3 = new Platypus("Peter", 7);  
15:  
16:         List<Platypus> list = Arrays.asList(p1, p2, p3);  
17:  
18:         Collections.sort(list, Comparator.comparing_____);  
19:  
20:         System.out.println(list);  
21:     }  
22: }
```

A.

`(Platypus::beakLength)`

B.

```
(Platypus::beakLength).reversed()
```

C.

```
(Platypus::name)  
  .thenComparing(Platypus::beakLength)
```

D.

```
(Platypus::name)  
  .thenComparing(  
    Comparator.comparing(Platypus::beakLength)  
    .reversed())
```

E.

```
(Platypus::name)  
  .thenComparingNumber(Platypus::beakLength)  
  .reversed()
```

F.

```
(Platypus::name)  
  .thenComparingInt(Platypus::beakLength)  
  .reversed()
```

7. Which of the following method signatures are valid overrides of the hairy() method in the Alpaca class? (Choose all that apply.)

```
import java.util.*;  
  
public class Alpaca {  
    public List<String> hairy(List<String> list) { return null; }  
}
```

A.

```
public List<String> hairy(List<CharSequence> list) { return null; }
```

B.

```
public List<String> hairy(List<String> list) { return null; }
```

C.

```
public List<String> hairy(List<Integer> list) { return null; }
```

D.

```
public List<CharSequence> hairy(List<String> list) { return null; }
```

E.

```
public Object hairy(List<String> list) { return null; }
```

F.

```
public ArrayList<String> hairy(List<String> list) { return null; }
```

8. Which of the following fills in the blank, allowing the code to compile and run without issue?

```
11: SequencedCollection<String> animals = new _____<>();
12: animals.addFirst("lions");
13: animals.addLast("tigers");
14: for(var a : animals)
15:     System.out.println(a);
16: System.out.println(animals.get(0));
```

- A. HashSet
- B. LinkedList
- C. TreeSetMap
- D. HashMap
- E. None of the above

Explanation: There is no get(int) method defined in SequencedCollection, meaning line 16 does not compile.

9. What is the result of the following program?

```
3: public class MyComparator implements Comparator<String> {
4:     public int compare(String a, String b) {
5:         return b.toLowerCase().compareTo(a.toLowerCase());
6:     }
7:     public static void main(String[] args) {
8:         String[] values = { "123", "Abb", "aab" };
9:         Arrays.sort(values, new MyComparator());
10:        for (var s: values)
11:            System.out.print(s + " ");
12:    }
13: }
```

- A. Abb aab 123
- B. aab Abb 123
- C. 123 Abb aab
- D. 123 aab Abb
- E. The code does not compile.
- F. A runtime exception is thrown.

10. Which of these statements can fill in the blank so that the Helper class compiles successfully? (Choose all that apply.)

```
2: public class Helper {
3:     public static <U extends Exception>
4:         void printException(U u) {
```

```

5:
6:     System.out.println(u.getMessage());
7: }
8: public static void main(String[] args) {
9:     Helper. _____;
10: } }

```

- A. `printException(new FileNotFoundException("A"))`
- B. `printException(new Exception("B"))`
- C. `<Throwable>printException(new Exception("C"))`
- D. `<NullPointerException>printException(new NullPointerException("D"))`
- E. `printException(new Throwable("E"))`

11. Which of the following will compile when filling in the blank? (Choose all that apply.)

```

var list = List.of(1, 2, 3);
var set = Set.of(1, 2, 3);
var map = Map.of(1, 2, 3, 4);

_____.forEach(System.out::println);

```

- A. `list`
- B. `set`
- C. `map`
- D. `map.keys()`
- E. `map.keySet()`
- F. `map.values()`
- G. `map.valueSet()`

12. Which of these statements can fill in the blank so that the Wildcard class compiles successfully? (Choose all that apply.)

```

3: public class Wildcard {
4:     public void showSize(List<?> list) {
5:         System.out.println(list.size());
6:     }
7:     public static void main(String[] args) {
8:         Wildcard card = new Wildcard();
9:         _____;
10:        card.showSize(list);
11:    } }

```

- A. `List<?> list = new HashSet<String>()`
- B. `ArrayList<? super Date> list = new ArrayList<Date>()`
- C. `List<?> list = new ArrayList<?>()`
- D. `List<Exception> list = new LinkedList<java.io.IOException>()`
- E. `ArrayList<? extends Number> list = new ArrayList<Integer>()`
- F. None of the above

13. What is the result of the following program?

```
3: public record Sorted(int num, String text)
4:     implements Comparable<Sorted>, Comparator<Sorted> {
5:
6:     public String toString() { return "" + num; }
7:     public int compareTo(Sorted s) {
8:         return text.compareTo(s.text);
9:     }
10:    public int compare(Sorted s1, Sorted s2) {
11:        return s1.num - s2.num;
12:    }
13:    public static void main(String[] args) {
14:        var s1 = new Sorted(88, "a");
15:        var s2 = new Sorted(55, "b");
16:        SequencedSet<Sorted> t1 = new TreeSet<Sorted>();
17:        t1.add(s1); t1.add(s2);
18:        var t2 = new TreeSet<Sorted>(s1);
19:        t2.add(s1); t2.add(s2);
20:        System.out.println(t1 + " " + t2);
21:    } }
```

- A. [55, 88] [55, 88]
- B. [55, 88] [88, 55]
- C. [88, 55] [55, 88]
- D. [88, 55] [88, 55]
- E. The code does not compile.
- F. A runtime exception is thrown.

14. What is the result of the following code?

```
Comparator<Integer> c1 = (o1, o2) -> o2 - o1;
Comparator<Integer> c2 = Comparator.naturalOrder();
Comparator<Integer> c3 = Comparator.reverseOrder();

var list = Arrays.asList(5, 4, 7, 2);
Collections.sort(list, _____);
Collections.reverse(list);
Collections.reverse(list);
System.out.println(Collections.binarySearch(list, 2));
```

- A. One or more of the comparators can fill in the blank so that the code prints 0.
- B. One or more of the comparators can fill in the blank so that the code prints 1.
- C. One or more of the comparators can fill in the blank so that the code prints 2.
- D. The result is undefined regardless of which comparator is used.
- E. A runtime exception is thrown regardless of which comparator is used.
- F. The code does not compile.

15. Which of the following lines can be inserted to make the code compile? (Choose all that apply.)

```
class W {}
class X extends W {}
class Y extends X {}
class Z<Y> {
    // INSERT CODE HERE
}
```

- A. `W w1 = new W();`
- B. `W w2 = new X();`
- C. `W w3 = new Y();`
- D. `Y y1 = new W();`
- E. `Y y2 = new X();`
- F. `Y y3 = new Y();`

Explanation: Y is both a class and a type parameter. This means that within the class Z, when we refer to Y, it uses the type parameter. All of the choices that mention class Y are incorrect because it no longer means the class Y.

16. Which options are true of the following code? (Choose all that apply.)

```
_____ q = new LinkedList<>();
var u = Collections.unmodifiableCollection(q);
q.add(10);
q.add(12);
q.remove(1);
System.out.print(u);
```

- A. If we fill in the blank with `List<Integer>`, the output is `[10]`.
- B. If we fill in the blank with `Queue<Integer>`, the output is `[10]`.
- C. If we fill in the blank with `var`, the output is `[10]`.
- D. One or more of the scenarios does not compile.
- E. One or more of the scenarios throws a runtime exception.

17. What is the result of the following code?

```
4: Map m = new HashMap();
5: m.put(123, "456");
6: m.put("abc", "def");
7: System.out.println(m.contains("123"));
```

- A. `false`
- B. `true`
- C. Compiler error on line 4.
- D. Compiler error on line 5.
- E. Compiler error on line 7.
- F. A runtime exception is thrown.

Explanation: Map does not have a `contains()` method. It has `containsKey()` and `containsValue()` instead.

18. What is the result of the following code? (Choose all that apply.)

```
48: var map = Map.of(1,2, 3, 6);
49: var list = List.copyOf(map.entrySet());
50:
51: List<Integer> one = List.of(8, 16, 2);
52: var copy = List.copyOf(one);
53: var copyOfCopy = List.copyOf(copy);
54: var thirdCopy = new ArrayList<>(copyOfCopy);
55:
56: list.replaceAll(x -> x * 2);
57: one.replaceAll(x -> x * 2);
58: thirdCopy.replaceAll(x -> x * 2);
59:
60: System.out.println(thirdCopy);
```

- A. One line fails to compile.
- B. Two lines fail to compile.
- C. Three lines fail to compile.
- D. The code compiles but throws an exception at runtime.
- E. If any lines with compiler errors are removed, the code throws an exception at runtime.
- F. If any lines with compiler errors are removed, the code prints [16, 32, 4].
- G. The code compiles and prints [16, 32, 4] without any changes.

Explanation: Line 48 is a `Map<Integer, Integer>`. Line 49 builds a `List` out of a `Set` of `Entry` objects, giving us `List<Entry<Integer, Integer>>`. This causes a compiler error on line 56. Lines 51–54 are all of type `List<Integer>`. The first three are immutable, and the one on line 54 is mutable. This means line 57 throws an `UnsupportedOperationException` since we attempt to modify the list.

19. What code change is needed to make the method compile, assuming there is no class named `T`?

```
public static T identity(T t) {
    return t;
}
```

- A. Add `<T>` after the `public` keyword.
- B. Add `<T>` after the `static` keyword.
- C. Add `<T>` after `T`.
- D. Add `<?>` after the `public` keyword.
- E. Add `<?>` after the `static` keyword.
- F. No change is required. The code already compiles.

20. Assuming keys are printed in order, what is the result of the following?

```
var map = new HashMap<Integer, Integer>();
map.put(1, 10);
map.put(2, 20);
map.put(3, null);
map.merge(1, 3, (a,b) -> a + b);
map.merge(3, 3, (a,b) -> a + b);
System.out.println(map);
```

- A. {1=10, 2=20}
- B. {1=10, 2=20, 3=null}
- C. {1=10, 2=20, 3=3}
- D. {1=13, 2=20}
- E. {1=13, 2=20, 3=null}
- F. {1=13, 2=20, 3=3}
- G. The code does not compile.
- H. An exception is thrown.

21. Which of the following statements are true? (Choose all that apply.)

- A. Comparable is in the java.util package.
- B. Comparator is in the java.util package.
- C. compare() is in the Comparable interface.
- D. compare() is in the Comparator interface.
- E. compare() takes one method parameter.
- F. compare() takes two method parameters.

22. What is the output of the following code snippet?

```
21: SequencedMap<Integer, String> cats = new TreeMap<>();
22: cats.put(3, "Snowball");
23: cats.put(2, "Sugar");
24: cats.put(1, "Minnie Mouse");
25: cats.pollFirstEntry();
26: var id = cats.lastEntry().getKey();
27: cats.pollFirstEntry();
28: System.out.print(cats.firstEntry().getValue());
```

- A. Minnie Mouse
- B. Snowball
- C. Sugar
- D. The code does not compile.
- E. The code compiles, but an exception is thrown at runtime.

23. What is the output of the following code snippet?

```
var fishes = new TreeSet<String>();
fishes.add("Koi");
fishes.addFirst("clown");
fishes.add("carp");
for(var fish : fishes)
    System.out.print(fish + ", ");
```

- A. carp, clown, Koi,
- B. carp, Koi, clown,
- C. clown, carp, Koi,
- D. clown, Koi, carp,
- E. Koi, carp, clown,
- F. Koi, clown, carp,
- G. The code does not compile.
- H. The code compiles but throws an exception at runtime.

Chapter 10: Streams

1. What could be the output of the following?

```
var stream = Stream.iterate("", (s) -> s + "1");  
System.out.println(stream.limit(2).map(x -> x + "2"));
```

- A. 12112
- B. 212
- C. 212112
- D. java.util.stream.ReferencePipeline\$3@4517d9a3
- E. The code does not compile.
- F. An exception is thrown.
- G. The code hangs.

Explanation: No terminal operation is called, so the stream never executes.

2. What could be the output of the following?

```
Predicate<String> predicate = s -> s.startsWith("g");  
var stream1 = Stream.generate(() -> "growl!");  
var stream2 = Stream.generate(() -> "growl!");  
var b1 = stream1.anyMatch(predicate);  
var b2 = stream2.allMatch(predicate);  
System.out.println(b1 + " " + b2);
```

- A. true false
- B. true true
- C. java.util.stream.ReferencePipeline\$3@4517d9a3
- D. The code does not compile.
- E. An exception is thrown.
- F. The code hangs.

Explanation: Both streams created in this code snippet are infinite streams. The variable b1 is set to true since anyMatch() terminates. Even though the stream is infinite, Java finds a match on the first element and stops looking. However, when allMatch() runs, it needs to keep going until the end of the stream since it keeps finding matches.

3. What could be the output of the following?

```
Predicate<String> predicate = s -> s.length() > 3;  
var stream = Stream.iterate("-",  
    s -> ! s.isEmpty(), (s) -> s + s);  
var b1 = stream.noneMatch(predicate);  
var b2 = stream.anyMatch(predicate);  
System.out.println(b1 + " " + b2);
```

- A. false false
- B. false true
- C. java.util.stream.ReferencePipeline\$3@4517d9a3
- D. The code does not compile.
- E. An exception is thrown.

F. The code hangs.

Explanation: An infinite stream is generated where each element is twice as long as the previous one. While this code uses the three-parameter `iterate()` method, the condition is never false. The variable `b1` is set to false because Java finds an element that matches when it gets to the element of length 4. However, the next line tries to operate on the same stream. Since streams can be used only once, this throws an exception that the “stream has already been operated upon or closed”.

4. Which are true statements about terminal operations in a stream that runs successfully? (Choose all that apply.)

A. At most one terminal operation can exist in a stream pipeline.

B. Terminal operations are a required part of the stream pipeline in order to get a result.

C. Terminal operations have `Stream` as the return type.

D. The `peek()` method is an example of a terminal operation.

E. The referenced `Stream` may be used after calling a terminal operation.

5. Which of the following sets result to 8.0? (Choose all that apply.)

A.

```
double result = LongStream.of(6L, 8L, 10L)
    .mapToInt(x -> (int) x)
    .collect(Collectors.groupingBy(x -> x))
    .keySet()
    .stream()
    .collect(Collectors.averagingInt(x -> x));
```

B.

```
double result = LongStream.of(6L, 8L, 10L)
    .mapToInt(x -> x)
    .boxed()
    .collect(Collectors.groupingBy(x -> x))
    .keySet()
    .stream()
    .collect(Collectors.averagingInt(x -> x));
```

C.

```
double result = LongStream.of(6L, 8L, 10L)
    .mapToInt(x -> (int) x)
    .boxed()
    .collect(Collectors.groupingBy(x -> x))
    .keySet()
    .stream()
    .collect(Collectors.averagingInt(x -> x));
```

D.

```
double result = LongStream.of(6L, 8L, 10L)
    .mapToInt(x -> (int) x)
    .collect(Collectors.groupingBy(x -> x, Collectors.toSet()))
    .keySet()
    .stream()
    .collect(Collectors.averagingInt(x -> x));
```

E.

```
double result = LongStream.of(6L, 8L, 10L)
    .mapToInt(x -> x)
    .boxed()
    .collect(Collectors.groupingBy(x -> x, Collectors.toSet()))
    .keySet()
    .stream()
    .collect(Collectors.averagingInt(x -> x));
```

F.

```
double result = LongStream.of(6L, 8L, 10L)
    .mapToInt(x -> (int) x)
    .boxed()
    .collect(Collectors.groupingBy(x -> x, Collectors.toSet()))
    .keySet()
    .stream()
    .collect(Collectors.averagingInt(x -> x));
```

Explanation: options B and E are incorrect because they do not cast the long to an int, resulting in a compiler error on the mapToInt() calls. Options A and D are incorrect because they are missing boxed() before the collect() call. Since groupingBy() is creating a Collection, we need a nonprimitive Stream.

6. Which of the following methods can fill in the blank so that the code prints out false?

```
var s = Stream.generate(() -> "meow");
var match = s._____ (String::isEmpty);
System.out.println(match);
```

A. Only allMatch

B. Only anyMatch

C. Only noneMatch

D. Both allMatch and anyMatch

E. Both allMatch and noneMatch

F. None of the above

Explanation: The anyMatch() and noneMatch() methods run infinitely. The stream has no way to know that a match won't show up later. Option A is correct because only allMatch() makes it safe to return false as soon as one element passes through the stream that doesn't match.

7. We have a method that returns a sorted list without changing the original. We want to rewrite it. Which of the following pairs can fill in the blanks in `refactored()` to do the same with streams?

```
private static List<String> sort(List<String> list) {
    var copy = new ArrayList<String>(list);
    Collections.sort(copy, (a, b) -> b.compareTo(a));
    return copy;
}

private static List<String> refactored(List<String> list) {
    return list.stream()
        ._____((a, b) -> b.compareTo(a))
        ._____;
}
```

- A. `compare` and `toList()`
- B. `compare` and `sort()`
- C. `compareTo` and `toList()`
- D. `compareTo` and `sort()`
- E. `sorted` and `collect()`
- F. `sorted` and `collect(Collectors.toList())`

Explanation: There is no `Stream<T>` method called `compare()` or `compareTo()`, so options A through D can be eliminated. The `sorted()` method is correct to use in a stream pipeline to return a sorted `Stream` and fills in the first blank. Either `toList()` or `collect(Collectors.toList())` can fill in the second blank.

8. Which of the following are true given this declaration? (Choose all that apply.)

```
var is = IntStream.empty();
```

- A. `is.average()` returns the type `int`.
- B. `is.average()` returns the type `OptionalInt`.
- C. `is.findAny()` returns the type `int`.
- D. `is.findAny()` returns the type `OptionalInt`.
- E. `is.sum()` returns the type `int`.
- F. `is.sum()` returns the type `OptionalInt`.

Explanation: The `average()` method returns an `OptionalDouble` since averages of any type can result in a fraction. The `findAny()` method returns an `OptionalInt` because there might not be any elements to find. The `sum()` method returns an `int` rather than an `OptionalInt` because the sum of an empty stream is zero.

9. Which of the following can we add after line 6 for the code to run without error and not produce any output? (Choose all that apply.)

```
4: var stream = LongStream.of(1, 2, 3);
5: var opt = stream.map(n -> n * 10)
6:     .filter(n -> n < 5).findFirst();
```

A.

```
if (opt.isPresent())
    System.out.println(opt.get());
```


B.

```
if (opt.isPresent())  
    System.out.println(opt.getAsLong());
```

C.

```
opt.ifPresent(System.out::println);
```

D.

```
opt.ifPresent(System.out::println);
```

E. None of these; the code does not compile.

F. None of these; line 6 throws an exception at runtime.

10. Given the four statements (L, M, N, O), select the order that would cause the code to output 10 lines.

```
Stream.generate(() -> "1")  
  L: .filter(x -> x.length() > 1)  
  M: .forEach(System.out::println)  
  N: .limit(10)  
  O: .peek(System.out::println);
```

- A. L, N
- B. L, N, O
- C. L, N, M
- D. L, N, M, O
- E. L, O, M
- F. N, M
- G. N, O

Explanation: Only one of the method calls, `forEach()`, is a terminal operation, so any answer in which M is not the last line will not execute the pipeline.

11. What changes need to be made together for this code to print the string 12345? (Choose all that apply.)

```
Stream.iterate(1, x -> x++)  
  .limit(5).map(x -> x)  
  .collect(Collectors.joining());
```

- A. Changing `Collectors.joining()` to `Collectors.joining(",")`
- B. Changing `map(x -> x)` to `map(x -> "" + x)`
- C. Changing `x -> x++` to `x -> ++x`
- D. Adding `.forEach(System.out::print)` after the call to `collect()`
- E. Wrapping the entire line in a `System.out.print` statement
- F. None of the above; the code already prints 12345

Explanation: As written, the code doesn't compile because the `Collectors.joining()` expects to get a `Stream<String>`. Option B fixes this, at which point nothing is output because the collector creates a String without outputting the result. Option E fixes this and causes the output to be 11111. Since the post-increment operator is used, the stream contains an infinite number of the character 1. Option C fixes this and causes the stream to contain increasing numbers.

12. Which is true of the following code?

```
Set<String> birds = Set.of("oriole", "flamingo");
Stream.concat(birds.stream(), birds.stream(), birds.stream())
    .sorted()           // line X
    .distinct()
    .findAny()
    .ifPresent(System.out::println);
```

- A. It is guaranteed to print flamingo as is and when line X is removed.
- B. It is guaranteed to print oriole as is and when line X is removed.
- C. It is guaranteed to print flamingo as is, but not when line X is removed.
- D. It is guaranteed to print oriole as is, but not when line X is removed.
- E. The output may vary as is.
- F. The code does not compile.
- G. It throws an exception because the same list is used as the source for multiple streams.

Explanation: The code does not compile because Stream.concat() takes two parameters, not the three provided.

13. Which of the following is true?

```
List<Integer> x1 = List.of(1, 2, 3);
List<Integer> x2 = List.of(4, 5, 6);
List<Integer> x3 = List.of();
Stream.of(x1, x2, x3).map(x -> x + 1)
    .flatMap(x -> x.stream())
    .forEach(System.out::print);
```

- A. The code compiles and prints 123456.
- B. The code compiles and prints 234567.
- C. The code compiles but does not print anything.
- D. The code compiles but prints stream references.
- E. The code runs infinitely.
- F. The code does not compile.
- G. The code throws an exception.

Explanation: If the map() and flatMap() calls were reversed, option B would be correct. In this case, the Stream created from the source is of type Stream<List>. Trying to use the addition operator (+) on a List is not supported in Java.

14. Which of the following are true? (Choose all that apply.)

```
4: Stream<Integer> s = Stream.of(1);
5: IntStream is = s.boxed();
6: DoubleStream ds = s.mapToDouble(x -> x);
7: Stream<Integer> s2 = ds.mapToInt(x -> x);
8: s2.forEach(System.out::print);
```

- A. Line 4 causes a compiler error.
- B. Line 5 causes a compiler error.
- C. Line 6 causes a compiler error.
- D. Line 7 causes a compiler error.

- E. Line 8 causes a compiler error.
- F. The code compiles but throws an exception at runtime.
- G. The code compiles and prints 1.

Explanation: `boxed()` is available only on primitive streams like `IntStream`, not `Stream<Integer>`. Line 7 does not compile for two reasons. First, converting from a double to an int would require an explicit cast. Also, `mapToInt()` returns an `IntStream`, so the data type of `s2` is incorrect.

15. Given the generic type `String`, the `partitioningBy()` collector creates a `Map<Boolean, List<String>>` when passed to `collect()` by default. When a downstream collector is passed to `partitioningBy()`, which return types can be created? (Choose all that apply.)
- A. `Map<boolean, List<String>>`
 - B. `Map<Boolean, List<String>>`
 - C. `Map<Boolean, Map<String>>`
 - D. `Map<Boolean, Set<String>>`
 - E. `Map<Long, TreeSet<String>>`
 - F. None of the above
16. Which of the following statements are true about this code? (Choose all that apply.)

```
20: Predicate<String> empty = String::isEmpty;
21: Predicate<String> notEmpty = empty.negate();
22:
23: var result = Stream.generate(() -> "")
24:     .limit(10)
25:     .filter(notEmpty)
26:     .collect(Collectors.groupingBy(k -> k))
27:     .entrySet()
28:     .stream()
29:     .map(Entry::getValue)
30:     .flatMap(Collection::stream)
31:     .collect(Collectors.partitioningBy(notEmpty));
32: System.out.println(result);
```

- A. It outputs `{}`.
 - B. It outputs `{false=[], true=[]}`.
 - C. If we changed line 31 from `partitioningBy(notEmpty)` to `groupingBy(n -> n)`, it would output `{}`.
 - D. If we changed line 31 from `partitioningBy(notEmpty)` to `groupingBy(n -> n)`, it would output `{false=[], true=[]}`.
 - E. The code does not compile.
 - F. The code compiles but does not terminate at runtime.
17. What is the result of the following?

```
var s = DoubleStream.of(1.2, 2.4);
s.peek(System.out::println).filter(x -> x > 2).count();
```

- A. 1
- B. 2
- C. 2.4
- D. 1.2 and 2.4
- E. There is no output.

- F. The code does not compile.
- G. An exception is thrown.

Explanation: The terminal operation is count(). Since there is a terminal operation, the intermediate operations run. The peek() operation comes before the filter(), so both numbers are printed.

18. What is the output of the following?

```
11: public class Paging {
12:     record Sesame(String name, boolean human) {
13:         @Override public String toString() {
14:             return name();
15:         }
16:     }
17:     record Page(List<Sesame> list, long count) {}
18:
19:     public static void main(String[] args) {
20:         var monsters = Stream.of(new Sesame("Elmo", false));
21:         var people = Stream.of(new Sesame("Abby", true));
22:         printPage(monsters, people);
23:     }
24:
25:     private static void printPage(Stream<Sesame> monsters,
26:         Stream<Sesame> people) {
27:         Page page = Stream.concat(monsters, people)
28:             .collect(Collectors.teeing(
29:                 Collectors.filtering(s -> s.name().startsWith("E"),
30:                     Collectors.toList()),
31:                 Collectors.counting(),
32:                 (l, c) -> new Page(l, c)));
33:         System.out.println(page);
34:     } }
```

- A. Page[list=[Abby], count=1]
- B. Page[list=[Abby], count=2]
- C. Page[list=[Elmo], count=1]
- D. Page[list=[Elmo], count=2]**
- E. The code does not compile due to Stream.concat().
- F. The code does not compile due to Collectors.teeing().
- G. The code does not compile for another reason.

Explanation: Since line 29 filters by names starting with E, that rules out options A and B. Finally, line 31 counts the entire list, which is of size 2, giving us option D as the answer.

19. What is the simplest way of rewriting this code?

```
List<Integer> x = IntStream.range(1, 6)
    .mapToObj(i -> i)
    .collect(Collectors.toList());
x.forEach(System.out::println);
```

A.

```
IntStream.range(1, 6);
```

B.

```
IntStream.range(1, 6)
    .forEach(System.out::println);
```

C.

```
IntStream.range(1, 6)
```

D.

```
.mapToObj(i -> i)
    .forEach(System.out::println);
```

E. None of the above is equivalent.

F. The provided code does not compile.

20. Which of the following throw an exception when an Optional is empty? (Choose all that apply.)

A. `opt.orElse("")`;

B. `opt.orElseGet(() -> "")`;

C. `opt.orElseThrow()`;

D. `opt.orElseThrow(() -> throw new Exception())`;

E. `opt.orElseThrow(RuntimeException::new)`;

F. `opt.get()`;

G. `opt.get("")`;

Explanation: Option D looks correct but will compile only if the throw is removed. Remember, the `orElseThrow()` should get a lambda expression or method reference that returns an exception, not one that throws an exception.

21. What is the output of the following?

```
var spliterator = Stream.generate(() -> "x")
    .spliterator();

spliterator.tryAdvance(System.out::print);
var split = spliterator.trySplit();
split.tryAdvance(System.out::print);
```

A. x

B. xx

C. A long list of x's.

D. There is no output.

E. The code does not compile.

F. The code compiles but does not terminate at runtime.

Chapter 11: Exceptions and Localization

1. Which of the following can be inserted on line 8 to make this code compile? (Choose all that apply.)

```
7: public void whatHappensNext() throws IOException {  
8:     // INSERT CODE HERE  
9: }
```

- A. `System.out.println("it's ok");`
- B. `throw new Exception();`
- C. `throw new IllegalArgumentException();`
- D. `throw new java.io.IOException();`
- E. `throw new RuntimeException();`
- F. None of the above

Explanation: Unchecked exceptions can be thrown in any method. Option B is incorrect because a broader exception is not allowed.

2. Which statement about the following class is correct?

```
1: class Problem extends Exception {  
2:     public Problem() {}  
3: }  
4: class YesProblem extends Problem {}  
5: public class MyDatabase {  
6:     public static void connectToDatabase() throw Problem {  
7:         throws new YesProblem();  
8:     }  
9:     public static void main(String[] c) throw Exception {  
10:         connectToDatabase();  
11:     }  
12: }
```

- A. The code compiles and prints a stack trace for YesProblem at runtime.
- B. The code compiles and prints a stack trace for Problem at runtime.
- C. The code does not compile because Problem defines a constructor.
- D. The code does not compile because YesProblem does not define a constructor.
- E. The code does not compile but would if Problem and YesProblem were switched on lines 6 and 7.
- F. None of the above.

Explanation: The code does not compile because the throw and throws keywords are incorrectly used on lines 6, 7, and 9.

3. Which of the following are common types to localize? (Choose all that apply.)

- A. Dates
- B. Lambda expressions
- C. Class names
- D. Currency
- E. Numbers
- F. Variable names

4. What is the output of the following snippet, assuming a and b are both 0?

```
3: try {
4:     System.out.print(a / b);
5: } catch (RuntimeException e) {
6:     System.out.print(-1);
7: } catch (ArithmeticException e) {
8:     System.out.print(0);
9: } finally {
10:    System.out.print("done");
11: }
```

- A. -1
- B. 0
- C. done-1
- D. done0
- E. The code does not compile.
- F. An uncaught exception is thrown.
- G. None of the above.

Explanation: The order of catch blocks is important because they're checked in the order they appear after the try block. Because `ArithmeticException` is a child class of `RuntimeException`, the catch block on line 7 is unreachable.

5. Assuming the current locale uses dollars (\$) and the following method is called with a double value of 100_102.2, which of the following values are printed? (Choose all that apply.)

```
public void print(double t) {

    System.out.print(NumberFormat.getCompactNumberInstance().format(t));

    System.out.print(
        NumberFormat.getCompactNumberInstance(
            Locale.getDefault(), Style.SHORT).format(t));

    System.out.print(NumberFormat.getCurrencyInstance().format(t));
}
```

- A. 100
- B. \$100,000.00
- C. 100K
- D. 100 thousand
- E. 100M
- F. \$100,102.20
- G. None of the above

Explanation: When a `CompactNumberFormat` instance is requested without a style, it uses the `SHORT` style by default. This results in both of the first two statements printing 100K. Option F is also correct, as the full value is printed with a currency formatter.

6. What is the output of the following code?

```
LocalDate date = LocalDate.parse("2025-04-30",  
    DateTimeFormatter.ISO_LOCAL_DATE_TIME);  
System.out.println(date.getYear() + " "  
    + date.getMonth() + " " + date.getDayOfMonth());
```

- A. 2025 APRIL 2
- B. 2025 APRIL 30
- C. 2025 MAY 2
- D. The code does not compile.
- E. A runtime exception is thrown.

Explanation: A `LocalDate` does not have a time element. Therefore, a date/time formatter is not appropriate. If `ISO_LOCAL_DATE` were used, the code would print 2025 APRIL 30.

7. What does the following method print?

```
11: public void tryAgain(String s) {  
12:     try (FileReader r = null, p = new FileReader("")) {  
13:         System.out.print("X");  
14:         throw new IllegalArgumentException();  
15:     } catch (Exception s) {  
16:         System.out.print("A");  
17:         throw new FileNotFoundException();  
18:     } finally {  
19:         System.out.print("O");  
20:     }  
21: }
```

- A. XAO
- B. XOA
- C. One line of this method contains a compiler error.
- D. Two lines of this method contain compiler errors.
- E. Three or more lines of this method contain compiler errors.
- F. The code compiles, but a `NullPointerException` is thrown at runtime.
- G. None of the above.

Explanation: The first compiler error is on line 12 because each resource in a try-with-resources statement must have its own data type and be separated by a semicolon (;). Line 15 does not compile because the variable `s` is already declared in the method. Line 17 also does not compile. The `FileNotFoundException`, which inherits from `IOException` and `Exception`, is a checked exception, so it must be handled or declared by the method.

8. Assume that all of the files mentioned in the answer choices exist and define the same keys. Which one will be used to find the key in line 8?

```
6: Locale.setDefault(Locale.of("en", "US"));  
7: var b = ResourceBundle.getBundle("Dolphins");  
8: System.out.println(b.getString("name"));
```

- A. `Dolphins.properties`
- B. `Dolphins_US.properties`

C. Dolphins_en.properties

D. Whales.properties

E. Whales_en_US.properties

F. The code does not compile.

9. For what value of pattern will the following print <005.21> <008.49> <1,234.0>?

```
String pattern = "_____";  
var message = DoubleStream.of(5.21, 8.49, 1234)  
    .mapToObj(v -> new DecimalFormat(pattern).format(v))  
    .collect(Collectors.joining("> <"));  
System.out.println("<"+message+">");
```

A. ##.#

B. 0,000.0#

C. #,###.0

D. #,###,000.0#

E. The code does not compile regardless of what is placed in the blank.

F. None of the above.

10. Which scenario is the best use of an exception?

A. An element is not found when searching a list.

B. An unexpected parameter is passed into a method.

C. The computer caught fire.

D. You want to loop through a list.

E. You don't know how to code a method.

11. Which of the following exceptions must be handled or declared in the method in which they are thrown?
(Choose all that apply.)

```
class Apple extends RuntimeException {}  
class Orange extends Exception {}  
class Banana extends Error {}  
class Pear extends Apple {}  
class Tomato extends Orange {}  
class Peach extends Throwable {}
```

A. Apple

B. Orange

C. Banana

D. Pear

E. Tomato

F. Peach

Explanation: An exception that must be handled or declared is a checked exception. A checked exception inherits Exception but not RuntimeException. Option F is also correct, as a class that inherits Throwable but not RuntimeException or Error is also checked.

12. Which of the following changes, when made independently, would make this code compile? (Choose all that apply.)

```
1:  import java.io.*;
2:  public class StuckTurkeyCage implements AutoCloseable {
3:      public void close() throws IOException {
4:          throw new FileNotFoundException("Cage not closed");
5:      }
6:      public static void main(String[] args) {
7:          try (StuckTurkeyCage t = new StuckTurkeyCage()) {
8:              System.out.println("put turkeys in");
9:          }
10:     } }
```

- A. Remove throws IOException from the declaration on line 3.
 - B. Add throws Exception to the declaration on line 6.
 - C. Change line 9 to } catch (Exception e) {}.
 - D. Change line 9 to } finally {}.
 - E. The code compiles as is.
 - F. None of the above.
13. Which of the following are true statements about exception handling in Java? (Choose all that apply.)
- A. A traditional try statement without a catch block requires a finally block.
 - B. A traditional try statement without a finally block requires a catch block.
 - C. A traditional try statement with only one statement can omit the {}.
 - D. A try-with-resources statement without a catch block requires a finally block.
 - E. A try-with-resources statement without a finally block requires a catch block.
 - F. A try-with-resources statement with only one statement can omit the {}.

Explanation: A try-with-resources statement does not require a catch or finally block. A traditional try statement requires at least one of the two.

14. Assuming -g:vars is used when the code is compiled to include debug information, what is the output of the following code snippet?

```
var huey = (String)null;
Integer dewey = null;
Object louie = null;
if(louie == huey.substring(dewey.intValue())) {
    System.out.println("Quack!");
}
```

- A. A NullPointerException that does not include any variable names in the stack trace
- B. A NullPointerException naming huey in the stack trace
- C. A NullPointerException naming dewey in the stack trace
- D. A NullPointerException naming louie in the stack trace
- E. A NullPointerException naming huey and louie in the stack trace
- F. A NullPointerException naming huey and dewey in the stack trace
- G. None of the above

15. Which of the following, when inserted independently in the blank, use locale parameters that are properly formatted? (Choose all that apply.)

```
import java.util.Locale;
public class ReadMap implements AutoCloseable {
    private Locale locale;
    private boolean closed = false;
    @Override public void close() {
        System.out.println("Folding map");
        locale = null;
        closed = true;
    }
    public void open() {
        this.locale = _____;
    }
    public void use() {
        // Implementation omitted
    }
}
```

- A. `Locale.of("xM")`
- B. `Locale.of("MQ", "ks")`
- C. `Locale.of("qw")`
- D. `Locale.of("wp", "VW")`
- E. `Locale.create("zp")`
- F. `new Locale.Builder().setLanguage("yw").setRegion("PM")`
- G. The code does not compile regardless of what is placed in the blank.

16. Which of the following can be inserted into the blank to allow the code to compile and run without throwing an exception?

```
var f = DateTimeFormatter.ofPattern("hh o'clock");
System.out.println(f.format(_____.now()));
```

- A. `ZonedDateTime`
- B. `LocalDate`
- C. `LocalTimestamp`
- D. `LocalTime`
- E. The code does not compile regardless of what is placed in the blank.
- F. `None of the above`

17. Which of the following statements about resource bundles are correct? (Choose all that apply.)

- A. All keys must be in the same resource bundle to be used.
- B. A resource bundle is loaded by calling the new `ResourceBundle()` constructor.
- C. Resource bundle values are always read using the `Properties` class.
- D. Changing the default locale lasts for only a single run of the program.
- E. If a resource bundle for a specific locale is requested, then the resource bundle for the default locale will not be used.
- F. It is possible to use a resource bundle for a locale without specifying a default locale.

Explanation: Option A is incorrect because Java will look at parent bundles if a key is not found in a specified resource bundle. Option B is incorrect because resource bundles are loaded from static factory methods. Option C is incorrect, as resource bundle values are read from the ResourceBundle object directly. Option E is incorrect, as the resource bundle for the default locale may be used if there is no resource bundle for the specified locale (or its locale without a country code). Finally, option F is correct. The JVM will set a default locale automatically.

18. What is the output of the following code?

```
import java.io.*;
public class FamilyCar {
    static class Door implements AutoCloseable {
        public void close() {
            System.out.print("D");
        }
    }
    static class Window implements Closeable {
        public void close() {
            System.out.print("W");
            throw new RuntimeException();
        }
    }
    public static void main(String[] args) {
        var d = new Door();
        try (d; var w = new Window()) {
            System.out.print("T");
        } catch (Exception e) {
            System.out.print("E");
        } finally {
            System.out.print("F");
        }
    }
}
```

- A. TWF
- B. TWDF
- C. TWDEF
- D. TWF followed by an exception
- E. TWDF followed by an exception
- F. TWEF followed by an exception
- G. The code does not compile.

19. Suppose that we have the following three properties files and code. Which bundles are used on lines 8 and 9, respectively?

```
Dolphins.properties
name=The Dolphin
age=0

Dolphins_en.properties
name=Dolly
age=4

Dolphins_fr.properties
name=Dolly
```

```
5: var fr = Locale.of("fr");
6: Locale.setDefault(Locale.of("en", "US"));
7: var b = ResourceBundle.getBundle("Dolphins", fr);
8: b.getString("name");
9: b.getString("age");
```

- A. Dolphins.properties and Dolphins.properties
- B. Dolphins.properties and Dolphins_en.properties
- C. Dolphins_en.properties and Dolphins_en.properties
- D. Dolphins_fr.properties and Dolphins.properties
- E. Dolphins_fr.properties and Dolphins_en.properties
- F. The code does not compile.
- G. None of the above.

20. What is printed by the following program?

```
1: public class DriveBus {
2:     public void go() {
3:         System.out.print("A");
4:         try {
5:             stop();
6:         } catch (ArithmeticException e) {
7:             System.out.print("B");
8:         } finally {
9:             System.out.print("C");
10:        }
11:        System.out.print("D");
12:    }
13:    public void stop() {
14:        System.out.print("E");
15:        Object x = null;
16:        x.toString();
17:        System.out.print("F");
18:    }
19:    public static void main(String n[]) {
20:        new DriveBus().go();
21:    } }
```

- A. AE
- B. AEBCD
- C. AEC
- D. AECD
- E. AE followed by a stack trace
- F. AEBCD followed by a stack trace
- G. AEC followed by a stack trace
- H. A stack trace with no other output

21. Which change allows the following program to compile?

```
1: public class AhChoo {
2:     static class SneezeException extends Exception {}
3:     static class SniffleException extends SneezeException {}
4:     public static void main(String[] args) {
5:         try {
6:             throw new SneezeException();
7:         } catch (SneezeException | SniffleException e) {
8:         } finally {}
9:     } }
```

- A. Add throws SneezeException to the declaration on line 4.
- B. Add throws Throwable to the declaration on line 4.
- C. Change line 7 to } catch (SneezeException e) {.
- D. Change line 7 to } catch (SniffleException e) {.
- E. Remove line 7.
- F. The code compiles correctly as is.
- G. None of the above.

Explanation: The code does not compile because the multi-catch block on line 7 cannot catch both a superclass and a related subclass.

22. What is the output of the following code?

```
try {
    LocalDateTime book = LocalDateTime.of(2025, 4, 5, 12, 30, 20);
    System.out.print(book.format(DateTimeFormatter.ofPattern("m")));
    System.out.print(book.format(DateTimeFormatter.ofPattern("z")));
    System.out.print(DateTimeFormatter.ofPattern("y").format(book));
} catch (Throwable e) {}
```

- A. 4
- B. 30
- C. 402
- D. 3002
- E. 3002025
- F. 402025
- G. None of the above

23. Fill in the blank: A class that implements _____ may be in a try-with-resources statement.
(Choose all that apply.)

- A. AutoCloseable
- B. Resource
- C. Exception
- D. AutomaticResource
- E. Closeable
- F. RuntimeException
- G. Serializable

24. What is the output of the following program?

```
public class SnowStorm {
    static class WalkToSchool implements AutoCloseable {
        public void close() {
            throw new RuntimeException("flurry");
        }
    }
    public static void main(String[] args) {
        WalkToSchool walk1 = new WalkToSchool();
        try (walk1; WalkToSchool walk2 = new WalkToSchool()) {
            throw new RuntimeException("blizzard");
        } catch (Exception e) {
            System.out.println(e.getMessage()
                + " " + e.getSuppressed().length);
        }
        walk1 = null;
    }
}
```

- A. blizzard 0
- B. blizzard 1
- C. blizzard 2
- D. flurry 0
- E. flurry 1
- F. flurry 2
- G. None of the above

Explanation: The code does not compile because the resource walk1 is not final or effectively final and cannot be used in the declaration of a try-with-resources statement.

25. Assuming U.S. currency is in dollars (\$) and German currency is in euros (€), what is the output of the following program?

```
import java.text.NumberFormat;
import java.util.Locale;
import java.util.Locale.Category;
public record Wallet(double money) {
    private String openWallet() {
        Locale.setDefault(Category.DISPLAY,
            new Locale.Builder().setRegion("us").build());
        Locale.setDefault(Category.FORMAT,
            new Locale.Builder().setLanguage("en").build());
        return NumberFormat.getCurrencyInstance(Locale.GERMANY)
            .format(money);
    }
    public void printBalance() {
        System.out.println(openWallet());
    }
    public static void main(String... unused) {
        new Wallet(2.4).printBalance();
    }
}
```

- A. 2,40 €
- B. \$2.40
- C. 2.4
- D. The code does not compile.
- E. None of the above.

26. Which lines can fill in the blank to make the following code compile? (Choose all that apply.)

```
void rollOut() throws ClassCastException {}

public void transform(String c) {
    try {
        rollOut();
    } catch (IllegalArgumentException | _____) {
    }
}
```

- A. IOException a
- B. Error b
- C. NullPointerException c
- D. RuntimeException d
- E. NumberFormatException e
- F. ClassCastException f
- G. None of the above. The code contains a compiler error regardless of what is inserted into the blank.

Chapter 12: Modules

1. Which statement is true of the following module?

```
|---zoo
  |-- staff
    |-- Vet.java
```

- A. The directory structure shown is a valid module.
 - B. The directory structure would be a valid module if module.java were added directly underneath zoo/staff.
 - C. The directory structure would be a valid module if module.java were added directly underneath zoo.
 - D. The directory structure would be a valid module if module-info.java were added directly underneath zoo/staff.
 - E. The directory structure would be a valid module if module-info.java were added directly underneath zoo.
 - F. None of these changes would make this directory structure a valid module
2. Suppose module puppy depends on module dog and module dog depends on module animal. Fill in the blank so that code in module dog can access the animal.behavior package in module animal.

```
module animal {
    _____ animal.behavior;
}
```

- A. export
 - B. exports
 - C. require
 - D. requires
 - E. require transitive
 - F. requires transitive
 - G. None of the above
3. Fill in the blanks so this command to run the program is correct:

```
java
_____ modules
_____ zoo.animal.talks/zoo/animal/talks/Peacocks
```

- A. -d and -m
- B. -d and -p
- C. -m and -d
- D. -m and -p
- E. -p and -d
- F. -p and -m
- G. None of the above

Explanation: The -p or --module-path option is used to specify the location of the modules. The -m or --module option is used to specify the module and class name. However, running a program requires the package name to be specified with periods (.) instead of slashes.

4. Which of the following pairs make up a service?
- A. Consumer and service locator
 - B. Consumer and service provider interface
 - C. Service locator and service provider
 - D. Service locator and service provider interface
 - E. Service provider and service provider interface

Explanation: A service consists of the service provider interface and logic to look up implementations using a service locator.

5. A(n) _____ module is on the classpath while a(n) _____ module is on the module path. (Choose all that apply.)
- A. automatic, named
 - B. automatic, unnamed
 - C. named, automatic
 - D. named, unnamed
 - E. unnamed, automatic
 - F. unnamed, named
 - G. None of the above

Explanation: Automatic modules are on the module path but do not have a module-info.java file. Named modules are on the module path and do have a module-info.java. Unnamed modules are on the classpath.

6. Which of the following statements are true in a module-info.java file? (Choose all that apply.)
- A. The opens directive allows the use of reflection.
 - B. The opens directive declares that an API is called.
 - C. The use directive allows the use of reflection.
 - D. The use directive declares that an API is called.
 - E. The uses directive allows the use of reflection.
 - F. The uses directive declares that an API is called.

Explanation: Options C and D are incorrect because there is no use directive. Options A and F are correct because opens is for reflection and uses declares that an API consumes a service.

7. An automatic module name is generated if one is not supplied. Which of the following JAR filenames and generated automatic module name pairs are correct? (Choose all that apply.)
- A. emily-1.0.0.jar and emily
 - B. emily-1.0.0-SNAPSHOT.jar and emily
 - C. emily_the_cat-1.0.0.jar and emily_the_cat
 - D. emily_the_cat-1.0.0.jar and emily-the-cat
 - E. emily\$.jar and emily
 - F. emily\$.jar and emily.
 - G. emily\$.jar and emily..

Explanation: Any version information at the end of the JAR filename is removed, making options A and B correct. Underscores () are turned into dots (.). The special characters like a dollar sign (\$) are also turned into dots. However, adjacent dots are merged, and leading/trailing dots are removed.

8. Which of the following statements are true? (Choose all that apply.)

- A. Modules with cyclic dependencies will not compile.
- B. Packages with a cyclic dependency will not compile.
- C. A cyclic dependency always involves exactly two modules.
- D. A cyclic dependency always involves at least two requires statements.
- E. An unnamed module can be involved in a cyclic dependency with an automatic module.

Explanation: A cyclic dependency is when a module graph forms a circle. Option A is correct because the Java Platform Module System does not allow cyclic dependencies between modules. No such restriction exists for packages, making option B incorrect. A cyclic dependency can involve two or more modules that require each other, making option D correct, while option C is incorrect. Finally, option E is incorrect because unnamed modules cannot be referenced from an automatic module.

9. Suppose you are creating a service provider that contains the following class. Which line of code needs to be in your module-info.java?

```
package dragon;  
import magic.*;  
public class Dragon implements Magic {  
    public String getPower() {  
        return "breathe fire";  
    }  
}
```

- A. provides dragon.Dragon by magic.Magic;
- B. provides dragon.Dragon using magic.Magic;
- C. provides dragon.Dragon with magic.Magic;
- D. provides magic.Magic by dragon.Dragon;
- E. provides magic.Magic using dragon.Dragon;
- F. provides magic.Magic with dragon.Dragon;

Explanation: The provides directive takes the interface name first and the implementing class name second and also uses with

10. What is true of a module containing a file named module-info.java with the following contents? (Choose all that apply.)

```
module com.food.supplier {}
```

- A. All packages inside the module are automatically exported.
- B. No packages inside the module are automatically exported.
- C. A main method inside the module can be run.
- D. A main method inside the module cannot be run since the class is not exposed.
- E. The module-info.java file contains a compiler error.
- F. The module-info.java filename is incorrect.

Explanation: Packages inside a module are not exported by default, making option B correct and option A incorrect. Exporting is necessary for other code to use the packages; it is not necessary to call the main() method at the command line, making option C correct and option D incorrect. The module-info.java file has the correct name and compiles, making options E and F incorrect.

11. Suppose module `puppy` depends on module `dog` and module `dog` depends on module `animal`. Which lines allow module `puppy` to access the `animal.behavior` package in module `animal`? (Choose all that apply.)

```
module animal {
    exports animal.behavior;
}
module dog {
    _____ animal; // line S
}
module puppy {
    _____ dog;      // line T
}
```

- A. require on line S
- B. require on line T
- C. requires on line S
- D. requires on line T
- E. require transitive on line S
- F. require transitive on line T
- G. requires transitive on line S
- H. requires transitive on line T

Explanation: Because a simple **requires** only gives *dog* access to *animal*, but it doesn't *re-export* that access. This means **puppy**—which depends only on *dog*—cannot see the `animal.behavior` package unless *dog* declares **requires transitive** `animal`; which makes *animal*'s exports visible to any module that depends on *dog*.

12. Which of the following modules are provided by the JDK? (Choose all that apply.)

- A. `java.base`
- B. `java.desktop`
- C. `java.logging`
- D. `java.util`
- E. `jdk.base`
- F. `jdk.compiler`
- G. `jdk.xerces`

13. Which of the following compiles and is equivalent to this loop?

A.

```
List<Unicorn> all = new ArrayList<>();
for (Unicorn current : ServiceLoader.load(Unicorn.class))
    all.add(current);
```

B.

```
List<Unicorn> all = ServiceLoader.load(Unicorn.class)
    .getStream()
    .toList();
```

C.

```
List<Unicorn> all = ServiceLoader.load(Unicorn.class)
    .stream()
    .toList();
```

D.

```
List<Unicorn> all = ServiceLoader.load(Unicorn.class)
    .getStream()
    .map(Provider::get)
    .toList();
```

E.

```
List<Unicorn> all = ServiceLoader.load(Unicorn.class)
    .stream()
    .map(Provider::get)
    .toList();
```

F. None of the above

Explanation: There is no `getStream()` method on a `ServiceLoader`, making options A and C incorrect. Option B does not compile because the `stream()` method returns a list of `Provider` interfaces and needs to be converted to the `Unicorn` interface we are interested in.

14. Which of the following is a legal command to run a modular program where `n` is the module name and `c` is the fully qualified class name?

- A. `java --module-path x -m n.c`
- B. `java --module-path x -p n.c`
- C. `java --module-path x-x -m n/c`
- D. `java --module-path x -p n/c`
- E. `java --module-path x-x -m n-c`
- F. `java --module-path x -p n-c`
- G. None of the above

Explanation: The module name and class name are separated with a slash, making option C the answer. Note that `x-x` is legal because the module path is a folder name, so dashes are allowed.

15. For a top-down migration, all modules other than named modules are _____ modules and are on the _____.

- A. automatic, classpath
- B. automatic, module path
- C. unnamed, classpath
- D. unnamed, module path
- E. None of the above

16. Suppose you have separate modules for a service provider interface, service provider, service locator, and consumer. If you add a second service provider module, how many of the existing modules do you need to recompile?

- A. Zero
- B. One
- C. Two
- D. Three
- E. Four

Explanation: Since this is a new module, you need to compile it. However, none of the existing modules needs to be recompiled, making option A correct. The service locator will see the new service provider simply by having that new service provider on the module path.

17. Suppose we have a JAR file named `cat-1.2.3-RC1.jar`, and `Automatic-Module-Name` in the `MANIFEST.MF` is set to `dog`. What should an unnamed module referencing this automatic module include in `module-info.java`?
- A. `requires cat;`
 - B. `requires cat.RC;`
 - C. `requires cat-RC;`
 - D. `requires dog;`
 - E. None of the above

Explanation: Trick question! An unnamed module doesn't use a `module-info.java` file.

18. Two commands create artifacts that include smaller versions of the JDK. Which are used to create an `.exe` file and a directory, respectively?
- A. `jimage` and `jlink`
 - B. `jimage` and `jpackage`
 - C. `jlink` and `jimage`
 - D. `jlink` and `jpackage`
 - E. `jpackage` and `jimage`
 - F. `jpackage` and `jlink`

Explanation: The `jpackage` command creates self-contained application such as an `.exe`. The `jlink` command creates a directory with a smaller Java runtime containing just what is needed. The `jimage` command is used to inspect a Java image file.

19. Which is a true statement about the following module?

```
class dragon {  
    exports com.dragon.fire;  
    exports com.dragon.scales to castle;  
}
```

- A. All modules can reference the `com.dragon.fire` package.
- B. All modules can reference the `com.dragon.scales` package.
- C. Only the `castle` module can reference the `com.dragon.fire` package.
- D. Only the `castle` module can reference the `com.dragon.scales` package.
- E. None of the above.

20. Which would you expect to see when describing any module?

- A. requires java.base mandated
- B. requires java.core mandated
- C. requires java.lang mandated
- D. requires mandated java.base
- E. requires mandated java.core
- F. requires mandated java.lang
- G. None of the above

Explanation: When running java with the -d option, all the required modules are listed. Additionally, the java.base module is listed since it is included automatically. The line ends with mandated, making option A correct.

21. Suppose you have separate modules for a service provider interface, service provider, service locator, and consumer. Which module(s) need to specify a requires directive on the service provider?

- A. Service locator
- B. Service provider interface
- C. Consumer
- D. Consumer and service locator
- E. Consumer and service provider
- F. Service locator and service provider interface
- G. Consumer, service locator, and service provider interface
- H. None of the above

Explanation: This question is tricky. The service locator must have a uses directive, but that is on the service provider interface. No modules need to specify requires on the service provider since that is the implementation.

22. Which are true statements? (Choose all that apply.)

- A. An automatic module exports all packages to named modules.
- B. An automatic module exports only the specified packages to named modules.
- C. An automatic module exports no packages to named modules.
- D. An unnamed module exports only the named packages to named modules.
- E. An unnamed module exports all packages to named modules.
- F. An unnamed module exports no packages to named modules.

Explanation: An automatic module exports all packages, making option A correct. An unnamed module is not available to any modules on the module path. Therefore, it doesn't export any packages, and option F is correct.

23. Which is the first line to contain a compiler error?

```
1: module snake {  
2:     exports com.snake.tail;  
3:     exports com.snake.fangs to bird;  
4:     requires skin;  
5:     requires transitive skin;  
6: }
```

- A. Line 1.
- B. Line 2.
- C. Line 3.

D. Line 4.

E. Line 5.

F. The code does not contain any compiler errors.

Explanation: The module name is valid, as are the exports statements. Lines 4 and 5 are tricky because each is valid independently. However, the same module name is not allowed to be used in two requires statements. The second one fails to compile on line 5.

24. Which is a true statement about a package in a JAR on the classpath containing a module-info.java file?

A. It is possible to make the package available to all other modules on the classpath.

B. It is possible to make the package available to all other modules on the module path.

C. It is possible to make the package available to exactly one other specific module on the classpath.

D. It is possible to make the package available to exactly one other specific module on the module path.

E. It is possible to make sure the package is not available to any other modules on the classpath.

Explanation: Since the JAR is on the classpath, it is treated as a regular unnamed module even though it has a module-info.java file inside. Remember from learning about top-down migration that modules on the module path are not allowed to refer to the classpath, making options B and D incorrect. The classpath does not have a facility to restrict packages, making option A correct and options C and E incorrect.

25. Suppose you have separate modules for a service provider interface, service provider, service locator, and consumer. Which statements are true about the directives you need to specify? (Choose all that apply.)

A. The consumer must use the requires directive.

B. The consumer must use the uses directive.

C. The service locator must use the requires directive.

D. The service locator must use the uses directive.

E. None of the above.

Explanation: Options A and C are correct because both the consumer and the service locator depend on the service provider interface. Additionally, option D is correct because the service locator must specify that it uses the service provider interface to look it up.